

THE Hybrid Engine — Final Evidence Dossier and Implementation Lock

Project status: final close-out document

Research date: 16 August 2026

Purpose: settle the evidence questions, define the product-safe rules, and close the progression/deload research loop without pretending that engineering heuristics are experimentally proven constants.

Executive verdict

This document closes the question. THE Hybrid Engine should be built as a transparent, local-first, deterministic adaptive training system. It should observe the athlete, estimate state with explicit uncertainty, preserve the purpose of the planned session where safe, and make bounded changes that are easy to audit. It should not be sold to the codebase or to the athlete as an artificial intelligence that has discovered the one true training formula.

The central research result is negative but useful: no convincing body of evidence was found that directly compares a progression rule of approximately 2.5% with 5% and 10% while holding exercise, population, trigger, volume, and context constant. The commonly repeated 2–10% recommendation is real as an ACSM position-stand recommendation, but the range is a practical prescription band with Category B support rather than an optimisation result. Plotkin et al. compared load progression with repetition progression; it did not test increment size. Hostler et al. showed that very small increments can work over eight weeks in upper-body exercises, but did not establish a universal optimum or a long-term adherence advantage. Buskard et al. found no clear strength or functional-performance winner among several progression methods in older adults. The evidence therefore supports a controller shape, not a magic constant.

The implementation decision is consequently firm but honestly labelled. The engine may use a default progression target of 2.5% of the last stable opening load, with equipment-aware rounding and a repetition/RPE fallback. That is an engineering default chosen conservatively inside the ACSM band. It is not a finding that 2.5% beats 5% or 10%. The engine may use a default 5% reduction from the last successful anchor after repeated, comparable deterioration. That is a product heuristic. It is not a validated physiological threshold, and the rule must not blindly compound a within-session failure correction.

The strongest product decision is more important than either percentage: do not let a single noisy observation control the whole programme. A progression decision should be based on comparable exposure, observed performance, exercise familiarity, technique, pain/symptom status, and the presence or absence of corroborating fatigue. A low HRV value, a bad night of sleep, or a subjective “flat” feeling may lower confidence, but none should independently authorise a heavier load or diagnose recovery. Pain must be represented separately from exertional fatigue. A training gap must enter a calibration state rather than trigger an invented percentage equation. Missing data must reduce certainty, not silently become normal data.

That is the final position: build the adaptive layer, but make its evidence status visible. The system’s intelligence should be its restraint, auditability, and ability to choose the next useful action when the evidence is incomplete.

How to read this dossier

Every recommendation is tagged conceptually in one of four ways. **Direct evidence** means that a study tested the mechanism or population reasonably close to the proposed claim. **Adjacent evidence** means that the study informs the mechanism but does not validate the exact app rule. **Coaching precedent** means that a named system or experienced practice uses a similar idea; it can guide design but cannot be presented as causal proof. **Product-design convention** means that the number or state exists because the product needs a deterministic and conservative behaviour where the literature has not supplied one.

The dossier contains two layers. The first is the final interpretation and implementation lock written for the project. The second is the inherited evidence bundle and design record preserved as appendices. The appendices are included so that the decision can be audited later without reopening the same research from memory. They are not a claim that every sentence in an inherited note has equal evidential weight; the reconciliation chapters above take precedence where wording or certainty has changed.

1. The exact question the project needed answered

The practical problem was not whether progressive overload exists. It does. The problem was how a software engine should decide what to do after a successful or unsuccessful exposure when the user trains strength and conditioning in the same week, reports RPE/RIR, may have wearable recovery data, has finite equipment increments, and needs a plan that remains usable on bad real-world days.

That problem contains several questions which are often collapsed into one:

1. Does increasing external load generally support strength and hypertrophy?
2. Is increasing load better than increasing repetitions when the athlete is already working hard?
3. Does the size of the load increment matter, and is there a tested optimum?
4. Should the next increment be a fixed number of kilograms or a relative percentage?
5. How much evidence should be required before a progression change?
6. What should happen after one miss, repeated misses, or a whole-session decline?
7. When should a deload reduce load, volume, frequency, effort, or some combination?
8. How should conditioning fatigue, sleep, HRV, pain, illness, and a training gap alter the decision?
9. How can an app communicate uncertainty without turning it into a decorative confidence score?

The evidence is much better for the first two questions than for the third through ninth. The first two are training questions. The later questions are control-system questions wrapped around training. A study showing that a programme improves squat strength does not automatically tell us how many consecutive misses a mobile app should tolerate, what a plate-limited jump should be, or whether the app should reduce the next session by 5% or 7.5%. Those are separate claims with separate evidential burdens.

The project went wrong whenever it treated a familiar coaching number as though it were a measured natural constant. “Two consecutive misses,” “three sessions,” “5% deload,” and “10% lower-body progression” may each be reasonable starting conventions. None becomes scientifically established merely because several programmes use it. The final engine therefore records the provenance of every decision: evidence, precedent, heuristic, or safety policy.

2. Evidence method and standards of honesty

The search was organised around mechanisms rather than slogans. The core searches targeted the ACSM progression recommendation, the source cited behind it, direct comparisons of load and repetition progression, direct or near-direct evidence on microloading, autoregulated progression methods, deload studies and surveys, reliability and measurement error, HRV-guided resistance training, detraining and return to training, pain-monitoring models, and uncertainty communication. The search was then extended to named coaching and product systems because those systems are relevant precedents for how a practical feedback loop can be presented, even though their internal algorithms are not independent scientific validation.

The evidence was assessed with a deliberately conservative hierarchy. A position stand can be authoritative and still contain a recommendation whose precise dose has only limited experimental support. A randomised trial can be rigorous and still answer a narrower question than the product wants to ask. A meta-analysis can produce a pooled effect while inheriting heterogeneity in load, exercise, participants, and outcome measurement. A survey can describe what experienced athletes do while remaining unable to show that the practice caused their outcomes. A product's help page can prove what the creator says the algorithm does, but not that the algorithm is optimal or generalisable.

The report therefore avoids three common errors. First, it does not convert absence of evidence into evidence of absence. “No direct trial found” means the reviewed search did not find one that supports the exact claim; it does not prove that no inaccessible dissertation or unpublished dataset exists. Second, it does not confuse a statistically significant result with a practically useful software rule. A tiny average difference may be statistically detectable and still not justify a different user-facing behaviour. Third, it does not hide population mismatch. Findings in untrained young men, older adults, trained powerlifters, or clinical tendinopathy patients are not automatically transferable to an unsupervised mixed-modality recreational user.

The engineering standard is therefore: every rule must state what it is allowed to claim, what it cannot claim, what inputs it requires, and what action it takes when its inputs are missing or contradictory. A rule that cannot explain itself is not ready for the engine, even if its number sounds familiar.

3. What ACSM supports—and what it does not

The 2009 ACSM position stand, “Progression Models in Resistance Training for Healthy Adults,” gives the familiar recommendation that when an athlete can perform one to two repetitions above the desired workload on two consecutive training sessions, the load may be increased by 2–10%. The paper also describes the lower end as more appropriate to smaller muscle-mass exercises and the higher end as more appropriate to larger, multi-joint exercises. This is directly relevant to the project because it validates the general direction of a relative, exercise-sensitive progression policy.

It does not validate a software constant. The sentence does not say that 2.5% is optimal for a bench press, that 5% is optimal for a squat, or that 10% is optimal for a deadlift. It does not provide a plate-rounding algorithm, a maximum acceptable rounded jump, or a fallback rule for when equipment resolution makes the target impossible. It does not establish that two sessions is superior to one or three. It is a practical prescription within an evidence category, not a dose-response experiment.

The source lineage matters. The 2009 position stand cites Feigenbaum and Pollock's prescription paper for the progression recommendation. That source is a prescription and review contribution, not a randomised comparison of 2%, 5%, and 10% jumps. The recommendation is therefore downstream of accumulated exercise-prescription practice and expert synthesis. That does not make it useless. It does mean the code documentation must say "ACSM-supported operating range" rather than "trial-proven optimal percentage."

The 2026 ACSM position stand strengthens the broader design philosophy. It synthesised 137 systematic reviews representing more than 30,000 participants and emphasised consistency, individualisation, goal-specific loading, and the fact that complicated techniques are not consistently necessary for the average healthy adult. It does not replace the missing direct increment-size experiment. Its value to this project is that it pushes the engine away from rigid one-size-fits-all rules and toward a system that is effective because the athlete can actually sustain it.

The product interpretation is precise:

- ACSM supports a progression band and a performance-based trigger.
- ACSM supports using smaller practical increases for smaller exercises and larger increases for larger exercises.
- ACSM does not supply the exact software constants.
- The 2026 update supports individualisation and adherence as first-order design requirements.
- The engine must preserve the uncertainty rather than laundering the recommendation into a false equation.

4. What Plotkin actually tested

Plotkin et al. compared two progression strategies over an eight-week lower-body resistance-training cycle. One group progressed by increasing load while keeping repetitions in an 8–12 repetition range. The other group kept the initial load and progressed repetitions. The study used trained young adults, lower-body exercises, two sessions per week, four sets per exercise, and sets performed to momentary concentric failure. This is a useful study because it asks whether the athlete must always add weight to keep progressing.

The answer was no. Both groups improved. Muscle-size changes were broadly similar in most measured regions. The load-progression group showed a small, uncertain advantage on the squat strength outcome, while one hypertrophy outcome modestly favoured the repetition-progression group. The intervals crossed practically unimportant values and, depending on the outcome, included zero. The conclusion was that both load and repetition progression appeared viable over the eight-week cycle.

The limitation is exactly the point for this project: Plotkin did not randomise athletes to 2.5%, 5%, and 10% increments. The load group was told to increase load while staying in the target range, but the paper does not turn that into a universal increment formula. The study also does not isolate load-increment size from the broader choice of progression axis. A repetition-progression group can accumulate a different external volume-load than a load-progression group; that is part of the intervention, not a nuisance that disappears.

Plotkin therefore supports a dual-axis engine. If a load jump is too large, the athlete can still make legitimate progress through repetitions, execution quality, reduced RPE at the same load, or another stated lever. The paper supports the existence of a pressure-release valve. It does not support the exact pressure setting.

This distinction protects the product from a common implementation mistake: if the engine treats a successful 10-repetition set as proof that the next load must increase, it will force a plate jump even when the equipment jump is disproportionate. A better engine says: “The athlete has demonstrated progress. The next available load jump is 10% and exceeds this movement’s configured jump ceiling. Hold the load, progress the repetition target or confirm with another comparable exposure.” That behaviour is consistent with Plotkin’s result and honest about what the paper did not test.

5. Direct evidence on increment size

The direct evidence is sparse. Hostler et al. studied the effectiveness of very small increments in two upper-body exercises, the bench press and triceps press. The small-increment protocol used changes of approximately 0.22 kg after completing seven to eight repetitions and approximately 0.44 kg after completing nine or more. The traditional comparison used substantially larger exercise-specific increments. Both approaches improved strength over eight weeks. The authors concluded that small-increment progression appeared as effective in the short term, while preliminary extended observations suggested that the traditional group might eventually increase resistance more effectively.

This study matters because it defeats the assumption that a small increment is automatically too small to work. It does not prove that the smallest possible increment is best. It is small, upper-body-specific, short, and not a clean answer to adherence or long-term stalling. It also does not tell us how to compare a 0.5 kg increase on a 25 kg press with a 5 kg increase on a 180 kg deadlift.

Buskard et al. compared four approaches to load progression in healthy older adults across an eleven-week structured resistance-training period. The groups differed in how load increases were determined, including repetition maximum, percentage-based, RPE, and repetitions-in-reserve approaches. No significant between-group differences emerged for the strength or functional-performance outcomes. The RPE group found the training more tolerable and enjoyable. This is not a direct increment-size trial, but it supports the principle that multiple reasonable progression policies can produce similar outcomes and that tolerability is not a trivial product variable.

The direct answer to the project’s main question is therefore:

> No convincing experiment was located that randomised comparable trainees to approximately 2.5%, 5%, and 10% progression increments while holding exercise, target repetitions, progression trigger, volume, frequency, and context constant.

That negative finding is load-bearing. It means the final number must be treated as a tunable starting parameter. The correct response is not to give up and use a flat 2.5 kg rule. It is to make the percentage visible, constrain it by equipment, provide a repetition fallback, and collect data that can later test the engine’s own behaviour.

6. Why relative progression is the least-wrong default

A flat 2.5 kg change has radically different meaning at different loads. At 25 kg it is a 10% increase. At 50 kg it is 5%. At 100 kg it is 2.5%. At 180 kg it is about 1.4%. Therefore a global “add 2.5 kg when successful” rule silently assigns an aggressive progression to light exercises and a conservative progression to heavy ones.

A percentage rule is not physiologically perfect. A 5% external-load increase is not guaranteed to create a 5% increase in internal stress because leverage, range of motion, technique, machine geometry, fatigue, and

bodyweight alter the task. Nonetheless, relative loading is a more coherent control variable than a fixed kilogram constant when one engine must serve different exercises and users.

The correct algorithm is a two-stage policy. First, calculate a desired relative change. Second, ask whether the available equipment can express that change without creating an excessive actual jump. The engine must store both values:

```
target_delta = anchor_load × progression_percent
actual_delta = selected_load - anchor_load
actual_jump_percent = actual_delta / anchor_load
```

If the smallest available increment at or above the target creates an actual jump below the movement's ceiling, use it. If it exceeds the ceiling, do not pretend that the result equals the target. Hold the load and use a permitted fallback: add a repetition, improve the achieved RPE margin, add a set only if the programme allows that lever, or schedule a later confirmation exposure. The fallback is not a failure to progress. It is the correct response to coarse hardware.

The engine should not always choose the smallest available increment below the target either. A 0.5% increase may be too small to matter or may be unavailable. The selection policy should be deterministic and configurable. A safe default is to choose the smallest available jump that reaches the desired target without exceeding the exercise cap. When no such jump exists, hold and progress another axis. For a stable lower-body compound with 180 kg as the anchor, a 2.5 kg jump is a 1.39% increase and is acceptable even if it is below a 2.5% target. For a 25 kg upper-body movement, a 2.5 kg jump is 10%; the engine should hold or microload rather than force it.

The cap is also a heuristic, not a scientific law. The recommended initial values are 5% for small-load or upper-body movements and 7.5% for large lower-body movements when equipment leaves no better choice. Those caps should be versioned and tested. They are a safety-oriented translation of the ACSM band and plate realities, not a claim about optimal adaptation.

7. The progression controller

The controller should not make a decision from a single “success” boolean. A set can be completed because the athlete overshot the intended RPE, because the target was too easy, because the exercise was changed, or because the user entered a value without actually performing the set. The engine needs a validity layer before a progression layer.

7.1 Exposure validity

An exposure is comparable only if the exercise identity, meaningful range of motion, target mode, load units, set structure, and general technique are sufficiently stable. A changed machine, exercise substitution, unusual pain, or a major change in rest interval may still be useful data, but it should not automatically count as equivalent evidence for a load jump.

Each exposure should record:

- exercise and variation identity;
- intended load and actual load;
- target repetitions or time;
- completed repetitions or time;
- set count and order;

- reported RPE or RIR;
- technique or range-of-motion flag;
- pain/symptom flag;
- rest interval where it matters;
- preceding conditioning load and recent training gap;
- data completeness and source;
- whether the athlete used an approved substitution.

7.2 Success classification

A valid success is not merely “all boxes ticked.” The first implementation should classify an exposure as `successful`, `successful\_but\_uncertain`, `held`, `missed`, `pain\_blocked`, `invalid\_for\_progression`, or `incomplete`. A successful exposure may be uncertain if RPE is missing, technique changed, equipment changed, or conditioning fatigue was unusually high. A valid completion under a lower dose can be a successful recovery-mode exposure without authorising progression.

7.3 Promotion gate

The base gate is two comparable successful exposures, because repeated confirmation is a practical noise filter and aligns with the broad repeated-performance logic in ACSM. The number is explicitly a product heuristic. The engine should allow the configuration to change, and telemetry should record what would have happened under one, two, or three exposures during validation replay.

When the gate is met, promote only one lever. Do not increase load, repetitions, set count, and conditioning intensity simultaneously. One-lever changes preserve causal clarity and reduce the risk that a temporary good day becomes an overlarge training dose.

7.4 Hold state

The hold state is a first-class outcome, not a failure. Use it when the athlete completes the work but the evidence is too noisy to justify a change; when the next load jump is too coarse; when readiness and performance conflict; or when missing information matters to the decision. The user-facing explanation should say what was preserved and what evidence is missing.

7.5 Repetition fallback

When load cannot rise safely, use repetition progression only inside the exercise’s prescribed range and fatigue budget. Do not turn a strength-focused set into an endless AMRAP merely because the plates are coarse. A fallback can be as small as one additional repetition on the next exposure, a reduced RPE target at the same load, or a second confirmation at the same load. The engine must retain the session purpose.

8. RPE and RIR: useful instruments, not truth oracles

RPE/RIR is valuable because a percentage based on an outdated maximum can be wrong on a particular day, while the athlete’s actual effort and completion can provide immediate information. Studies in trained lifters show meaningful relationships between perceived effort, velocity, and proximity to failure. But perception has error. Users are less accurate when many repetitions remain, the exercise is unfamiliar, the set is technically unstable, or the athlete has not been familiarised with the scale.

The engine should therefore use RPE/RIR conditionally. For familiar exercises in trained users and sets close enough to failure for the scale to be meaningful, it can inform within-session adjustment and progression confidence. For a novel exercise, a high-repetition set, pain-affected movement, or an RPE entry that conflicts with the observed repetitions and technique, it should be advisory only.

The app should learn user-specific calibration rather than assuming that every athlete's RPE 8 means the same thing. Calibration does not require a complicated machine-learning model. The engine can calculate simple bias summaries: how often an athlete's reported RPE 8 was followed by a large overshoot, how often an RPE 9 ended early, and how stable the relationship is for a specific exercise. These summaries should be shown as evidence quality, not as a permanent personality label.

RPE/RIR should not be used to make the claim that autoregulation is always superior to fixed percentages. Trials and reviews are mixed. In some comparisons, the autoregulated group achieved a different actual intensity, which makes it difficult to isolate the decision method. Other studies found similar gains. The correct product claim is that RPE/RIR is an additional input that can help the engine remain responsive when a fixed percentage is stale.

Within-session correction must also be separated from cross-session programming. If the athlete misses at 100 kg and the session corrects to 94 kg, 94 kg is the effective load for that session. It is not automatically the new long-term anchor. The engine should retain `session\_opening\_load`, `effective\_load`, and `last\_successful\_anchor\_load` separately. Otherwise a failed session can cause an invisible compound reduction when the next cross-session deload is calculated from an already reduced number.

9. Deloads: common, plausible, and under-tested

Deloading is one of the clearest areas where coaching practice outruns direct evidence. A 2024 survey of 246 competitive strength and physique athletes found that deloads were commonly used for about 6.4 days every 5.6 weeks. Athletes typically reduced volume through sets and repetitions, often reduced load and effort, and generally kept exercise selection and frequency similar. This describes what experienced athletes report doing. It does not prove that every athlete benefits from a deload on that schedule.

The 2023 Delphi consensus provides useful terminology and a practical framework, but consensus is not a dose-response trial. The 2024 Coleman study found that a one-week midpoint deload involving training cessation could negatively affect some lower-body strength outcomes compared with continuous training, even when hypertrophy outcomes were similar. The 2026 Pancar within-subject study in 19 untrained young men found that planned reductions in weekly sets and frequency at weeks 4 and 8 did not hinder hypertrophy or 10RM strength-endurance over eight weeks. These studies are not contradictory once their protocols are respected: they tested different populations, goals, exercise selections, deload definitions, and outcome measures.

The lesson is not “deloads are bad” or “deloads are mandatory.” The lesson is that deload is a family of interventions. Reducing sets while keeping some exposure is not the same as complete cessation. A reactive deload after corroborated decline is not the same as a pre-planned week inserted on a calendar. A strength test is not the same as hypertrophy or local endurance. The app must preserve those distinctions.

The final reactive rule is:

1. One comparable miss: hold, inspect context, and do not automatically deload.
2. Repeated comparable misses or corroborated deterioration: enter a conservative reduction state.

3. Isolated movement failure with normal session performance: reduce the affected load or repeat the exposure; keep overall volume unless there is reason not to.
4. Several related movements failing, or performance declining across the session: reduce one principal stress lever, commonly volume or intensity, and reassess.
5. System-wide poor performance with high conditioning fatigue, illness, or poor readiness: reduce volume and/or intensity according to the configured safety policy, but do not stack unexplained reductions.

If a default number is required for the first release, use 5% from the last successful opening anchor. Label it ``product_heuristic``. Escalate toward 7.5–10% only when broader evidence supports it, and consider reducing volume rather than merely lowering load when the problem appears systemic. Never calculate a nominal “5% deload” from a load that has already been walked down by a within-session miss unless the product explicitly wants the compound effect and displays it.

10. Why “two consecutive misses” is not science—but can still be useful

No direct trial was located that compares one, two, and three consecutive failures as trigger thresholds for load reduction. The threshold is therefore not physiology. It is a control-system debounce choice. A single missed set contains many possible explanations: sleep, warm-up, technique, rest, equipment, illness, work stress, preceding conditioning, or a bad RPE estimate. Requiring repeated comparable evidence reduces the chance that the engine overreacts to noise.

The word “comparable” carries more weight than the number two. Two failures after a heavy conditioning session and a poor night’s sleep are not the same evidence as two failures under normal conditions. Two misses on a different machine are not the same as two misses on the same exercise. A failed set because of pain should not enter the same counter as a non-painful strength miss. A missing set should not be treated as a completed failure.

The engine should maintain separate counters or evidence states:

- ``performance_miss_streak`` for comparable non-painful misses;
- ``pain_block_state`` for symptom-related interruption;
- ``systemic_decline_evidence`` for broad cross-movement deterioration;
- ``data_quality_conflict`` for contradictory or incomplete records;
- ``calibration_state`` for a return after a meaningful gap.

This is superior to a single “failed sessions” integer because it prevents unrelated events from being combined into a fake pattern. If the product UI wants to show “two misses,” it should be able to explain which two exposures counted and why.

11. Hybrid training changes the meaning of a miss

A strength-only programme can interpret a missed squat mainly through the recent strength plan. A hybrid athlete may have performed intervals, long Zone 2, a physically demanding job, poor sleep, or another lower-body session nearby. The same failed squat may represent local strength limitation, systemic fatigue, glycogen availability, dehydration, accumulated conditioning load, or a technical issue.

The engine should not pretend that a single composite readiness score solves this. It should preserve modality-specific evidence and use a transparent priority order. Safety and pain come first. Actual performance and technique come next. Session purpose follows. Recent comparable strength evidence matters. Wearable or HRV data is advisory context.

The programming layer should store conditioning exposure in a way that permits interpretation: modality, duration, intensity, heart-rate zones, interval count, perceived effort, and time since the session. It should not silently convert a hard row into “leg fatigue” or use a cycling metric as though it were a squat readiness measure. The system can infer that a hard lower-body interval session may be relevant context, but it must label the inference.

The most useful hybrid behaviour is not to cancel every strength session after a hard conditioning day. It is to preserve the primary purpose with a controlled adjustment when multiple signals converge. A normal strength performance despite a low wearable recovery score should not be demoted automatically. Poor performance plus poor perceived recovery plus unusually high preceding conditioning load is stronger evidence for reducing the day’s dose. This multi-signal logic remains an engineering policy; no reviewed study validates the exact fusion algorithm for this product.

12. HRV and wearables: advisory context only

HRV-guided training has more support in endurance distribution than in strength-load prescription. Individualised endurance studies show that using HRV can alter the distribution of hard and easy training and sometimes produce useful outcomes. Strength studies are smaller and mixed. HRV is influenced by measurement protocol, sleep, illness, stress, posture, breathing, alcohol, and device quality. It is not a direct measure of local muscle capacity, tendon status, pain, technical readiness, or medical safety.

The product rule is therefore simple: HRV can lower confidence or support a conservative option when it agrees with performance and context, but HRV alone must not increase load, authorise high-risk work, diagnose overtraining, or override pain. The engine should normalise HRV within the athlete and measurement protocol, retain the raw value and timestamp, and mark stale or missing data. Cross-device values must not be treated as directly interchangeable without a declared normalisation policy.

The UI should avoid an authoritative “recovery score says train hard” message. A better statement is: “Today’s wearable recovery signal is lower than your usual range. Your recent comparable performance is normal and no pain was reported, so the planned session remains available, but progression is held until performance provides stronger evidence.” That statement tells the truth about both the signal and its limits.

13. Return after a layoff

Detraining evidence does not provide a universal time-off-to-load-reduction equation. Short breaks can preserve much of the strength and muscle adaptation, but performance expression, exercise skill, conditioning, connective tissue tolerance, and confidence may change at different rates. A trained powerlifter returning after two weeks is not the same case as a beginner returning after three months, and a barbell lift is not the same as a running interval.

The engine should represent a return as calibration. It should reduce complexity, avoid unnecessary failure exposure, and observe current technique, RPE/RIR, completion, and symptoms. The first session should gather information without treating the athlete as untrained or forcing a fixed percentage reduction. The

exact amount of load or volume reduction can be configured by modality and risk profile, but the product documentation must state that it is a conservative policy rather than a validated equation.

The calibration state should expire based on successful comparable exposures, not merely time. A user who returns and completes two stable exposures with normal technique may leave calibration sooner than a user who returns with pain, repeated misses, or contradictory data. The system must not silently promote a calibration session to a normal progression anchor without recording that it was a calibration exposure.

14. Pain is not fatigue

Pain-monitoring evidence can support continued activity in specific rehabilitation contexts. Silbernagel's Achilles tendinopathy work is an important example of a condition-specific protocol with pain and next-morning monitoring. It cannot be converted into a universal "5/10 pain is safe" rule for every user and exercise. A consumer training app usually does not know the diagnosis, tissue state, red flags, or rehabilitation plan.

Pain, exertional fatigue, breathlessness, soreness, technical breakdown, and fear are different signals. The data model must keep them separate. A pain event should stop, modify, or substitute the affected movement according to safety policy, record location and quality, and block automatic progression for that movement or pattern until the symptom state is updated. The engine should offer a safe alternative where appropriate, but it must not diagnose or reassure beyond its evidence.

The human-logic behaviour is bounded: distinguish a difficult set from a painful set; preserve the session purpose through a safe substitution or reduced dose when possible; do not let motivation override a hard safety state; and make the next check explicit. This is a product safety policy informed by clinical evidence boundaries, not a universal treatment protocol.

15. Trends, reliability, and the minimum data problem

No universal number of sessions was found that makes an e1RM or volume trend trustworthy. The correct number depends on measurement error, exercise consistency, sampling frequency, missingness, and the magnitude of change that matters to the decision. Three comparable exposures may be enough to decide "hold and gather more data" but not enough to claim a long-term trend. Ten non-comparable exposures may be less informative than three standardised ones.

The engine should use reliability concepts rather than a magical minimum. Store the raw observations, standardise what can be standardised, estimate within-user variability when enough repeated data exists, and compare observed change with the estimated error band. Typical error, standard error of measurement, limits of agreement, and minimum detectable change are more informative than a raw correlation alone.

For e1RM, store the formula version, load, repetitions, RPE/RIR, exercise, technique flags, and whether the set was valid for trend use. Do not mix a rep-max estimate from a machine with a barbell estimate without an explicit modality boundary. Smoothing can reduce noise, but a three-point moving average is not a universal law. It is a declared filter. The engine should show how many observations were used and whether the window contains gaps or exercise changes.

The practical product states are:

- ``insufficient_evidence``: not enough comparable data;

- `uncertain\_change`: observed change does not exceed estimated noise;
- `stable`: no meaningful change detected;
- `probable\_improvement`: change exceeds current error estimate with adequate data;
- `probable\_decline`: change exceeds current error estimate and is corroborated;
- `conflicted`: signals disagree or protocol changed.

This is more honest than showing “progress +4.3%” without explaining whether the number is a stable trend or one unusually good set.

16. Confidence and human-like logic

A confidence score is meaningful only if it is confidence in a defined decision over a defined horizon. “Confidence 72%” is not useful if the user does not know whether it means the load will be completed, that recovery is adequate, or that hypertrophy will occur. Calibration research also shows that an apparently confident estimate can be poorly matched to real-world correctness.

The engine should therefore communicate evidence state rather than decorative precision. Use statuses such as `approved`, `held`, `reduced`, `repeated`, `substituted`, `calibration`, `blocked`, `needs\_input`, and `needs\_clinical\_review`. Each decision should expose observed signals, estimated state, conflicts, missing data, reason codes, evidence status, and the next recheck point.

The human-logic layer is not an unconstrained language model. It is a bounded policy that behaves like a calm coach: preserve the session purpose, distinguish danger from inconvenience, avoid punishment and make-up debt, and choose the smallest safe change. An AI language layer may translate the deterministic decision into natural language later, but it must not change the decision or invent a rationale.

The priority order is safety gate, pain/symptoms, immediate training quality, session purpose, real-world constraints, recent comparable evidence, and advisory wearable signals. Lower-priority context cannot override a higher-priority safety state. A good readiness score cannot override pain. A low HRV score cannot override strong performance and safe technique by forcing a demotion. A user request to “push through” cannot override a blocked state.

17. Final implementation specification

The following is the locked first-release specification. Numbers marked as heuristics must be versioned and exposed in the engine metadata.

17.1 Core constants

DEFAULT_PROGRESS_TARGET = 0.025	# product heuristic inside ACSM 2-10% band
SMALL_MOVEMENT_JUMP_CAP = 0.05	# product heuristic
LARGE_MOVEMENT_JUMP_CAP = 0.075	# product heuristic
DEFAULT_REACTIVE_DELOAD = 0.05	# product heuristic
ESCALATED_DELOAD_RANGE = 0.075-0.10	# only with corroborated systemic decline
SUCCESS_CONFIRMATION_EXPOSURES = 2	# product heuristic / debounce rule
SINGLE_MISS_ACTION = hold_and_context	# product policy
PAIN_ACTION = block_or_modify	# safety policy
LAYOFF_ACTION = calibration	# product policy
HRV_ROLE = advisory	# evidence boundary

17.2 Anchor and load fields

```
type ExerciseProgressState = {
  exerciseId: string;
  variationId: string;
  lastSuccessfulOpeningLoad: number | null;
  currentOpeningLoad: number | null;
  currentEffectiveLoad: number | null;
  targetIncrementPercent: number;
  actualIncrementPercent: number | null;
  availableLoadStep: number | null;
  successStreak: number;
  comparableMissEvidence: number;
  painBlock: boolean;
  calibration: boolean;
  lastDecision: DecisionStatus;
  engineVersion: string;
};
```

`lastSuccessfulOpeningLoad` is the cross-session anchor. `currentOpeningLoad` is what the user was asked to start with today. `currentEffectiveLoad` is what was actually used after within-session corrections. These must not be collapsed into one field.

17.3 Promotion pseudocode

```
if safety_gate or pain_block:
  return blocked_or_modified

if calibration:
  return calibration_prescription

if not comparable_success:
  if valid_nonpainful_miss:
    record miss evidence
    return hold
  return insufficient_evidence

if success_streak < confirmation_exposures:
  return repeat_current_prescription

target_delta = anchor_load × target_increment_percent
candidate = choose_available_load(anchor_load, target_delta)
actual_jump = (candidate - anchor_load) / anchor_load

if candidate exists and actual_jump <= movement_jump_cap:
  promote load only
else:
  hold load and activate permitted repetition/RPE fallback
```

17.4 Reactive-reduction pseudocode

```
if pain or red_flag:
  do not calculate a normal deload
  enter safety pathway

if one comparable nonpainful miss:
  hold from last successful anchor
  collect context
```

```

if repeated comparable misses and local problem only:
    new_opening = round(anchor_load × (1 - 0.05))
    keep volume unless other evidence says otherwise

if broad decline across related movements:
    choose one primary reduction lever
    prefer volume reduction when fatigue is systemic
    use 0.05–0.075 load reduction if load reduction is selected

if systemic decline, illness, or extreme conditioning stress:
    use conservative mode or recovery mode
    do not stack hidden reductions

```

17.5 Explanation contract

What I noticed: [observed facts].
 What that means today: [bounded interpretation and confidence].
 What we are doing: [specific action and preserved session purpose].
 Why: [reason codes in plain language].
 What would change it: [next check or missing information].

The explanation is generated from structured fields. It is not allowed to invent physiology, diagnose fatigue, or claim that a heuristic is a study result.

18. Worked examples

Example A: 25 kg upper-body movement with coarse plates

The athlete's last successful opening load is 25 kg. The target increment is 2.5%, or 0.625 kg. The gym can only express a 2.5 kg increase, which would be 10%. The movement cap is 5%. The correct result is not 27.5 kg disguised as a 2.5% progression. The engine holds 25 kg, advances the repetition or RPE-quality target within the planned range, and records that equipment resolution prevented the target load change.

Example B: 60 kg lower-body movement

The target increment is 1.5 kg. A 2.5 kg step creates a 4.17% jump, below the configured large-movement cap. The engine can select 62.5 kg, provided the exposure was comparable and the success gate was met. It records target 2.5%, actual 4.17%, and equipment rounding.

Example C: 100 kg movement with a missed set

The session opens at 100 kg. The athlete misses the target, and the within-session rule corrects the effective load to 94 kg. The session record stores opening 100, effective 94, and failed exposure. If repeated comparable misses later trigger a 5% reactive reduction, the next opening load is calculated from the last successful anchor, 100 kg, producing 95 kg before equipment rounding. It is not calculated from 94 kg unless the product explicitly chooses a compounded policy and displays it.

Example D: low HRV but normal performance

Wearable recovery is lower than the athlete's usual range. The athlete completes the planned warm-up and working sets with normal technique, target repetitions, and expected RPE. The engine does not force a demotion or progression. It completes the planned purpose and holds progression if the low HRV creates

uncertainty. The explanation says the wearable signal was advisory and performance was the stronger direct evidence today.

Example E: pain during a press

The athlete reports sharp shoulder pain during the third set. The engine does not treat this as RPE 10. It enters a pain pathway, stops or modifies the movement, records location and symptom detail, and offers a safe substitution only within the configured policy. Automatic progression for the affected movement is blocked pending symptom update. The app does not diagnose the shoulder.

Example F: missed training week

The athlete returns after seven days away. The engine enters calibration for the next exposure, caps effort, and observes current performance. It does not automatically apply a universal 10% reduction. If the user completes the calibration exposure with stable technique and no symptoms, the next session can return toward normal prescription with the calibration evidence recorded.

19. Validation plan for THE Hybrid Engine itself

Because the literature does not identify the exact constants, the app must be designed to learn whether its own policy is useful. This is not permission to claim that a single-user log proves general efficacy. It is a product validation programme.

Phase 1: deterministic replay

Build fixtures in which identical inputs produce identical outputs across browser, coach, offline, and cloud-rehydrated execution. Include every decision state and reason code. A changed engine version must be able to explain why its output changed.

Phase 2: boundary testing

Test loads around the jump cap: 24.9, 25, 25.1, 49.9, 50, 50.1, 99.9, 100, 100.1, and heavy lower-body anchors. Test available increments below, equal to, and above the target. Verify that displayed target percentage and actual percentage cannot be confused.

Phase 3: evidence-quality testing

Replay missing RPE, missing HRV, missing heart rate, changed exercise, changed equipment, incomplete session, pain, illness flag, unusual conditioning, poor sleep, and conflicting user/device reports. The expected behaviour is not always to stop. It is to lower confidence, hold, reduce, substitute, or request the smallest useful clarification according to the state.

Phase 4: historical replay

Replay real completed sessions without changing the historical record. Compare the current rule, a flat-kilogram rule, a 2.5% rule, a 5% rule, and a repetition-fallback rule. Evaluate missed-session frequency, unnecessary load jumps, repeated stalls, total completed work, pain-block violations, and the frequency of

“hold” decisions. The goal is not to crown a single metric. It is to see whether the engine behaves proportionately.

Phase 5: prospective pilot

Run the locked policy prospectively with the user or a small invited group. Log decisions and overrides. Measure whether users understand the reason for a hold, whether they follow safe substitutions, whether they perceive the system as useful, and whether the engine’s decisions remain stable under work stress and hybrid scheduling. Any outcome claims must remain exploratory until a properly designed study exists.

Primary product metrics

- decision determinism;
- percentage of adaptive decisions with complete reason codes;
- rate of silent data coercion, which must be zero;
- rate of progression decisions with a valid comparable exposure;
- rate of pain events that incorrectly enter normal fatigue logic, which must be zero;
- unnecessary progression after a single noisy observation;
- user comprehension of hold and calibration states;
- user override rate and whether overrides occur in blocked states;
- recovery of incomplete sessions after reload or offline use;
- integrity of coach-target versus athlete-result ownership.

20. Product architecture required by the evidence

The evidence conclusion is inseparable from the project’s existing architecture. The athlete and coach apps must remain separate surfaces connected by one tested data boundary. The coach owns targets, prescriptions, exercise instructions, and planned structure. The athlete owns actual load, actual repetitions, RPE, notes, completion, symptoms, and session results. The engine may calculate an adaptive recommendation, but that recommendation must be a versioned target snapshot, not a mutation of historical athlete results.

The application must remain local-first and work offline. Local storage is the first write path; cloud sync is optional, debounced, and reconciled by stable identifiers. Authenticated secrets and WHOOP provider credentials remain server-side. The adaptive layer must not become an excuse to collapse the data ownership boundary or create a hidden server-only brain that the user cannot audit.

The database and sync model should preserve raw data and derived state separately. Raw logs are facts: what the athlete entered or what the device supplied. Derived fields are estimates: e1RM, trend, fatigue state, confidence, or recommendation. If the algorithm changes, the raw facts should remain available for replay. This is the technical equivalent of the evidence rule that a product heuristic must remain testable.

21. Claims the product may and may not make

The product may say that it uses a conservative, performance-informed, equipment-aware progression policy. It may say that the progression target is inspired by ACSM’s 2–10% operating range. It may say that

load and repetition progression are both supported pathways. It may say that the system uses RPE/RIR conditionally, wearable data as advisory context, and explicit hold states when evidence is insufficient.

The product must not say that 2.5% is the scientifically optimal increase. It must not say that two missed sessions is a validated deload threshold. It must not say that a 5% reduction is proven to restore performance. It must not say that HRV is a direct strength-readiness or injury-readiness score. It must not say that a universal pain score is safe. It must not imply that MacroFactor, JuggernautAI, or RTS independently validated this product. It must not imply that a confidence number predicts hypertrophy or injury prevention unless that exact outcome has been calibrated and tested.

22. Final answers to the original questions

Does ACSM actually recommend a percentage increase?

Yes. The 2009 position stand recommends a 2–10% increase after the athlete exceeds the desired workload by one to two repetitions on two consecutive sessions, with smaller increases for smaller exercises and larger increases for larger exercises. The recommendation is real and useful. Its exact dose is not established by a direct percentage-comparison trial.

What did Plotkin test?

Plotkin compared progressing load while staying within a repetition range with progressing repetitions at the starting load over eight weeks of lower-body training in trained young adults. Both strategies produced meaningful adaptations. It did not test 2.5%, 5%, or 10% increments and cannot justify a particular percentage constant.

Does a trial compare increment size directly?

No convincing trial was found that cleanly compares approximately 2.5%, 5%, and 10% progression increments under otherwise matched conditions. Hostler supports the feasibility of microloading in upper-body exercises. Buskard supports the practical viability of multiple progression methods. Neither identifies a universal optimum.

Is two consecutive failure the right deload threshold?

There is no direct evidence that two is superior to one or three. It is a defensible product debounce rule only when “comparable” is defined and pain, illness, missing data, and unusual conditioning are kept separate. The engine should treat corroborated deterioration as more important than the raw count.

Should deload reduce load or volume?

The evidence does not establish one universal lever. Surveys suggest athletes commonly reduce volume, effort, and sometimes load. A narrow 2026 study found that planned reductions in sets and frequency did not hinder hypertrophy or 10RM strength-endurance in untrained young men. For the engine, isolated local failure can justify a load hold or reduction; broader decline should often reduce volume or overall stress rather than merely lowering the bar weight.

What is the final implementation?

Use a 2.5% target from the last stable opening load, equipment-aware rounding, movement-specific jump caps, and a repetition/RPE fallback. Promote after two comparable successful exposures as a product heuristic. Hold after one miss. After repeated comparable deterioration, reduce from the last successful anchor by a default 5% and escalate only with corroborating systemic evidence. Keep pain, HRV, layoff, conditioning, and data quality in separate state paths. Record every decision with reason codes and evidence status.

23. Closure statement

This research question is now closed for the first product build. Further literature searching may find another adjacent study, but it is unlikely to change the central conclusion without a direct increment-size or reactive-de-load trial. The engine does not need another round of searching before implementation. It needs the rules above encoded, tested, and measured.

The project should proceed with the following sentence in its code documentation:

> THE Hybrid Engine uses a versioned, deterministic, evidence-informed progression and fatigue policy. Its 2.5% progression target and 5% reactive reduction are configurable product heuristics selected inside broader evidence-supported practice ranges; they are not presented as experimentally established optima. The engine prioritises safety, pain separation, comparable performance, equipment-aware rounding, explicit uncertainty, and preservation of the session purpose.

That is the end of the research loop. Build the system, test the system, and let future user data challenge the heuristics rather than hiding them behind the word “science.”

Appendix 1. Evidence ledger for the load-progression decision

The following ledger records the major sources used in the final judgement. It is intentionally explicit about transferability.

Source	Design / population	What it contributes	What it cannot prove	Final status
Kraemer et al., 2002, ACSM position stand, PMID 11828249	Earlier position stand	Historical continuity of repeated-performance progression logic	Increment-size dose response	Position-stand guidance
Feigenbaum & Pollock, 1999, PMID 9927008	Prescription/review paper	Source lineage behind practical progression language	Direct comparison of 2.5/5/10%	Prescription precedent
Plotkin et al., 2022, PMID 36199287 / PMCID PMC9528903	Randomised load versus repetition progression; trained young adults	Load and repetition progression both viable over eight weeks	Increment-size optimum; hybrid generalisation	Direct but narrow
Hostler et al., 2001, PMID 11708713	Small-increment upper-body trial	Microloading can work over eight weeks	Universal long-term optimum; lower-body	Direct but small

			transfer	
Buskard et al., 2019, DOI 10.1249/MSS.00000000000002038	82 older adults; four progression methods	No clear strength/functional winner; RPE tolerability signal	Young hybrid athlete; exact increments	Direct adjacent
Teixeira et al., 2019, PMCID PMC6616272	Novel load-progression strategy	Shows progression can vary across load/volume and still be studied	Universal practical prescription	Adjacent
Schoenfeld et al., 2021, PMCID PMC7927075	Loading recommendations review	Goal-specific load and rep interpretation	Increment-size optimum	Review / adjacent
Huang et al., 2025, PMCID PMC12336695	Autoregulation network meta-analysis	Suggests autoregulated methods may improve strength in pooled data	Product-specific rule; clean separation of achieved intensity	Emerging pooled evidence
Hickmott et al., 2022, PMCID PMC8762534	Autoregulation systematic review	Comparative evidence is mixed and heterogeneous	Universal RPE superiority	Systematic review
Helms et al., 2018, DOI 10.3389/fphys.2018.00247	RIR versus percentage-style training	No clear universal RIR superiority in small trial	All populations/exercises	Direct but small
Graham & Cleather, 2021, DOI 10.1519/JSC.00000000000003164	RIR / autoregulation comparison	Supports possible strength benefit with different achieved intensity	Exact product controller	Direct but confounded by achieved dose
Zourdos et al., 2016, DOI 10.1519/JSC.00000000000001049	RPE/velocity relation	Supports RPE as a structured measurement	Perfect accuracy	Direct acute evidence
Helms et al., 2017, DOI 10.1519/JSC.00000000000001517	RPE/RIR reliability in trained lifters	Supports familiarisation and conditional use	Universal calibration	Direct small study
Mansfield et al., 2020, DOI 10.1519/JSC.00000000000003779	RIR accuracy	Estimates can be wrong, especially away from failure	One universal error correction	Adjacent
Steele et al., 2017, DOI 10.7717/peerj.4105	RIR and failure interpretation	Highlights conceptual and measurement issues	App-ready threshold	Conceptual/adjacent
Vieira et al., 2022, PMID 34881412	Failure-training systematic	Failure adds acute fatigue and	Two-miss deload	Adjacent pooled

	review/meta-analysis	performance cost	threshold	evidence
Rogerson et al., 2024, DOI 10.1186/s40798-024-00691-y	Survey of 246 competitive athletes	Describes common deload timing and levers	Optimal causal schedule	Survey / practice description
Bell et al., 2023, PMCID PMC10511399	International Delphi consensus	Defines deloading terminology and practice	Efficacy or dose-response	Consensus
Coleman et al., 2024, PMCID PMC10809978	One-week midpoint deload comparison	Shows planned cessation can affect strength outcomes differently from hypertrophy	Reactive deload rule	Direct narrow study
Pancar et al., 2026, DOI 10.1038/s41598-026-40612-5	Randomised within-subject; 19 untrained men	Reduced sets/frequency did not hinder 8-week hypertrophy/10RM	Trained hybrid athletes; best deload dose	Direct narrow study
Ogasawara et al., 2013, PMID 23053130	Periodic training with 3-week cessation	Long breaks did not erase all adaptations in a narrow protocol	Normal 7-day reactive deload	Direct narrow study
Bickel et al., 2011	Reduced maintenance volume	Small volume may maintain adaptation	Reactive deload efficacy	Adjacent
Tavares et al.	Reduced volume/frequency after training	Reduced training can maintain prior gains in some contexts	Universal return formula	Adjacent
Atkinson & Nevill, 1998, PMID 9820922	Reliability methods review	Error, limits of agreement, and repeatability principles	Universal session count	Methods review
Hopkins, 2000, PMID 10907753	Reliability and validity methods	Typical error and meaningful-change logic	Product-specific window	Methods review
Grgic et al., 2020, PMID 32681399	1RM reliability systematic review	Reliability depends on test context and protocol	Universal e1RM confidence	Systematic review
Bellenger et al., 2016, DOI 10.1007/s40279-016-0484-2	HRV-guided endurance review/meta-analysis	Possible benefit in endurance distribution	Strength-load gate	Systematic review
Manresa-Rocamora et al., 2021, DOI 10.3390/ijerph181910299	Individualised HRV endurance study	Supports personalisation but not universal cutoff	Mixed-modality app rule	Direct adjacent

de Oliveira et al., 2019, DOI 10.1080/17461391.2019.1572227	HRV-guided resistance study	Small and limited evidence	HRV-only superiority	Direct small
Bittencourt et al., 2024, DOI 10.3389/fphys.2024.1472702	HRV-individualised recovery in older women	Tests autonomic-guided recovery	Young hybrid generalisation	Direct narrow
Thamm et al., 2019, DOI 10.3390/ijerph16224353	HRV and resistance-exercise response	HRV not a direct local muscle measure	Tissue readiness	Adjacent
Hwang et al., 2017, PMID 28328712	Short detraining in trained men	Retention after a short break can be substantial	Fixed return percentage	Direct narrow
Mujika & Padilla, 2000, PMID 10966148	Detraining review	Modality- and duration-specific detraining	Universal strength equation	Review
Encarnação et al., 2022, DOI via MDPI review	Detraining review	Strength and hypertrophy time courses vary	App-ready percentage	Review
Silbernagel et al., 2007, DOI 10.1177/0363546506298279	Achilles pain-monitoring RCT	Condition-specific symptom monitoring can support activity	Universal 5/10 rule	Clinical direct narrow
Sprague et al., 2021, DOI 10.1186/s40814-021-00792-5	Patellar tendinopathy pilot	Pain-monitoring transfer is condition-specific	General recreational rule	Pilot
Tran et al., 2025, DOI 10.2519/jospt.2025.13253	Pain-monitoring review	General evidence certainty is limited	Universal pain threshold	Review
Guo et al., 2017	Calibration study of neural networks	Confidence must be calibrated to an outcome	Training decision confidence UI	Methods / adjacent
Hüllermeier & Waegeman, 2021	Uncertainty taxonomy	Distinguishes uncertainty types	Product-specific safety efficacy	Methods review
Lee & See, 2004	Human-automation trust review	Appropriate reliance matters	App confidence metric	Conceptual review
Kompa et al., 2021	Abstention in medical ML	Uncertainty can trigger review/abstention	Consumer training outcome benefit	Adjacent

MacroFactor public algorithm documents	Product documentation	Smoothing, feedback loops, data-sufficiency precedent	Scientific validation of Hybrid Engine	Product precedent
RTS / Juggernaut public coaching material	Named coaching methodology	RPE/RIR, fatigue, wave, and feedback-loop precedents	Independent validation of complete systems	Coaching precedent

Appendix 2. Source metadata and current links

The principal sources are discoverable through the identifiers below. URLs are included for retrieval, not as a claim that a landing page is itself the evidence.

- Ratamess NA, Alvar BA, Evetoch TK, et al. Progression Models in Resistance Training for Healthy Adults. **Medicine & Science in Sports & Exercise**. 2009;41(3):687–708. DOI: 10.1249/MSS.0b013e3181915670. PubMed: <https://pubmed.ncbi.nlm.nih.gov/19204579/>
- American College of Sports Medicine. Resistance Training Prescription for Muscle Function, Hypertrophy, and Physical Performance in Healthy Adults: An Overview of Reviews. 2026 position stand. ACSM summary: <https://acsm.org/resistance-training-guidelines-update-2026/>
- Plotkin DL, et al. Progressive overload without progressing load? The effects of load or repetition progression on muscular adaptations. 2022. PubMed: <https://pubmed.ncbi.nlm.nih.gov/36199287/>. Full text: <https://pmc.ncbi.nlm.nih.gov/articles/PMC9528903/>
- Hostler D, et al. The effectiveness of 0.5-lb increments in progressive resistance exercise. 2001. PubMed: <https://pubmed.ncbi.nlm.nih.gov/11708713/>
- Buskard ANL, et al. Optimal Approach to Load Progressions during Strength Training in Older Adults. **Medicine & Science in Sports & Exercise**. 2019;51(11):2224–2233. DOI: 10.1249/MSS.0000000000002038. PubMed: <https://pubmed.ncbi.nlm.nih.gov/31107348/>
- Hickmott LM, et al. The Effect of Load and Volume Autoregulation on Muscular Strength and Hypertrophy: A Systematic Review. 2022. Full text: <https://pmc.ncbi.nlm.nih.gov/articles/PMC8762534/>
- Rogerson D, et al. Deloading Practices in Strength and Physique Sports. **Sports Medicine - Open**. 2024;10:26. DOI: 10.1186/s40798-024-00691-y. Full text: <https://pmc.ncbi.nlm.nih.gov/articles/PMC10948666/>
- Pancar Z, et al. Effects of deload periods in resistance training on muscle hypertrophy and strength endurance in untrained young men. **Scientific Reports**. 2026;16:10299. DOI: 10.1038/s41598-026-40612-5. Full text: <https://www.nature.com/articles/s41598-026-40612-5>
- Coleman M, et al. Gaining more from doing less? The effects of a one-week deload period. 2024. Full text: <https://pmc.ncbi.nlm.nih.gov/articles/PMC10809978/>
- Bell L, et al. Integrating Deloading into Strength and Physique Sports Training. 2023. Full text: <https://pmc.ncbi.nlm.nih.gov/articles/PMC10511399/>
- Addleman JS, et al. Heart Rate Variability Applications in Strength and Conditioning. 2024. Full text: <https://pmc.ncbi.nlm.nih.gov/articles/PMC11204851/>
- Grgic J, et al. Test-Retest Reliability of the One-Repetition Maximum Test: A Systematic Review. 2020. PubMed: <https://pubmed.ncbi.nlm.nih.gov/32681399/>
- Silbernagel KG, et al. Continued sports activity using a pain-monitoring model during rehabilitation in patients with Achilles tendinopathy. DOI: 10.1177/0363546506298279.

Appendix 3. Inherited project evidence bundle and design record

The following appendices preserve the project's prior evidence bundle, mechanism audit, and product design record. They remain part of the project audit trail. The reconciled conclusions in the main body govern where earlier wording was broader than the final evidence permits.

Appendix 4. Detailed decision-state catalogue

The following catalogue turns the research conclusion into observable product behaviour. It is intentionally more specific than the evidence itself. These states are implementation choices, not claims that a study has validated each one. Their value is that they prevent the product from collapsing every difficult day into one opaque readiness number.

State: approved

Use `approved` when the planned prescription is compatible with the current safety state, the athlete has enough comparable evidence for the requested action, and no higher-priority conflict is present. Approval does not mean the athlete is guaranteed to complete the work. It means the prescription is reasonable to attempt. The app should still monitor technique, symptoms, and unexpected RPE during the session. An approved decision can become held, reduced, substituted, or blocked after new information appears.

State: held

Use `held` when the current dose remains reasonable but the evidence does not justify increasing it. This is the normal result for noisy or incomplete data. The athlete should not interpret a hold as a failure. A hold preserves the planned exposure and protects the next decision. The explanation should name the missing or conflicting evidence, such as an unusually coarse plate jump, missing RPE, a single poor set, or a wearable signal that conflicts with normal performance.

State: repeated

Use `repeated` when the athlete should repeat the current exposure to obtain a comparable observation. This differs from held: held may complete the current session without a progression change, while repeated explicitly schedules the same or nearly same exposure again. Repetition is appropriate after one non-painful miss, an unfamiliar movement, a large technique change, or an uncertain return from a short gap. It should not be used to pressure an athlete through pain.

State: reduced

Use `reduced` when the planned purpose can be preserved with a lower stress dose. The reduction should identify its lever: load, sets, repetitions, density, complexity, or conditioning intensity. The engine should avoid changing multiple levers unless the state is systemic or safety-related. A reduced strength session can retain the primary lift and remove accessories. A reduced conditioning session can retain aerobic work while removing intervals. The user should be told what was protected.

State: substituted

Use `substituted` when the original movement or modality is unsuitable but the session purpose remains achievable through an approved alternative. The substitution must retain the intended movement or energy-system role as closely as possible and must be recorded as a different exposure for trend purposes. A substitution is not equivalent data unless the exercise library declares it equivalent for a specific decision. The engine should never silently combine a substituted row with the original exercise's strength trend.

State: calibration

Use `calibration` after a meaningful training gap, a major programme change, a new exercise, a new device, or another event that makes the current estimate unreliable. Calibration is not punishment and is not a diagnosis of detraining. It is a deliberate information-gathering state. The first exposure should be conservative enough to reduce unnecessary failure risk and informative enough to observe technique, RPE, symptoms, and completion. Exit criteria must be based on successful comparable evidence, not on a hidden time counter alone.

State: recovery

Use `recovery` when the session purpose is temporarily changed to restore capacity without abandoning the training habit. Recovery can mean easy Zone 2, mobility, technique, a short minimum viable session, or complete rest when appropriate. It is not automatically triggered by one low HRV value. It should be selected when direct performance, symptoms, recent training stress, and context support the conclusion that the planned dose is currently too expensive.

State: blocked

Use `blocked` for a hard safety condition, a movement-specific pain block, invalid data that could make the action unsafe, or a user action that requires professional review. A block is not the same as a low-confidence recommendation. The product must make clear what is blocked, what safe alternative exists, and what information or professional input is needed before the blocked path can resume. The app must not allow a motivational override to bypass the state.

State: needs_input

Use `needs\_input` when one missing piece of information would materially change the decision and can reasonably be collected from the athlete or coach. Ask one useful question rather than presenting a questionnaire. Examples include whether an incomplete set was stopped by pain or fatigue, whether a substitution was intentional, or whether the load was entered in kilograms or pounds. Missing optional data should lower confidence without blocking the session.

State: needs_clinical_review

Use `needs\_clinical\_review` for red-flag symptoms, persistent or worsening pain, neurological symptoms, chest pain, fainting, severe breathlessness outside the intended conditioning stimulus, or another configured reason to leave autonomous programming. The product should not diagnose. It should state that the training decision is outside the engine's authority and advise appropriate professional or urgent care according to the configured safety copy.

Appendix 5. Acceptance matrix for the adaptive layer

The matrix below is the minimum test inventory before the adaptive engine is considered complete. Each test should run against the pure rules engine and, where relevant, through the athlete UI, coach assignment boundary, offline persistence, sync merge, and rehydrated session state.

Load arithmetic

1. A 25 kg anchor with a 2.5% target and 2.5 kg equipment step must display an actual 10% jump and choose hold or fallback, never label it 2.5%.
2. A 50 kg anchor with a 2.5 kg step must calculate an actual 5% jump and apply the movement cap before promoting.
3. A 100 kg anchor with a 2.5 kg step must calculate an actual 2.5% jump without floating-point drift.
4. A 180 kg anchor with a 2.5 kg step must calculate an actual 1.388...% jump and retain the unrounded calculation in the audit record.
5. A zero or negative anchor must fail validation and return `needs_input` rather than producing an infinite or negative percentage.
6. A missing equipment step must not be treated as zero; the engine must hold or request gym setup information.
7. A unit conversion from pounds to kilograms must be explicit, versioned, and visible in the decision inputs.
8. A candidate load below the anchor must not be selected as a promotion even if it is the closest available equipment value.
9. A plate step that exceeds the movement cap must trigger fallback without deleting the success evidence.
10. Two candidate loads equally close to target must be resolved by a deterministic tie-break rule.
11. A machine with a fixed 5 kg step must not inherit a barbell microplate configuration without an explicit equipment identity.
12. A changed bar weight must invalidate a direct comparison unless the exercise configuration declares the change safe.
13. A candidate load must preserve the programmed unit and display unit conversion only as presentation when appropriate.
14. The engine must store target percentage and actual percentage separately when rounding changes the physical jump.
15. A progression percentage outside configured safety bounds must be rejected or require coach authorisation.

Exposure validity

16. Changing from a flat bench to an incline bench must create a separate variation identity and not increment the flat-bench success streak.

17. Changing the prescribed repetition range must mark the exposure non-comparable for the old progression rule.
18. A set completed with a pain flag must not count as a normal successful exposure.
19. A set completed with missing repetitions must not be inferred from the target value.
20. An exercise substitution must preserve the session purpose but remain separate trend data unless equivalence is configured.
21. A large rest-interval change must lower comparability when the exercise rule declares rest material.
22. A deleted or edited historical set must create an auditable correction rather than silently changing the old decision.
23. A duplicate offline submission must be idempotent and must not create a second success exposure.
24. A session marked incomplete must not be promoted as complete solely because all entered sets appear valid.
25. A coach target update must not overwrite athlete-owned actual load or repetitions.
26. An athlete note saying 'felt easy' without an RPE value must remain qualitative context, not a numeric RPE.
27. A set entered after the session date must retain its actual timestamp and not be backfilled as a normal exposure without an audit note.
28. A changed exercise video or cue alone should not invalidate a comparable exposure if the movement identity remains stable.
29. A new device source must be marked as a data-source change until its measurement behaviour is known.
30. A coach-published target snapshot must remain reproducible even if the coach later edits the template.

Promotion and holds

31. One valid success must hold the next load when the confirmation count is two.
32. Two comparable successes must permit promotion only if no pain, calibration, or unresolved conflict state exists.
33. A successful exposure with missing RPE may count toward completion but not toward high-confidence progression when RPE is required by the movement rule.
34. A strong performance after unusually high conditioning stress must not automatically increase both strength and conditioning dose.
35. A good readiness score with poor performance must produce hold or investigation, not automatic escalation.
36. A low wearable recovery score with normal direct performance must not force a reduction by itself.
37. A successful repetition fallback must be logged as progress without falsely claiming a load promotion.
38. A progression decision must change only one primary lever unless a composite state explicitly allows more.

- 39. A promotion must carry engine version, reason codes, source observations, and actual rounded jump.
- 40. A later engine version must be able to replay the old decision under the old version without rewriting history.
- 41. A coach override must be recorded as an override and not masquerade as automatic engine approval.
- 42. An athlete override must not bypass a pain block or clinical-review state.
- 43. A hold should not reset the success streak unless the exposure was invalid or the rule explicitly says it does.
- 44. A failed repetition fallback must not create an additional hidden load reduction.
- 45. A long period of holds must trigger review or data-quality guidance rather than endless silent repetition.

Misses and reductions

- 46. One comparable non-painful miss must hold and record context rather than automatically reduce.
- 47. Two comparable misses must be identifiable as the exact exposures that formed the heuristic evidence.
- 48. A pain-related miss must not increment the normal performance-miss streak.
- 49. A miss after an unusually hard interval session must carry conditioning context into the decision.
- 50. A miss caused by equipment failure must be invalid for progression and not treated as athlete decline.
- 51. A cross-session reduction must calculate from the last successful opening anchor, not from the walked-down effective load.
- 52. A 5% reactive reduction must display the rounded load and the intended unrounded value.
- 53. A systemic decline state must be able to reduce volume without changing the load anchor silently.
- 54. A local movement miss must not automatically reduce unrelated exercises in the same session.
- 55. A whole-session decline must not be inferred from one accessory miss.
- 56. A successful calibration exposure must not be counted as a normal promotion unless the calibration rule permits it.
- 57. Repeated misses separated by a long gap must not be combined as consecutive comparable misses.
- 58. A missed session with no performed sets must be distinct from a failed session with attempted sets.
- 59. The engine must never stack missed work into the next session without an explicit, safe programme rule.
- 60. A user-requested 'make-up' session must preserve recovery spacing and return a reasoned alternative if unsafe.

Pain and safety

- 61. Sharp pain during a movement must immediately leave normal fatigue logic and enter the configured symptom pathway.
- 62. Chest pain, fainting, or severe unexplained dizziness must block autonomous training decisions.
- 63. Delayed soreness must be stored separately from acute movement pain.

- 64. A pain score alone must not be treated as a diagnosis or universal permission to continue.
- 65. A pain block must apply to the affected movement or pattern without unnecessarily blocking unrelated safe work.
- 66. A substitution after pain must be recorded as substitution data, not silently merged with the original trend.
- 67. A pain block must persist across reload and offline restart until a valid update changes the state.
- 68. An athlete pressing 'continue anyway' must not bypass a hard safety state.
- 69. Clinical-review copy must avoid diagnosis, certainty, and false reassurance.
- 70. The engine must preserve the user's symptom description even if the coach later changes the planned exercise.

HRV and context

- 71. A missing HRV value must be represented as missing, not as a normal score or zero.
- 72. An HRV value outside the athlete's personal baseline must lower confidence only when protocol and timestamp are valid.
- 73. HRV alone must never increase load or volume.
- 74. A low HRV plus normal performance must remain eligible for the planned purpose with progression held if configured.
- 75. A low HRV plus poor performance and high recent conditioning load may enter conservative mode.
- 76. A high HRV value must not authorise a risky progression when technique or pain is poor.
- 77. Two wearable devices with incompatible values must create a conflict state rather than an averaged fiction.
- 78. A stale recovery score must be marked stale and must not control a same-day decision.
- 79. Sleep, stress, illness, and work context must remain distinct fields even if they contribute to a summary.
- 80. The explanation must identify whether the recommendation was driven by direct performance or advisory wearable context.

Layoff and calibration

- 81. A return after a configured meaningful gap must enter calibration rather than use an invented percentage equation.
- 82. A one-day scheduling miss must not automatically trigger the same calibration state as a multi-week gap.
- 83. Calibration must cap unnecessary failure exposure while retaining enough work to observe current ability.
- 84. A calibration session with pain must remain in the safety pathway and not exit calibration as a normal success.
- 85. A successful calibration exposure must record why it was not used as a normal anchor.
- 86. Calibration exit must be deterministic and based on stated evidence, not an invisible timer.

- 87. Different modalities must have separate layoff rules and must not share a single cross-modality reduction percentage.
- 88. An illness return must not be treated as an ordinary training gap when symptoms are still active.
- 89. An equipment change after a layoff must extend or restart calibration when comparability is lost.
- 90. The user must see that calibration protects information quality and is not a punishment for time away.

Data, sync, and ownership

- 91. Local writes must be durable before cloud sync is attempted.
 - 92. The same session submitted twice from offline replay must merge idempotently by stable identifiers.
 - 93. Coach target fields must never be overwritten by athlete result fields during reconciliation.
 - 94. Athlete result fields must never be erased because a coach publishes a later target snapshot.
 - 95. A deletion tombstone must prevent an old device from resurrecting a removed assignment.
 - 96. Derived estimates must retain the raw observations and engine version that produced them.
 - 97. A cloud conflict must be visible and resolved by the declared merge policy, not last-write-wins everywhere.
 - 98. An assignment must be self-contained enough for the phone to use it if the coach UI is unavailable.
 - 99. WHOOP secrets and provider credentials must never appear in browser storage or cached static assets.
1. Authenticated function routes must not be cached by the service worker.
 2. A corrupted backup must fail closed and leave the current local state intact.
 3. Exported data must contain enough metadata to reproduce a decision without private provider tokens.
 4. Reset-local-data must be explicit, confirmed, and must not silently delete cloud history.
 5. The adaptive layer must remain usable offline with an honest degraded-data state.
 6. Coach and athlete apps must agree on the same exercise identity and prescription measurement schema.

User experience and explanations

- 7. Every automatic change must have a plain-language explanation generated from structured reason codes.
- 8. A hold message must say what was preserved and what evidence would permit progression.
- 9. A reduction message must identify the changed lever and the reason for choosing that lever.
- 10. A calibration message must explain that the engine is gathering current evidence rather than assuming detraining.
- 11. A blocked message must distinguish safety limitation from missing optional data.
- 12. The app must not show an unqualified confidence percentage without outcome, horizon, and data context.
- 13. The user must be able to inspect the observations behind a decision without reading raw database JSON.
- 14. The user must not be shamed for missing a session or asking for a shorter option.
- 15. A minimum viable session must state which part of the original purpose was preserved.
- 16. A coach must be able to see whether a target changed automatically, manually, or through athlete feedback.
- 17. The app must avoid physiological explanations that exceed the evidence or imply diagnosis.

18. The interface must make the next useful action obvious even when the answer is hold or rest.
19. Reduced-motion and accessible touch targets must remain intact in the live logger.
20. A user can reject an optional recommendation without losing the underlying record.
21. History must show the original prescription and actual result separately.

Appendix 6. Operational protocols for evidence-safe iteration

Protocol A: changing a constant

No constant should be changed merely because a user had one unusually good or bad session. A proposed change must identify the current constant, its evidence status, the expected benefit, the possible failure mode, the affected movement classes, and the replay fixtures that will be rerun. The change should be represented as an engine-version update so old decisions remain reproducible. If the change affects a safety gate, it requires a separate review from ordinary progression tuning.

Protocol B: adding a new data source

When adding a wearable, heart-rate bridge, machine sensor, or coach-entered field, first define its measurement semantics. What exactly is measured? At what time? With what missingness? How is it normalised? What population or device validation exists? Which decisions may use it, and which decisions may it never control? A new source begins in advisory mode. It may graduate to a stronger role only after the app has enough within-user data and a documented validation result. The presence of a number is not proof of usefulness.

Protocol C: interpreting a surprising athlete result

When an athlete dramatically exceeds a target, do not immediately increase all future work. Check whether the target was entered correctly, whether the exercise and equipment were comparable, whether the repetitions were valid, whether the athlete misread RPE, and whether the result persists. When an athlete dramatically underperforms, check the same categories plus sleep, illness, conditioning, pain, and work stress. The first response should usually be classification, not storytelling. The engine can say “unexpected result; hold and verify” without inventing a mechanism.

Protocol D: handling user overrides

Overrides are valuable evidence because they show where the engine’s default does not fit the athlete’s reality. They must not be hidden. Store the requested action, the engine action, the reason for the override, whether a safety state was bypassed, and the result. An override may change a non-safety prescription, such as removing an accessory because of time. It must not bypass a pain block, clinical-review state, or red-flag stop. Repeated overrides of the same type should become a product-review signal rather than a reason to silently adapt the rule.

Protocol E: distinguishing adherence from efficacy

If a user completes more sessions under the adaptive engine, that is an adherence or usability signal. It is not by itself proof that the engine produces more strength or hypertrophy. If strength improves, the engine still cannot claim causation without a suitable comparison. Product validation should report adherence,

completion, user comprehension, safety-state integrity, and performance trends separately. The system is allowed to be useful before it is proven superior, but it is not allowed to call usefulness superiority.

Protocol F: closing a research question

A research question is closed for implementation when the evidence has been searched to a reasonable depth, direct evidence and gaps have been stated, the product decision is explicit, the decision's provenance is recorded, and the remaining uncertainty is assigned to validation rather than endless browsing. The question can reopen if a new direct trial appears, a safety incident occurs, or product data shows the heuristic is systematically inappropriate. "Closed" means ready to build and test; it does not mean eternally beyond revision.

Protocol G: audit log minimum

Every adaptive decision should retain the engine version, timestamp, user and exercise identity, session purpose, anchor load, opening load, effective load, target increment, actual increment, equipment step, success/miss evidence, RPE/RIR, symptoms, recent conditioning summary, wearable status, missing fields, conflicts, reason codes, chosen state, and next recheck point. The user interface need not show every field at once, but the record must exist for replay and safety review.

Protocol H: release gate

The adaptive layer is not release-ready until static checks, browser smoke tests, offline reload tests, sync merge tests, security checks, and the acceptance matrix pass. A polished screen is not evidence that the engine works. A passing unit test is not evidence that the coach and athlete data boundary is intact. A green deployment is not evidence that a WHOOP secret is protected. Each layer has its own acceptance criteria.

Protocol I: what to do when evidence conflicts

Conflicting studies should not be reduced to a vote. First identify whether they tested the same outcome, population, exercise, dose, and deload definition. Then determine whether the conflict is real or caused by different questions. If the conflict remains, choose the lower-risk product policy, expose the uncertainty, and preserve an experiment path. A conflict is not a failure of science; it is information about where the product should avoid false certainty.

Protocol J: the human handoff

The engine should know when not to decide. Persistent pain, red flags, complex medical history, severe illness, or a request for rehabilitation guidance should leave the autonomous path. A coach or clinician can provide a target, but the data contract must record that the decision was human-authored or clinically informed. The system should never imply that a coach override became objective truth merely because it was entered through the same interface.

Appendix 7. Final engineering checklist

Before the project is called complete, confirm the following in the repository and deployed application:

- The adaptive rules are pure, deterministic functions with versioned configuration.

- The current target, actual result, and derived estimate are separate data types.
- A flat kilogram rule is not used globally across exercises.
- Target and actual percentage changes are both calculated and displayed correctly.
- Equipment rounding cannot silently exceed the configured movement cap.
- Repetition fallback exists where a safe load jump cannot be expressed.
- One miss holds and gathers context; it does not trigger an unexplained large reduction.
- Repeated comparable deterioration is distinguished from pain, illness, and invalid data.
- Reactive reduction is calculated from the last successful opening anchor.
- Within-session correction is not silently compounded into cross-session reduction.
- HRV is advisory and cannot independently increase load or override pain.
- Layoff return enters calibration and has no universal time-off percentage equation.
- Pain is separate from fatigue and has a persistent block or modification path.
- Every decision has reason codes, evidence status, missing data, conflicts, and next recheck.
- Coach target fields and athlete result fields cannot overwrite one another.
- Local-first persistence survives reload, screen lock where supported, and offline use.
- Sync merges stable records without resurrecting deleted work.
- Secrets remain server-side and authenticated routes are not cached.
- Historical raw observations can replay derived decisions under the original engine version.
- The app communicates holds and reductions without shame or false certainty.
- The repository includes this dossier or an equivalent decision record next to the implementation.

The project is closed when these checks are either passed or deliberately recorded as a known release limitation with an owner and next action. A known limitation is acceptable. An invisible limitation is not.

Appendix 8. Sensitivity analysis: what changing the heuristic would do

Because the literature does not identify an optimal increment, the product should understand the practical consequences of plausible settings. The following analysis is not a physiological prediction. It is a control-policy sensitivity analysis: how often would the engine create a large hardware-driven jump, how often would it hold, and how much historical confirmation would be needed before the athlete sees a different prescription?

At a 20 kg anchor, a 2.5 kg equipment step is a 12.5% jump. At a 25 kg anchor it is 10%. At 30 kg it is 8.33%. At 40 kg it is 6.25%. At 50 kg it is 5%. At 60 kg it is 4.17%. At 80 kg it is 3.125%. At 100 kg it is 2.5%. At 140 kg it is 1.79%. At 180 kg it is 1.39%. A single flat step therefore behaves like several different prescriptions. The engine's cap is needed even if the target percentage is changed later.

If the target is reduced from 2.5% to 2%, more heavy-lift promotions will be expressed by the same available step because the step is already above the target, while more light movements will fall into the fallback state because the next physical jump remains too large. If the target is increased to 5%, more medium and heavy movements will wait for a larger jump or use repetition progression. That may be appropriate for a strength-focused block, but it will also delay physical load changes on machines with coarse steps. If the target is increased to 10%, most light movements will exceed the small-movement cap and most heavy movements will be vulnerable to a larger-than-necessary stress change. The engine should therefore treat target percentage and actual jump cap as separate controls.

The same sensitivity applies to the reactive reduction. A 5% reduction from 100 kg is 95 kg; a 10% reduction is 90 kg. On a 25 kg movement, the same settings produce 23.75 kg and 22.5 kg before rounding, which may be

impossible with standard equipment. Rounding 23.75 kg to 22.5 kg creates a 10% physical reduction, while rounding to 25 kg creates no reduction. The app must display the physical load that will be used and the intended reduction so that a coach can see when hardware has distorted the nominal policy.

The safest first release is therefore not the one with the most precise-looking percentages. It is the one that makes distortion visible. For every load change, the audit record should answer: what was the anchor, what target change was requested, what hardware steps were available, what actual change was applied, whether the change exceeded the cap, and what fallback was used. This lets later validation evaluate the policy in the form the athlete actually experienced rather than in the abstract percentage the software intended.

Appendix 9. What a decisive future study would need to test

The central unanswered question is experimentally tractable. A decisive study would recruit participants with sufficient training experience to make progression meaningful, stratify or randomise by sex and training status, and assign comparable exercises to progression policies that differ in increment size. The trial would need at least three arms, such as 2.5%, 5%, and 10%, with the trigger and target repetition range held constant. It would need to report actual load changes, not only intended rules, because plate granularity can make the arms converge or diverge.

The study should separate upper-body and lower-body movement classes and record the relative size of each physical jump. It should use familiar exercises, standardise range of motion and rest, record RPE/RIR and technique, and report adherence, missed sets, repeated exposure count, time to stall, strength, hypertrophy, and adverse events. It should pre-register how a successful exposure is defined and whether one, two, or three confirmations are required. Otherwise the study may answer the wrong question while appearing to answer the right one.

For deloading, the decisive design would compare a hold-only policy, a load-reduction policy, a volume-reduction policy, and a combined policy after a clearly defined and corroborated deterioration state. It should distinguish reactive from planned deloads, use trained participants, include a hybrid or concurrent-training condition, and report subsequent performance, adherence, perceived recovery, pain events, and training quality. A study that simply inserts a one-week cessation period in untrained people cannot settle the product's reactive rule.

For HRV, the decisive design would compare a fixed programme, an HRV-only programme, and a multi-signal programme with pre-specified thresholds and missing-data handling. It would need to standardise measurement protocol, use within-user baselines, and test whether the policy improves meaningful outcomes rather than merely changing the number of hard sessions. The strength outcome should be separated from recovery or adherence outcomes.

Until such studies exist, the engine should not wait for certainty that the field does not currently possess. It should implement the conservative controller, label the constants, preserve the raw data, and make future comparison possible.

Appendix 10. Why this dossier is sufficient to close the project

The purpose of the research phase was not to find a sentence that could be pasted into code as an unquestionable answer. It was to determine whether the proposed rules were supported, unsupported, or supportable as explicit engineering choices. That work is complete. The report has identified the strongest

direct sources, traced the ACSM recommendation to its prescription lineage, corrected the overextension of Plotkin, located the closest microloading evidence, reviewed the available deload evidence, and separated the strength-specific questions from the hybrid-conditioning and product-safety questions.

It has also answered the uncomfortable questions that a less careful report would hide. There is no known direct percentage-band trial that proves 2.5% is better than 5% or 10%. There is no validated universal consecutive-failure threshold. There is no validated time-off-to-load equation. HRV is not a complete readiness gate. A pain-monitoring threshold from a specific rehabilitation protocol cannot become a universal consumer rule. A confidence badge is not calibration. These are not gaps that can be solved by writing more assertive prose.

The project can now move from research to implementation because each gap has a bounded response. The absence of a direct increment trial becomes a configurable percentage target, equipment cap, repetition fallback, and validation log. The absence of a deload threshold becomes repeated corroborated deterioration, a visible heuristic count, separate local and systemic pathways, and no hidden compounding. The absence of a layoff equation becomes calibration. The limits of HRV become advisory use. The limits of pain evidence become symptom separation and a safety handoff. The limits of confidence scores become action-linked evidence states.

This is the correct meaning of “evidence-informed” for a software system. It does not mean every branch was discovered by a randomised trial. It means the system knows which branches were studied, which are transferred cautiously, which are inherited from coaching practice, and which are product decisions made under uncertainty. It preserves enough raw data to be corrected later. It makes its own behaviour testable. It tells the athlete what it knows and what it does not know.

The remaining work is ordinary product work: encode the state machine, connect it to the existing local-first data boundary, write tests, run the app in the browser, inspect mobile and coach flows, and measure real behaviour. Reopening the same literature search before doing that would not increase the quality of the first release. The research phase has supplied the operating boundaries. The build phase must now prove that the implementation respects them.

Final decision: ship the bounded deterministic adaptive layer with the constants and labels in this dossier, then let controlled product validation—not endless argument—determine whether a later version should move those constants.

One final practical point deserves emphasis. A training engine does not become more intelligent by making a larger number of adjustments. It becomes more intelligent by making fewer unjustified adjustments and by learning from the ones it does make. A system that progresses every time the athlete reports a good set may appear responsive while quietly creating unstable load jumps. A system that deloads after every imperfect day may appear cautious while quietly destroying momentum. The controller must therefore be judged by proportionality: did it respond to the size and quality of the evidence, did it preserve the intended training purpose, and did it leave the next decision clearer than the last one?

That standard also protects the athlete from the psychological damage of over-interpretation. A hold is not a verdict on capability. A calibration session is not a declaration of weakness. A reduction is not punishment. A blocked movement is not the end of training. By making these states explicit, the app can support adherence without lying about certainty. This is especially important for a hybrid athlete whose capacity changes with work, sleep, conditioning, stress, and the ordinary unpredictability of life.

The final dossier is therefore both a research conclusion and a product boundary. It says what the engine can responsibly do now, what it must not claim, and what evidence would be required to change the rule later. That is enough to close the research phase and begin implementation with a clean conscience.

In short, the engine now has a defensible centre of gravity: direct performance outranks a single wearable signal; repeated comparable evidence outranks one dramatic session; safety and symptoms outrank training ambition; raw observations outrank derived scores; and explicit uncertainty outranks confident-looking folklore. Those priorities should remain stable even if the visible interface, programme templates, or future data sources change.

That stability is the final safeguard against scope drift. New features may add information, but they should not quietly promote an advisory signal into authority or convert a temporary heuristic into a permanent scientific claim. Any future change should pass through the same contract: identify the evidence, name the uncertainty, define the action, preserve the audit trail, and test the edge cases before release.

The next action is therefore implementation, not another debate about the decimal place. Start with the smallest coherent controller, keep every decision reversible, and use the athlete's actual logged experience to decide whether the defaults deserve refinement.

That is the practical finish line: an honest rule, a clear boundary, a reproducible record, and a product that can improve without rewriting what happened.

No further research pass is required for the initial build. Future evidence belongs in versioned change records, not in an endless loop that prevents the system from being tested.

This document is the final research baseline for the first release and the reference point for all later rule changes.

Part VII. Repository-grounded implementation audit

Audit scope and source boundary

This part records what the current repository contributes to the close-out decision. It is not a reconstruction from the product brief and it is not a claim that every planned surface is live. The inspected repository is ``reflectprotect123-max/THE-HYBRID-ENGINE1``, public on GitHub, default branch ``main``, inspected at the 16 August 2026 close-out point. The current tree is identified in the audit notes by commit ``e07eeae8dc3d863a71272985a66b5b52b278bf8d``. The direct repository source is [THE-HYBRID-ENGINE1 on GitHub](<https://github.com/reflectprotect123-max/THE-HYBRID-ENGINE1>). File-level claims below should be rechecked against a later commit before release.

The repository changes the character of the project. Earlier research could only specify what a safe engine ought to do. This codebase shows which decisions have already been made, where they have been made, and which boundaries have been recognised by the author. It also shows where a good local rule can remain ineffective because the caller, product shell, or cross-package contract does not consume it. For that reason the audit separates five states: implemented and exercised; implemented but not proven effective in the product; designed but not wired; deliberately excluded; and unresolved.

Architecture readout

The monorepo is organised around a shared engine rather than a single screen. ``packages/engine`` contains the legacy-compatible training model and a local merge boundary. The package has explicit modules for sessions, logging, lifts, autoregulation, conditioning, heart-rate interpretation, balance, insights, database sanitisation, cloud mapping, coach-to-athlete emission, numeric helpers, plate math, and storage. That decomposition is valuable because it puts arithmetic, persistence, and presentation in separable places. It also creates a verification obligation: every decision must have one authoritative calculation and every consumer must call it.

``packages/shared-core`` holds versioned facts and cross-application contracts. ``packages/whole-athlete-state`` holds recovery, life context, constraints, and data quality. ``packages/session-authoring`` represents authoring boundaries. Nutrition is separated into core, engine, and adapter packages. The Android athlete application is the whole-athlete surface; the web application is the coach workspace. The README records that the earlier athlete web surface and several coordinator/auto-coach/strength-engine/conditioning-engine packages were deleted. This is a healthy reduction in duplicate orchestration, but it means old rows, migrations, and compatibility names must not be mistaken for active producers.

What the engine already does well

The lift path has unusually clear provenance. ``lift.ts`` distinguishes the last completed non-warm-up working set from warm-ups, requires real repetitions, uses the same plan-anchored fold to compute the next opener, and banks the result as a replacement map rather than mutating settings in place. It protects against a late or restored session overwriting a newer record. ``nextWorkingWeight`` applies today's recovery adjustment at read time, so a bad morning can ease the offered weight without erasing what the athlete previously earned. ``openingLoadFor`` makes the precedence rule explicit: today's logged fact, authored percentage, earned weight, then honest blank/zero.

The conditioning path is equally important because it demonstrates restraint. ``conditioning.ts`` distinguishes formats and modalities, treats recovery as a prescription-time modifier rather than a reason to erase earned progression, excludes simulated sessions, refuses to earn progress when the heart-rate record contains no usable zone time, uses work time rather than rest-inclusive duration for the denominator, and keeps the HRR gate provisional rather than pretending it is validated. The one-session success path is not allowed to become a universal dose equation.

The adaptive strength layer in ``packages/engine/src/adaptive/strength.ts`` adds a read-only, explanation-oriented decision function rather than a second storage system. It requires three exposures before making a suggestion, uses the same exposure selection rules as lift progression, supports a repetition route before a load route, and returns an explicit ``pause_insufficient_data`` state when history is thin. Its result carries action, confidence, reason codes, safety state, data limitations, and an optional prescription. That contract is the right shape for a coach or athlete surface because the number is not detached from why it was offered.

``plates.ts`` is a small module with a large safety implication. It sanitises inputs, searches a deterministic greedy-plus-correction space, and returns both the achievable load and the delta from the requested load. It refuses to pretend that an unachievable prescription was hit. Equipment-aware rounding is not cosmetic: a 2.5% target can become a 10% physical jump on a light lift, and a safe controller must know the difference.

Whole-athlete state: model present, integration still decisive

The whole-athlete package has a clear schema. It represents readiness as a band with data quality, capacity separately for strength and conditioning, recovery debt, illness status, advisory HRV, constraints, and rationale. Pain and illness are hard constraints; low readiness, high recovery debt, high physical load, time limitation, and low energy availability are softer training constraints. HRV is explicitly advisory and does not by itself alter readiness or create a pain gate in the tests. Nutrition is passed as observed intake and estimated expenditure, not as a target or instruction.

The crucial close-out finding is not that this model is absent. It is that the repository documentation says the package raises pain and illness flags but, at the inspected point, nothing consumes them downstream. That is a wiring gap, not a research gap. A hard constraint that is never read by the session selector, prescription layer, logger, or coach review surface is only a typed intention. The project cannot be considered closed until the consumer path is proven with an integration test: create the flag, ask for a session decision, verify that normal progression is blocked or redirected, and verify that the explanation names the hard constraint.

Verification and deployment boundary

The root scripts show a serious verification posture: typecheck, unit tests, ecosystem contracts, reachability, CSS-state checks, site build, CSP checks, React smoke, deploy smoke, and parity harnesses. The README also calls out offline reload, live sync behind an explicit environment gate, server-side WHOOP and Concept2 functions, encrypted provider tokens, and service-worker update behaviour. These are not training-science findings, but they are part of a safe adaptive engine because a correct rule that is stale, duplicated, unsynced, or bypassed by a browser cache is not correct in use.

The repository's `.claude/agents` directory contains a read-only locator role and a review-only role, while the dispatching skill asks for independent domain passes followed by integration and a full suite. This dossier follows that discipline as a multi-pass audit of the code and evidence. No external agent output is being presented as if it ran; the source of the implementation findings is the repository itself. The distinction matters because orchestration metadata is not test evidence.

Repo-grounded close-out judgement

The project has crossed the line from idea to implementable system. The core engine contains real guards, explicit precedence, persistence semantics, explanations, and tests. The evidence-supported product shape is therefore strong enough to freeze as a deterministic baseline. It has not crossed the line to “finished” merely because the code is extensive. The remaining closure work is integration proof: consume whole-athlete hard constraints, prove the adaptive decision output reaches the actual athlete surface, verify coach/athlete ownership through sync, and run the full verification command at the commit being released. Those are finite release gates. They are not invitations to reopen the entire research question.

Repository evidence map

Area	Current repository evidence	Close-out interpretation
Within-session RPE	autoreg.ts, fold.ts, session tests	Use as a control signal, not a diagnosis
Conditioning	conditioning.ts, conditioning tests	Good separation of earn, prescribe,

		exclude, and hold
HRV/recovery	hr.ts, whole-athlete state, state tests	Advisory unless corroborated; missingness must remain visible
Pain/illness	whole-athlete state types/state	Hard flags exist; consumer integration is the major gap
Equipment	plates.ts, numeric helpers	Physical jump must be computed, displayed, and audited
Data boundary	db.ts, cloud.ts, merge/restore tests	Preserve raw facts and deterministic reconciliation
Product surfaces	Android athlete app plus web coach workspace	Verify one decision path reaches the athlete
Release proof	root verify and smoke/parity scripts	Run at release commit; do not infer from unit tests

Part VIII. Full mechanism-by-mechanism close-out monograph

This part expands the implementation lock into a searchable review record. Each subsection is intentionally framed as a decision, evidence boundary, failure analysis, state transition, validation requirement, and closure judgement. The repeated structure is a control against selective attention: every mechanism must answer the same questions before it is allowed to change training.

Chapter 1 — Repository anatomy and ownership

1.1 One engine, many surfaces

Question. The same training fact must have one authoritative calculation even when it appears in mobile logging, coach review, summaries, and sync payloads. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The repository separates engine, shared-core, session-authoring, nutrition, mobile, and web packages. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of

observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. Duplicated formulas create silent disagreement between what the athlete sees and what is stored. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Trace one completed lift from authored target through logger, local persistence, sync, recap, and coach view. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Keep calculation in the engine and make every surface consume the typed output. This is strong enough to freeze as the implementation position for the current release, subject to the explicit

integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

1.2 The local-first boundary

Question. Offline use is a product requirement, not a convenience, because the athlete may be training without a network connection. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The README describes durable local storage, restore, merge, and live-sync checks. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A cloud-first design can lose a completed session or make a stale target appear current. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Kill the network after a completed set, reload, resume, sync later, and verify idempotent reconciliation. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Local completion must be durable before remote acknowledgement is required. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

1.3 Coach ownership versus athlete facts

Question. A coach-authored prescription and an athlete-entered result are different authorities. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The repo names coach workspace and athlete app boundaries and contains cloud mapping and merge modules. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. Last-write-wins can let a later plan overwrite the actual load performed or erase a user correction. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Submit conflicting target and result edits from separate clients and inspect the merged record and audit trail. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Declare field ownership explicitly; never resolve semantic conflicts with timestamp alone. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

1.4 Deleted orchestration packages

Question. Removed packages and historical database names must not be treated as live behaviour. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The README records deletion of coordinator, auto-coach, strength-engine, conditioning-engine, and AI-prescription packages. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A migration or compatibility writer can leave the impression that an old producer still exists. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A

coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Search imports, build outputs, row writers, and route reachability for each deleted namespace. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Retain compatibility reads only when they are documented and tested; remove ambiguous live paths. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

1.5 Session authoring as a single surface

Question. There should be one place where a coach builds the canonical day/session representation. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The README identifies `DayBuilder.tsx` as the authoring surface and rejects a second builder. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. Two builders drift in defaults, exercise identity, or progression metadata. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation

is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Create the same session through every exposed route and compare the serialised assignment. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. One canonical authoring boundary reduces ambiguity and makes replay possible. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

1.6 Package contracts as safety boundaries

Question. A package boundary is useful only if its types prevent forbidden data from crossing it. The implementation question is not whether this sentence sounds sensible. It is whether the product can

represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. Nutrition context intentionally omits targets, while whole-athlete state exposes observations and constraints. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A broad `any` or convenience field can reintroduce a forbidden instruction through a side door. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Compile contract fixtures that attempt to pass a nutrition target into training state and expect failure. The test must exist at the pure function level and at the boundary where the decision reaches

the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Use types, tests, and review rules together; types alone are not sufficient evidence. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

1.7 Versioned namespaces

Question. A decision must remain reproducible after schemas and algorithms evolve. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. Shared-core and whole-athlete state expose schema/version concepts. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. Without versioning, a later reader can reinterpret an old field under new semantics. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Persist an old snapshot, update the package, restore it, and compare the replayed decision under the recorded version. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Version facts and rules separately when either can change the meaning of a decision. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

1.8 Mobile as the athlete truth surface

Question. The screen under the bar is where an adaptive rule becomes behaviour. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The repository has an Android athlete app while the athlete web surface was removed. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A correct helper that is not called by the mobile logger is functionally absent. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Start from `openDraft`, follow the weight/reps prefill, and prove the value comes from the canonical engine function. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Prioritise reachability and parity at the actual athlete interaction point. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

1.9 Web as coach oversight

Question. Coach review needs more context than a single recommendation number. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The web app is described as a coach workspace with readiness, strength, conditioning, nutrition, and progression routes. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A coach view that hides reason codes can turn a heuristic into an unchallengeable command. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order

matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Verify that each changed prescription displays source, reason, confidence, and the underlying exposure. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. The coach surface should expose the decision record, not merely the final load. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

1.10 Verification as part of the model

Question. Build and smoke checks protect the semantics of an adaptive system at runtime. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The root `verify` script includes typecheck, test, reachability, build, CSP, React smoke, and deploy smoke. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree,

or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. Passing unit tests does not prove that a route is reachable or that a service worker serves current code. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Run the complete verification command in a clean checkout and archive its output with the release commit. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant

supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Treat verification results as release evidence with a timestamp and commit identity. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

Chapter 2 — Strength progression and loading

2.1 Opening load as a fact hierarchy

Question. The engine must resolve what to put on the bar without letting several valid sources disagree silently. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. `openingLoadFor` documents fold, authored percentage, earned weight, then honest none/bodyweight handling. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A stale banked load can override a coach's explicit percentage, while an untouched fold can hide all history with zero. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Exercise a fresh movement, a percentage target, a banked target, a completed set, a bodyweight movement, and a missing-data case. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. One function returns kilogram value, message, and source together. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

2.2 Last working set versus warm-up

Question. Warm-up data is useful for readiness but must not become the working-load anchor. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. `lift.ts` filters warm-up blocks and warm-up sets before banking movement progress. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. An empty-bar warm-up at easy RPE can teach the system an absurdly low working weight. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Log warm-ups before a heavy movement and assert that the banked opener follows the working set only. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Keep warm-up exclusion in the shared selection helper, not in individual screens. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

2.3 Ramp exercises

Question. A ramped movement must compare the next opener with the same opener that the decision describes. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The lift tests include ramped exercises and protect `from`, `to`, delta, and reps as a single semantic pair. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. Reading the last set for `from` makes a valid 100/110/120 ramp look like an unexplained 120-to-100 change. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be

represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Use flat and ramped fixtures with easy, on-target, hard, and incomplete final sets. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Every displayed delta must compare like with like; otherwise the explanation is numerically false. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

2.4 Authored percentage loads

Question. A percentage of e1RM is a prescription, not a replacement for today's observed result. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. ``prescribedKg`` resolves an authored percentage against the same e1RM used elsewhere and rounds to the configured increment. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact

constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. Parsing `80%` as eighty repetitions or allowing a percentage with no usable history can produce unsafe nonsense. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Test `5 @80%`, `@80%`, `80%`, out-of-range percentages, no history, and a conflicting earned load. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Keep the percentage syntax explicit and refuse to guess when the reference e1RM is absent. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law

of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

2.5 Microloading and actual percentage

Question. A target increment must be evaluated after equipment rounding, not before. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The repo has plate math and numeric helpers that expose achievable load and delta. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A nominal 2.5% instruction can become a 10% physical jump on a light bar or an impossible plate combination. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence

would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Compare nominal target, rounded candidate, actual percentage, and equipment delta across light, medium, and heavy anchors. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Store both intended and achieved changes so the controller can choose hold or fallback honestly. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

2.6 The repetition fallback

Question. When equipment cannot express a safe load step, progress can move through repetitions or another declared lever. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The adaptive strength layer includes a rep route before a load route and checks the current displayed target. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A rep fallback that is lower than the plan's existing target becomes a disguised downgrade. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a

communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Test low-load exercises, top-of-range repetitions, bodyweight movements, and a plan already asking for the proposed rep count. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. The fallback must be explicit in the decision record and must not masquerade as load progression. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

2.7 One lever at a time

Question. Load, repetitions, sets, rest, and density are not interchangeable interventions. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The strength layer separates ``progress_load``, ``progress_reps``, hold, and reduction actions. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. Changing two or three levers at once destroys attribution and can overspend recovery. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A

coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Create a two-session success streak and verify exactly one primary prescription field changes. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Composite changes require a separately named state and a stronger reason than ordinary success. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

2.8 Stable anchors

Question. Progression should use a successful opening anchor rather than a failed, walked-down effective load. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The lift comments distinguish what was earned across the session from what the next opener is reported against. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. Compounding a within-session back-off with a cross-session reduction can spiral downward. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying

observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Run a hard final set that walks the fold down, then inspect the next session's anchor and reduction calculation. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Record anchor, effective load, and next offered load as separate fields. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

2.9 Comparability of exercise identity

Question. A progression streak is meaningful only when the exposures represent the same movement and context. The implementation question is not whether this sentence sounds sensible. It is whether the

product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The engine normalises names and the adaptive layer selects a first comparable occurrence per session. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. An incline bench or substituted machine can falsely continue a flat-bench streak. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Change variation, equipment, grip, range, and target while preserving the display name and assert that comparability breaks. The test must exist at the pure function level and at the boundary

where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Exercise identity must include the dimensions that matter for the rule, not only a label. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

2.10 When to hold

Question. A hold is an active control output that preserves information and avoids overreacting. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The adaptive contract includes a hold action with reason codes such as mixed results and already-earned load. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. Users may interpret hold as failure if the interface does not explain what was preserved. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Trigger one success, one miss, conflicting wearable data, and insufficient history; verify distinct hold explanations. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. A good hold states the next evidence that would permit a change. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

2.11 Three exposures before suggestion

Question. Sparse history should lower confidence and prevent the system from writing a confident prescription. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. ``adaptive/strength.ts`` defines a minimum exposure count and returns ``pause_insufficient_data`` below it. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A single easy set can be a measurement error, novelty effect, or unusual day. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Provide zero, one, two, and three valid exposures, including invalid and non-comparable sessions. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Use a minimum data rule as a safety feature, not as a claim that three is physiologically optimal. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

Chapter 3 — RPE, RIR, and within-session control

3.1 RPE as an observed control signal

Question. RPE can guide a next-set adjustment without becoming a diagnosis of fatigue or readiness. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. ``autoreg.ts`` parses target and felt effort while the fold applies a bounded plan-anchored adjustment. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. Treating one rating as a precise physiological measurement can create false confidence. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering

evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Use missing, nonnumeric, boundary, and contradictory RPE values and verify safe fallback behaviour. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Use RPE to control the local dose and preserve the uncertainty around what caused the rating. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

3.2 Target RPE versus felt RPE

Question. The controller must compare what was felt with what was asked, not compare the athlete with the target itself. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The lift implementation explicitly distinguishes `felt` from prescribed `rpe`. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test

establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. Judging every set against its own target makes every set look perfect and prevents adaptation. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Run identical target sets with felt RPE below, at, and above target and inspect the verdict and next load. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change,

data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Store both values and name the deviation used by the rule. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

3.3 RIR conversion

Question. RIR is useful when the interface or coaching practice uses it, but conversion must be explicit. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The repo's RPE/RIR utilities are tested as parsing and classification helpers. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. An unlabelled seven can mean RPE 7 or seven repetitions in reserve depending on the field. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Test notation, units, missingness, and impossible values across the logger and persisted session. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Never infer the scale from a bare number when the input contract can require a label. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

3.4 The hard-set guard

Question. A set that misses the repetition floor is qualitatively different from a set that simply feels a little hard. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The strength layer treats a missed rep floor as a miss and the fold can lock a hard exercise walk. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A later easy back-off can incorrectly raise the movement again if the hard set does not lock the sequence. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Create a missed target followed by an easy set and assert that the earned movement does not rise. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. A failed set must remain visible as the event that constrained the session. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

3.5 Dead bands and noise

Question. Small deviations around target should not trigger constant up-and-down movement. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The repo documents a plan-anchored walk and uses bounded adjustments rather than unlimited set-to-set movement. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. Without a dead band, normal rating noise becomes oscillation. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Replay a sequence of target-minus-one, target, target-plus-one, and half-point ratings. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Define the dead band as a product heuristic, record it, and tune it from replay data rather than intuition. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

3.6 RPE with technical breakdown

Question. A numeric effort score cannot replace a technique or symptom flag. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. Whole-athlete state introduces hard pain constraints separate from ordinary readiness signals. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A user may rate a painful set as easy while still making it unsafe to progress. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Pair each RPE band with pain, form, and completion flags and assert that hard flags outrank effort. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. The safety state must be resolved before the ordinary effort controller is allowed to act. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

3.7 Missing effort ratings

Question. Missing RPE should reduce what the engine claims to know, not silently become a normal rating. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. Adaptive explanations carry data limitations and distinguish low confidence. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. Defaulting missing effort to target creates artificial success streaks. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A

coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Omit RPE from one set, one session, and all sessions; inspect bank, streak, and confidence outputs. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. A missing field may preserve the session but should block high-confidence escalation where effort is required. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

3.8 RPE across exercise classes

Question. An RPE rule that works for a barbell lift may not transfer to a timed conditioning interval or isolation movement. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The repository separates lift and conditioning modules and does not force one universal prescription path. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. Cross-domain reuse can turn modality-specific observations into a false common scale. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation

is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Run matched ratings through strength, intervals, steady conditioning, and bodyweight paths. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Keep shared parsing small and keep decision semantics modality-specific. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

3.9 Athlete override of the number

Question. The athlete standing under the bar has information the app may not have. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant

fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. `nextWorkingWeight` keeps the field typeable and presents an eased offer rather than an immutable block. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. An override can become invisible data loss or a bypass of a hard safety state. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Record a voluntary load change, a pain-related attempt to continue, and a blocked override separately. The test must exist at the pure function level and at the boundary where the decision

reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Allow non-safety override, preserve the reason, and never let motivation bypass a hard stop. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

3.10 Explaining effort decisions

Question. The explanation must be generated from the same inputs that produced the number. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. `adaptive/explain.ts` reshapes existing outputs into typed reason codes without recomputing them. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A second UI formula can display a reason that does not match the actual load. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Compare every explanation to the underlying `WorkingWeight`, `Prescription`, or `AdaptResult` fixture. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. One computation, one structured result, one human-readable note. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

Chapter 4 — Conditioning and hybrid interference

4.1 Format-specific conditioning

Question. Steady work, intervals, rowing, running, and mixed formats have different meaningful exposure variables. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action,

and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. ``conditioning.ts`` keys progression by format and modality and avoids collapsing all conditioning into one level. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A global conditioning level can reward an easy format and raise an unrelated high-intensity prescription. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Progress each format independently, substitute modality, and inspect the progression map. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a

delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Use format identity as part of the evidence key and expose it in the explanation. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

4.2 Work time versus rest-inclusive time

Question. The denominator for conditioning quality should match the physiological question being asked. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The repo comments explicitly use work-time denominator rather than total duration including rests. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. Including rest can make a hard interval session look artificially easy or a steady session look incomplete. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering

evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Construct equal work with different rest and equal total duration with different work; compare completion classification. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Keep work, rest, total duration, and zone time as separate raw facts. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

4.3 Heart-rate data quality

Question. No usable heart-rate zone time should not become a quiet success or failure. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. `conAdapt` excludes sessions with no zone time from earn/deload logic. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test

establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A missing sensor can be misread as low intensity and wrongly earn progression. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Test no sensor, partial sensor, dropout, simulated record, and complete zone data. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change,

data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Missing cardiovascular data means no cardiovascular verdict, not zero effort. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

4.4 RPE primary in short intervals

Question. Short intervals can make HR lag or obscure the intended work signal. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The conditioning comments use RPE as primary for short intervals and HR as secondary/diagnostic. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A universal HR gate can penalise a valid interval session or reward a delayed heart-rate response. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved

percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Compare short intervals with identical work but different HR response and use direct effort and completion as the primary evidence. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. The evidence hierarchy must be format-specific and visible. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

4.5 No HRR gate by default

Question. Heart-rate recovery is attractive as a marker but not automatically a validated training gate. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The current code marks the HRR condition provisional and leaves it non-gating. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of

observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A named metric can acquire authority merely because it appears precise. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Replay records with high, low, missing, and delayed HRR and verify that the current rule does not invent a hard stop. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Keep HRR advisory until a protocol and validation dataset justify a stronger role. This is strong enough to freeze as the implementation position for the current release, subject to the explicit

integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

4.6 Conditioning progression and recovery

Question. A low recovery signal may ease today's prescription without destroying the earned conditioning level. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. ``conPrescription`` applies a daily adjustment while ``conAdapt`` banks earned level from the completed session. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. Using the same low score to both reduce today and erase the banked level can double-penalise the athlete. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Earn a level, prescribe on a low-recovery day, complete an on-target adjusted session, and inspect the next level. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Separate what the athlete earned from what today is wise to attempt. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

4.7 Overcooked sessions

Question. High-zone exposure or excessive effort can make completion inadequate even when minutes are high. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The conditioning logic has an overcooked guard based on high-zone share and effort context. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. Volume alone cannot distinguish productive work from a session that overspent capacity. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Hold duration constant while varying high-zone share and RPE; check progress, hold, and miss paths. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. The engine should describe overcooked evidence without diagnosing the cause. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

4.8 Strength-conditioning interference

Question. A hybrid engine must account for the interaction of hard conditioning and strength exposures without pretending to model every mechanism. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. Whole-athlete state counts recent hard sessions and lower-body hard sessions; balance and conditioning modules remain separate. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. An easy-looking lift after a hard interval may be a poor place to escalate load, while one bad lift should not erase all aerobic work. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A

coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Use paired-week fixtures with varied ordering, density, and modality and inspect domain-specific decisions. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Use a conservative context signal and preserve domain-specific evidence. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

4.9 Minimum viable conditioning

Question. A short session can preserve the habit and purpose even when the full prescription is not appropriate. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. Whole-athlete state contains a time-limited constraint and the acceptance matrix calls for a minimum viable session. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. Treating a short day as failure encourages make-up work and recovery debt. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a

missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Limit available minutes to several bands and verify that the output states what purpose is retained. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Reduce optional work before discarding the training purpose. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

4.10 Conditioning substitutions

Question. A substitute can preserve an energy-system purpose while remaining distinct data. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the

action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The product architecture includes multiple modalities and the dossier requires explicit substitution records. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. Merging a bike substitute into a running trend can create a false baseline. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Substitute modalities with identical nominal duration and compare trend keys and explanations. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior

session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Substitution is an intentional change of evidence, not a silent equivalence. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

Chapter 5 — Readiness, HRV, and whole-athlete state

5.1 Readiness is a state estimate

Question. Readiness should summarise available observations with data quality rather than pretend to measure a hidden physiological truth. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. Whole-athlete state returns score, band, signals, rationale, and data quality. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A single composite score can hide whether the problem is sleep, soreness, stress, illness, or missing data. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell

whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Create the same score from different signal combinations and inspect the rationale and confidence. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Show the inputs and limitations alongside the band. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

5.2 HRV as advisory context

Question. HRV can inform a decision but should not independently prescribe load or create a diagnosis. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or

sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. State tests assert that HRV alone does not change readiness or create a pain gate. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A low value can be caused by measurement conditions, illness, stress, or normal variation. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Hold direct performance constant while varying HRV and verify advisory-only behaviour. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data,

missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. If HRV changes an action, the decision must also state the corroborating evidence and protocol. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

5.3 Missingness as information

Question. A missing wearable signal is not a zero, a normal value, or a negative result. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The state model distinguishes unknown and data quality bands; conditioning excludes absent zone data. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. Default values manufacture confidence and can drive a false reduction or progression. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a

new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Remove one signal at a time and then all signals; compare decisions and explanations. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Represent missingness explicitly through every boundary. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

5.4 Sleep and stress context

Question. Sleep and life stress may alter today's capacity but do not identify a local movement limitation. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. State derives signals and rationale from sleep, stress, soreness, life load, and training density. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live

caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A general readiness reduction can be applied too broadly to a pain-specific movement or too narrowly to a systemic issue. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Vary one context domain at a time and inspect whether strength and conditioning capacity remain separate. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these

fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Context should constrain the session purpose proportionally and transparently. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

5.5 Recovery debt

Question. Accumulated context can justify avoiding another hard session without becoming a permanent label. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The state model estimates debt from recent observations and hard-session density. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A debt score can create a feedback loop where reduced training prevents new evidence and the score never clears. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved

percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Run hard weeks, recovery weeks, missing-observation weeks, and a single unusually stressful day. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Debt needs decay, observation windows, and a clear path back to normal. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

5.6 Pain hard constraint

Question. Pain is not an ordinary fatigue score and must outrank progression logic. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The state package emits ``pain_hold_active`` as a hard constraint with an adjustment to modify or stop. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of

observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. If the flag is only displayed but not consumed, the system can still offer normal escalation. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Raise the flag through the canonical input, request a session, and verify that the final athlete action is held, modified, or blocked. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. The consumer integration test is a release blocker. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in

the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

5.7 Illness hard constraint

Question. Illness status requires a return-to-training process rather than a generic low-readiness adjustment. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. State emits ``illness_flag_active`` as a hard constraint and explicitly avoids diagnosis. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. An illness flag that only lowers load may still recommend high-intensity work. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Test active, suspected, and clear statuses across mobile, coach, and sync paths. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Illness must select a safe return state and preserve the distinction from ordinary tiredness. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

5.8 Low energy availability context

Question. Nutrition facts can inform training context without becoming a training app's food prescription. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. Nutrition context contains logged intake and estimated expenditure but no calorie target or macro instruction. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A target passed into state can make training secretly prescribe nutrition through a side door. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Attempt to pass target-like fields, test sparse logging, and verify only soft training constraints are emitted. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Keep nutrition facts observational and keep food instruction in the nutrition engine. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

5.9 Capacity by domain

Question. Strength and conditioning may be affected differently by the same context. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The state snapshot contains separate capacity bands for overall, strength, and conditioning. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. One overall band can flatten a useful choice such as easy aerobic work while holding heavy lower-body loading. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A

coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Create domain-specific training facts and verify the selector can choose a preserved purpose. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Use the least restrictive safe domain-specific action. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

5.10 The integration gap

Question. A well-typed state package is not a safety feature until a caller reads it. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The README's implementation note says pain and illness flags were not consumed downstream at the audit point. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. The engine may continue to offer normal prescriptions despite a hard flag. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a

missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Build a full path test from state derivation to actual session card, not only a unit test of ``deriveAthleteState``. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Close this gap before describing the project as finished. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

Chapter 6 — Pain, illness, layoff, and calibration

6.1 Pain hold is movement-specific

Question. A pain event should protect the affected movement or pattern without unnecessarily blocking unrelated safe work. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. Conditioning holds are keyed by exact format and modality, while whole-athlete state can represent a broader safety hold. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A global stop can be overly restrictive; a local hold can be dangerously narrow when symptoms are systemic. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence

would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Raise a lower-back pain hold, request upper-body and lower-body sessions, and verify the intended scope. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Scope must be declared and the explanation must name the affected area or pattern. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

6.2 Pain stop versus hard effort

Question. A set stopped for pain cannot be scored as a normal miss. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. Conditioning completion distinguishes ``pain_stop``; pain holds persist until acknowledgement or a non-pain result according to the current rule. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. Mixing pain with fatigue contaminates progression streaks and can reward pushing through symptoms. The important distinction is between an arithmetic error, a data-quality error, a state-precedence

error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Stop an interval for pain, stop a lift for effort, and compare their state transitions and stored evidence. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Pain is a separate pathway from the performance controller. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

6.3 Acknowledgement semantics

Question. A hold should persist until a meaningful acknowledgement or a valid state change, not merely until reload. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The conditioning hold is keyed to an acknowledgement record and exact format/modality. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A stale UI flag or an overly eager auto-clear can re-expose the athlete to the same problematic stimulus. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A

coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Reload offline, sync across devices, acknowledge on one device, and inspect the other device's state. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Persist the hold and the acknowledgement event with timestamps and scope. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

6.4 Red flags and clinical review

Question. Some inputs exceed the authority of an autonomous training engine. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The state model says illness is not diagnosis and the dossier requires clinical review for red-flag symptoms. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A confident exercise substitution can sound like medical advice. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or

incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Test configured red flags and verify that the interface stops at handoff language rather than prescribing around them. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Autonomy must include a principled abstention state. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

6.5 Short training gaps

Question. A missed day should not be treated like detraining. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In

this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The repository's adaptive design distinguishes insufficient history and calibration from ordinary progression. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A universal time-off percentage overreacts to scheduling noise. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Use one day, three days, one week, and a longer break with otherwise identical history. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data,

missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Gap length and modality-specific context should decide whether calibration is needed. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

6.6 Longer returns

Question. After a meaningful break, the engine should gather current evidence before chasing old loads. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The dossier's calibration state and adaptive contracts provide a place for conservative re-entry. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. Using the previous peak directly can create avoidable failure; using a huge arbitrary reduction can waste information. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a

new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Return at several fractions of the old anchor, record completion/RPE/symptoms, and verify deterministic calibration exit. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Calibration is a measurement protocol, not a universal detraining equation. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

6.7 Illness return

Question. Return from illness is not the same as a normal layoff because symptoms and systemic tolerance matter. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. Whole-athlete state carries illness status and separates it from readiness signals. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its

result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. An athlete can be recovered from a scheduling gap but still not ready for intensity after illness. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Keep illness active across a gap, then clear it and test the first two exposures. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these

fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Require a clear state transition and conservative return evidence. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

6.8 Equipment changes during calibration

Question. A new bar, machine, or plate set changes comparability even if the exercise name is unchanged. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. Plate math and exercise identity provide the foundation for equipment-aware comparisons. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A familiar load on unfamiliar equipment may not represent the same exposure. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved

percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Change equipment identity in the middle of a return sequence and inspect comparability and anchoring. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Calibration should restart or widen uncertainty when equipment changes materially. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

6.9 Substitution after pain

Question. A safe alternative must preserve purpose while remaining separate evidence. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The product contract distinguishes substitution from the original movement trend. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of

observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. Silently assigning a substitute's success to the painful movement can falsely clear the hold. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Substitute, complete, and inspect both the original hold and the alternative trend. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. A substitute does not prove the original movement is ready. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and

verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

6.10 Exit criteria

Question. States need explicit exit conditions or they become permanent or arbitrary. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The adaptive contract includes action and safety fields but the consumer must define transition rules. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A hidden timer can exit calibration or a pain hold without new evidence. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Document and test every entry and exit edge, including missing data and sync conflicts. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. A state is closed only when entry, persistence, exit, and audit are all specified. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

Chapter 7 — Data quality, persistence, and sync

7.1 Raw facts versus derived decisions

Question. A derived load or readiness band should never replace the observations that made it possible. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The engine contains database sanitisation, restore, cloud mapping, and merge boundaries. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact

constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. Without raw facts, a later rule cannot replay or challenge an earlier decision. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Delete a derived field and replay from raw session and context facts; compare the result. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Persist observations, rule version, and derived output together. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a

bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

7.2 Idempotent session completion

Question. Offline retries must not create duplicate exposures. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The README highlights live sync and the acceptance matrix requires stable identifiers and idempotent replay. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. Duplicate success records can falsely satisfy a confirmation count. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A

coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Submit the same completed session repeatedly from an offline queue and inspect history and progression. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Idempotency is an adaptive-safety property because streaks depend on count. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

7.3 Out-of-order completion

Question. A late-restored session must not overwrite a newer earned load. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. `liftAdapt` checks stored timestamps before replacing the progression map. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A backup or delayed sync can rewind an athlete to an older opener. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or

incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Complete sessions in chronological and reverse arrival order and compare the final map and audit log. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Use event time and arrival time separately. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

7.4 Sanitising numeric inputs

Question. Invalid numbers must fail safely at the boundary before they reach a progression formula. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or

sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. ``plates.ts`` and numeric helpers reject nonfinite values and clamp or sanitise where appropriate. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. Infinity, NaN, empty strings, or negative loads can create false huge jumps or broken UI. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Fuzz load, reps, RPE, increments, units, and timestamps through the public functions. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data,

missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Sanitisation must preserve the fact that input was invalid rather than quietly inventing a value. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

7.5 Merge authority

Question. Different fields may have different owners and conflict policies. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The repo separates coach assignment and athlete result concepts and provides merge/restore tests. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A single record-level last-write-wins policy is too coarse for adaptive data. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order

matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Conflict target, actual set, note, pain flag, and derived output independently. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Merge by declared field authority and retain conflict metadata. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

7.6 Tombstones and deletions

Question. A deleted assignment or movement must not reappear from an old device. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The acceptance matrix calls for deletion tombstones and offline conflict protection. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be

bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. Resurrection can expose an outdated or unsafe prescription. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Delete on one device, replay an old queue on another, and sync in both orders. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant

supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Deletion is a state transition with provenance, not absence of a row. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

7.7 Provider data boundaries

Question. Wearable credentials and provider secrets must never be treated as ordinary client data. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The README says WHOOP and Concept2 functions are server-side and tokens are encrypted in Netlify Blobs. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A cached browser response or service-worker asset can leak credentials or stale provider data. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen

at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Inspect browser storage, service-worker cache, function responses, and CSP behaviour in a deployed smoke test. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Security boundaries are part of adaptive correctness. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

7.8 Stale recovery data

Question. A yesterday's recovery value must not silently control today's same-day prescription. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The code distinguishes session-captured recovery from late current data and the dossier requires stale markers. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact

constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. Retroactive readings can change a verdict after the athlete has already trained. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Use timestamps around midnight, delayed sync, and a late WHOOP update. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Persist the input timestamp and apply freshness rules at the decision point. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a

bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

7.9 Audit record completeness

Question. A user or reviewer must be able to reconstruct why a number changed. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. Adaptive explanations expose reason codes, confidence, safety state, and data limitations. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A final load without anchor, actual jump, equipment step, or evidence cannot be audited. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence

would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Snapshot a progression event and verify replay with the same engine version. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. The audit record is a first-class product output, not debug logging. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

7.10 Reset and restore

Question. Local reset must be explicit and restore must fail closed on corruption. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The repository includes restore and storage modules and the README describes local-first behaviour. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A destructive reset or partial restore can erase evidence or create a mixed-version state. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a

communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Corrupt backups, cancel resets, restore old versions, and verify current state remains intact on failure. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Make recovery operations reversible where practical and visibly confirmed. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

Chapter 8 — Explanations, confidence, and human control

8.1 Reason codes are contracts

Question. A stable reason-code vocabulary lets UI, analytics, and audits agree on meaning. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. ``adaptive/types.ts`` defines a closed reason-code set and explanation shape. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. Free-form strings drift, become untestable, and can imply a stronger claim than the engine supports. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence

would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Exhaustively switch over reason codes and ensure every action has a non-empty note. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Keep machine reason and human note separate but generated from the same decision. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

8.2 Confidence is not probability of success

Question. A high-confidence engine decision should not be rendered as a guarantee that the athlete will succeed. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The explanation contract distinguishes confidence levels and data limitations rather than claiming a calibrated probability. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A percentage badge can be interpreted as a physiological forecast without validation. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a

communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Inspect copy for low, medium, and high confidence under both rich and sparse data. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Describe confidence as evidence quality and rule certainty, not outcome certainty. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

8.3 Safety state versus confidence

Question. A decision can be low-confidence but still safe to try, or high-confidence that the correct action is to block. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The contract carries confidence and safety state as separate fields. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. Combining them into one score can make a hard stop look optional. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A

coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Create low-confidence approved, high-confidence reduced, and blocked cases and verify UI hierarchy. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Safety state outranks persuasive copy and user motivation. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

8.4 The explanation must match the number

Question. A reason that describes an eased load while the field shows an earned load is a functional defect. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. `explainWorkingWeight` and `explainConPrescription` consume already-computed objects rather than recomputing them. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. Parallel calculation in a view can drift after a constant changes. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or

incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Compare all visible numbers and notes against one fixture across mobile and web. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Make the engine result the single source for value and explanation. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

8.5 Hold explanations

Question. A hold should tell the athlete what was protected and what would unlock progression. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or

sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. Adaptive reason codes include insufficient history, mixed results, and already-earned load. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. ‘No change’ sounds arbitrary and encourages manual escalation. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Trigger each hold reason and verify a specific next-evidence sentence. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting

data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. The next useful action is part of the hold decision. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

8.6 Reduction explanations

Question. A reduction must name the changed lever and avoid blaming the athlete. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The contract differentiates reduce load, reduce volume, and deload actions. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A generic ‘recovery low’ message cannot tell whether load, sets, or intensity was changed. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a

new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Compare strength, conditioning, pain, illness, and time-limited reductions. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Explain purpose preserved, lever changed, and recheck condition. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

8.7 Coach override provenance

Question. Human judgement can be correct without being an automatic engine result. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The dossier and repository boundary support coach workspace ownership and explicit decision sources. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is

imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A coach override can later be mistaken for validated algorithmic evidence. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Override a progression, sync it to mobile, and inspect source, author, timestamp, and subsequent outcome. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these

fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Store automatic, coach-authored, athlete-adjusted, and clinical-review sources distinctly. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

8.8 Athlete agency

Question. An adaptive offer should support the athlete's judgement without turning a safe block into a challenge. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The lift path keeps the field typeable while hard safety states are defined separately. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A motivational override can undermine the exact safety layer the engine exists to protect. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved

percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Try editing an eased offer, a hold, a pain block, and a clinical-review state. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Permit agency inside safe bounds and make prohibited bypasses clear and respectful. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

8.9 Accessibility of explanations

Question. A safe explanation must be understandable under time pressure and available to different users. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The web/mobile verification surface includes UI and reachability checks; the dossier adds accessible touch and reduced-motion requirements. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of

observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A color-only red state or a hidden tooltip can make the safety decision invisible. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Test screen reader labels, touch targets, contrast, reduced motion, and offline rendering. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Safety information must survive presentation changes. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates

in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

8.10 Audit view versus athlete view

Question. The athlete needs clarity; the reviewer needs detail; both should derive from one record. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The typed explanation includes concise note plus structured limitations and prescription. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. Showing raw JSON to athletes is unusable, while hiding it from reviewers prevents audit. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Render the same decision in compact athlete, coach, and audit formats and compare semantic fields. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Use layered presentation without creating layered truth. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

Chapter 9 — Testing strategy and release evidence

9.1 Unit tests as local proofs

Question. Pure functions are the right place to prove arithmetic, parsing, and state transitions. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The engine contains focused tests for lift, autoregulation, conditioning, plates, cloud, restore, parity, and adaptive decisions. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact

constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A green local test can still miss caller reachability and deployment semantics. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Map each rule to at least one direct unit fixture and one boundary test. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Unit tests prove local contracts; they do not prove the product path. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a

bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

9.2 Property-based numeric testing

Question. Load and equipment arithmetic has a large edge space that hand-picked examples do not cover. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. Numeric helpers and plate math expose deterministic public functions suitable for fuzzing. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. Rare NaN, unit, tie, or rounding cases can create large real-world jumps. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence

would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Generate finite and invalid anchors, increments, plate sets, and unit conversions and assert invariants. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Test monotonicity, boundedness, achievable reporting, and no false hit claims. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

9.3 Scenario replay

Question. Adaptive behaviour is a sequence problem, not only a single-call problem. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The engine persists progression maps and has session history and adaptive exposure selection. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A rule can pass isolated tests while oscillating, compounding reductions, or forgetting a hold over time. The important distinction is between an arithmetic error, a data-quality error, a state-precedence

error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Replay weeks containing successes, misses, illness, pain, gaps, substitutions, and device changes. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Store scenario fixtures as decision timelines with expected states. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

9.4 Integration tests for hard constraints

Question. The most important missing proof is that state output reaches the final prescription. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. Whole-athlete tests currently prove derivation; README notes downstream consumption is incomplete. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A unit-perfect safety state can coexist with an unsafe logger offer. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A

coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Run state derivation, session selection, opening-load resolution, UI rendering, and sync as one test. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Do not close the project until this path is green. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

9.5 Reachability checks

Question. Dead code is not product behaviour even when it is tested. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The root scripts include reachability checks and the repo has removed unused surfaces/packages. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A helper can remain green in isolation while no route imports it. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or

incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Trace adaptive functions from app entry points and fail if the intended consumer is absent. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Every safety-critical function needs a live caller test. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

9.6 Parity checks

Question. Coach and athlete surfaces must agree on the same facts and decisions. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor

observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The root scripts include behavioural, visual, harness, and mobile parity checks. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A coach can see one load while the athlete's logger opens another. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Publish a target, log a result, sync, and compare both surfaces at each state. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data,

conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Parity is semantic first and visual second. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

9.7 Offline and reload tests

Question. The safety decision must survive the conditions in which an athlete actually trains. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. README and scripts mention offline reload and service-worker behaviour. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A stale cached app can offer an old rule or lose a pain hold. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order

matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Go offline before and after state changes, reload, update the service worker, and inspect the visible decision. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Offline behaviour must be an explicit degraded-data state, not an accident. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

9.8 Deployment smoke

Question. A deployed function, CSP policy, and service worker can alter the effective product. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The root verification includes build, CSP, React smoke, and deploy smoke. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by

an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. Local tests cannot see production headers, route wiring, or cache behaviour. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Run deployment smoke against the release URL with browser section failures treated as failures. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant

supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Archive deployed commit, environment mode, and smoke results. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

9.9 Mutation testing of guards

Question. A test suite should fail when a critical guard is removed. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The code contains deliberate guards for warm-ups, nonfinite plates, missing HR data, and pain/illness separation. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A test that only checks output on happy paths may not protect the guard itself. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be

represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Temporarily remove or invert each guard and verify a test fails. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Use mutation-style review for safety-critical conditions. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

9.10 Release evidence bundle

Question. A close-out release needs a compact evidence package that a future reviewer can reopen. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The project already preserves evidence bundles, design notes, source links, and acceptance matrices. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic

when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. Without commit, test, and rule-version metadata, later review becomes archaeology. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Package source snapshot, test output, decision schema, known gaps, and rollback instructions. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Freeze the baseline and reopen only for a named trigger. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product

policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

Chapter 10 — Product closure, governance, and future validation

10.1 What ‘closed’ means

Question. Closing the research loop means the product rule is explicit and buildable, not that every uncertainty has disappeared. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The dossier records evidence status, provisional constants, implementation rules, and release gates. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. Endless research can become avoidance of a finite engineering decision. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be

represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Review each open question and assign it to implemented rule, validation study, or release blocker. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Close the question when the decision, provenance, and test are assigned. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

10.2 The 2.5% default

Question. A conservative default can be useful without being promoted to a scientific optimum. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The dossier chooses 2.5% of a stable opening load inside the broader ACSM practical range. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic

when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. Users may read a product default as a universally validated prescription. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Display the rule status in developer and coach documentation and replay across equipment scales. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Label it an engineering heuristic and tune it from observed bounded outcomes. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of

training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

10.3 The 5% reactive reduction

Question. A repeated-decline heuristic can protect the next exposure while avoiding a compounding collapse. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The dossier and acceptance matrix specify reduction from the last successful anchor, separate from within-session back-off. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A flat or compounded reduction can punish one bad day or reduce too far on a light movement. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence

would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Compare one miss, repeated misses, pain miss, gap-separated misses, and anchor changes. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Keep the heuristic bounded, auditable, and reopenable from incident data. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

10.4 Confirmation count

Question. Requiring more than one comparable success filters noise but delays progression. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The product lock uses two comparable confirmations as a debounce heuristic; adaptive strength code also requires a minimum history before suggestions. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. Too little confirmation creates volatility; too much creates stagnation. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Simulate fast responders, slow responders, beginners, and sparse logs with the same rule. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Treat confirmation as a product tuning parameter, not a claim of universal physiology. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

10.5 Outcome validation

Question. The engine needs product validation that is separate from its scientific rationale. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The evidence dossier separates adherence, comprehension, safety-state integrity, performance, and superiority claims. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. More completed workouts or improved strength cannot by themselves prove causal superiority. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A

coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Define pre/post and comparison analyses before collecting outcomes. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Report usefulness honestly and do not overclaim experimental efficacy. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

10.6 Incident review

Question. A safety incident should reopen the relevant rule rather than trigger a vague promise to improve AI. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The engine has explicit states, reason codes, raw facts, and versioned decisions suitable for reconstruction. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. Without an incident taxonomy, the project may change constants without understanding the failure. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids

converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Classify incident source, data quality, state precedence, UI bypass, sync, and human override. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Reproduce first, patch second, retest the neighbouring states third. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

10.7 Rule change governance

Question. Changing a constant changes athlete behaviour and must be versioned like a schema migration. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the

action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The operational protocol requires engine version, replay fixtures, risk review, and old-decision reproducibility. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A silent constant change makes historical decisions impossible to explain. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Change one constant in a fixture set and inspect old and new outputs under their recorded versions. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a

prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Release rule changes intentionally and retain the old interpretation. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

10.8 Human handoff

Question. A good engine knows when the next action belongs to a coach or clinician. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The state and explanation contracts include blocked, held, safety, and review concepts. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. Forcing an autonomous answer can be more harmful than admitting uncertainty. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering

evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Trigger red flags, persistent pain, complex conflict, and severe missingness and inspect the handoff. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Abstention is a completed decision when the engine lacks authority. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

10.9 Agent-assisted review discipline

Question. Independent review roles can increase coverage when their outputs are integrated and verified. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The repository contains locator/reviewer agent instructions and a dispatching skill. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit

test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. Agent output can be mistaken for evidence or can duplicate work without a final integration pass. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Require exact file/symbol findings, classify each finding, then run the full test and human review. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change,

data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Use agents as bounded reviewers; never let them become an unverified source of truth. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

10.10 Final release checklist

Question. Closure requires a finite checklist that can be signed off. The implementation question is not whether this sentence sounds sensible. It is whether the product can represent the relevant fact, preserve it across the session lifecycle, choose a bounded action, and explain the action without overstating the evidence. In this project, the answer must be traceable from an input field or sensor observation to an engine function, then to a persisted decision and the screen that the athlete or coach actually uses.

Repository reading. The repository's `verify` script and dossier acceptance matrix provide the base. That evidence shows an intentional boundary, not automatically a complete feature. A named type, helper, or unit test establishes a local contract. It does not establish that the function is imported by the live caller, that its result survives offline storage, that the coach and athlete surfaces agree, or that a hard state cannot be bypassed by an alternate route. The audit therefore treats implementation, reachability, integration, and release proof as separate claims.

Evidence status. The scientific conclusion for this mechanism is deliberately narrower than the product ambition. Research can support the direction of a controller, the usefulness of individualisation, the value of observing performance and symptoms, or the danger of false certainty. It usually does not validate the exact constant, confirmation count, UI wording, or threshold selected here. The product may still choose a heuristic when it is bounded, reversible, transparent, and low-risk, but the documentation must call it a heuristic and identify what would change it.

Failure mode. A project can feel finished while one consumer path or safety state remains unproven. The important distinction is between an arithmetic error, a data-quality error, a state-precedence error, and a communication error. Each can produce a different visible result even when the same underlying observation is present. A safe implementation names the failure class, retains the raw input, and avoids converting a missing or incomparable observation into apparent evidence. If the rule cannot tell whether the event was pain, fatigue, equipment failure, or incomplete logging, it should hold or ask for input rather than invent a cause.

State transition. The mechanism should enter a declared state before it changes a prescription. A normal approved path may allow the planned exposure. A held path preserves the current dose while gathering evidence. A reduced path changes one lever and states what purpose remains. A calibration path gathers a new anchor after a meaningful change. A blocked or review path prevents autonomous escalation. The order matters: hard pain or illness constraints outrank ordinary performance progression; missing optional data lowers confidence; one noisy miss does not automatically become a deload.

Numerical discipline. Any number shown to the athlete should retain its units, reference anchor, rounding rule, and actual physical meaning. The engine should distinguish intended percentage from achieved

percentage, prescribed load from logged load, and eased offer from earned baseline. Rounding must happen at the equipment boundary, not earlier in a way that hides the real jump. When a number cannot be represented safely, the correct output is a fallback, hold, or request for setup information—not a false statement that the target was achieved.

User experience. The athlete should see the next useful action, not a lecture. A progression note should identify the evidence and the single changed lever. A hold should say what was protected and what evidence would unlock a change. A reduction should say whether load, volume, density, or complexity moved. A blocked state should be respectful, unambiguous, and impossible to mistake for a motivational challenge. A coach should be able to open the underlying observations and see whether the decision was automatic, athlete-adjusted, coach-authored, or review-bound.

Required validation. Run the checklist at the release commit and record every pass, skip, failure, and owner. The test must exist at the pure function level and at the boundary where the decision reaches the real product surface. Sequence tests are essential because adaptive errors often arise from a prior session, a delayed sync, a stale wearable value, or a previously active hold. The fixture should include valid data, missing data, conflicting data, a user override, and a hard safety flag. The expected result should include action, source, reason codes, safety state, and the next recheck condition.

Audit record. Every decision in this area should retain the engine version, time, subject and exercise identity, session purpose, source observations, comparability status, anchor, equipment step, actual rounded change, data limitations, safety constraints, reason codes, and chosen action. The interface can summarise these fields, but a reviewer must be able to reconstruct the decision without guessing which screen or constant supplied the number. This is the difference between a transparent controller and a black-box recommendation that merely happens to be deterministic.

Closure judgement. Release only when hard-constraint integration, athlete reachability, parity, and deployment smoke are closed. This is strong enough to freeze as the implementation position for the current release, subject to the explicit integration and verification gates in the repository audit. It should not be described as a universal law of training. It is a bounded product policy whose safety depends on truthful missingness, correct state precedence, equipment-aware arithmetic, and an honest handoff when the evidence or authority runs out.

Part IX. Final implementation lock

The deterministic controller that is now approved

The approved baseline is a transparent controller with separate pathways. The strength pathway reads comparable working exposures, excludes warm-ups from the working anchor, evaluates completion and effort, respects equipment reality, and chooses one primary lever. The conditioning pathway keys progression to format and modality, separates earned level from today's recovery adjustment, excludes simulated or data-empty sessions, and keeps provisional thresholds visibly provisional. The whole-athlete pathway collects context and hard constraints but does not allow a composite readiness score to impersonate a diagnosis. The pain and illness pathway outranks ordinary fatigue logic. The data boundary preserves raw facts and decision provenance. The explanation boundary returns the same number and the same reason to every surface.

The chosen values are defaults, not laws: a 2.5% target relative to a last stable opening load; equipment-aware rounding; a 5% reactive reduction from the last successful anchor after repeated comparable deterioration; two comparable successes as a progression debounce; one miss as hold/context; repeated comparable misses as reduction or recheck; and a minimum history gate before high-confidence adaptive suggestions. These values are safe only because they are bounded and because the controller can hold, fall back, calibrate, or abstain. If the application removes those escape states, the constants become materially riskier.

The finite blockers before public closure

The project should not reopen the entire research question. It should close the finite implementation blockers. First, prove that whole-athlete pain and illness hard constraints are consumed by the actual session/prescription path and cannot be bypassed by the mobile logger or a stale synced assignment. Second, prove that the new adaptive strength output reaches the athlete's live opening-load and explanation surface. Third, prove coach/athlete field ownership and idempotent sync with conflict fixtures. Fourth, run the complete repository verification and deployment smoke at the release commit, including browser sections that must not silently skip. Fifth, archive the decision record, test output, repository commit, known limitations, and rollback path.

What would reopen the decision

The rule should reopen only for a named trigger: a safety incident, a reproducible systematic bias, a new direct comparative trial that changes the evidence boundary, a strong product validation signal showing repeated under- or over-progression, a new modality whose measurement semantics do not fit the current controller, or a schema/security change that breaks replay. A single unusually good workout, a single low HRV value, a request for a more aggressive number, or the existence of a new general review is not by itself enough. Reopening should preserve the current baseline while the new evidence is tested.

Final verdict

THE Hybrid Engine is ready to close as a deterministic adaptive-training project once the finite integration gates are green. The science supports individualised, bounded, feedback-informed progression and cautions against false precision. The repository contains the beginnings—and in several areas the substance—of that design: authoritative lift calculations, equipment-aware feasibility, format-specific conditioning, explanation contracts, whole-athlete state, explicit missingness, and layered verification. The remaining work is not to discover a perfect percentage. It is to connect the safety state to the real consumer, prove the live path, preserve the audit trail, and release the bounded controller with its uncertainty honestly labelled.

Preserved file: [hybrid_adaptive_evidence_bundle_2026-08-01.md](#)

THE Hybrid Engine — adaptive-layer evidence bundle

Research date: 1 August 2026

Scope: A local-first, deterministic strength-and-conditioning rules engine. No AI model is assumed.

Purpose: Evidence for the eight proposed adaptive mechanisms, with a clear boundary between tested evidence, adjacent evidence, coaching precedent, and engineering choice.

High-Level Overview

Executive verdict

The evidence supports building a controlled adaptive layer, but it does **not** support pretending that the common thresholds already used in training software are validated science.

The strongest defensible conclusions are:

- RPE/RIR is a useful, conditionally valid input for load prescription, especially for familiar exercises, trained users, and sets close to failure. It is not perfectly accurate, and comparative studies do not establish that RPE/RIR is universally superior to fixed percentages.
- Deloads are common coaching practice, but no study located here validates “two missed sessions,” “three failed sets,” or any other universal consecutive-failure trigger. Exact deload timing, duration, and dose remain under-researched.
- Reliability science supports estimating measurement error and treating small changes as uncertain. It does not provide a universal number of logged sessions or a universal trailing window for e1RM or volume trends.
- HRV can sometimes help redistribute endurance training, but evidence for same-day strength-load modulation is small and mixed. HRV is not a direct measure of local muscle, tendon, pain, technique, or injury readiness.
- Short training breaks do not imply a validated percentage reduction. A return after a layoff should be a conservative calibration exposure, not an automatic “one week off equals 10% down” formula.
- Pain-monitoring evidence is useful in specific rehabilitation conditions, especially tendinopathy, but it does not validate a general traffic-light model or a universal pain threshold for recreational training.
- Uncertainty communication is evidence-informed as a discipline of calibration, transparency, data coverage, and action-linked abstention. A generic confidence percentage or red/amber/green badge is not itself validated.
- MacroFactor and Juggernaut/RTS provide valuable product and coaching precedent. Their public documentation does not validate THE Hybrid Engine, and the proprietary portions of their algorithms cannot be reproduced from the public material.

Evidence hierarchy used for every claim

1. **Peer-reviewed direct evidence:** a study on the exact population and mechanism in question.
2. **Peer-reviewed adjacent evidence:** informs the mechanism but is not directly transferable to the proposed app rule.
3. **Established coaching methodology:** a named, published or creator-documented system such as RTS, the Juggernaut Method, or DUP. This is practice precedent, not proof.
4. **Product-design convention:** an explicit engineering choice made because direct evidence is thin, such as a hold state, conservative gate, or data-completeness policy.

“Not found” means that this review did not find a source validating the exact rule. It does not prove that no unpublished or inaccessible study exists.

Decision summary

Mechanism	Evidence status	Safe interpretation for the app
2. Deload and repeated-failure thresholds	Sparse for deload efficacy; exact thresholds not found	Use repeated, corroborated deterioration rather than a universal miss count; label any count as a product heuristic.
3. Minimum data for trends	Reliability principles supported; exact window not found	Estimate individual error, standardize exposures, and hold when the data cannot distinguish signal from noise.
4. HRV/recovery modulation	More support in endurance than strength; HRV-only gate unsupported	Use HRV as one advisory signal, never as an autonomous safety or tissue-readiness oracle.
5. Return after layoff	Detraining effects are duration-, population-, and modality-dependent	Use a conservative re-entry session and observe current RPE/performance; no universal percentage formula.
6. Pain versus fatigue	Narrow rehab evidence; general app rule not validated	Separate pain from exertional fatigue, stop or modify safely, and block automatic progression until the symptom state is clarified.
7. Confidence and uncertainty	Communication/calibration principles supported; specific UI not validated	Show what is estimated, the evidence window, missing data, reasons, and the action taken; abstain when uncertainty matters.
8. Juggernaut/RTS and MacroFactor	Strong precedent; not independent validation	Borrow the feedback-loop pattern, not their marketing claims or undisclosed equations.

Deep Dive Analysis

1. RPE/RIR-based autoregulation for load prescription

Evidence table

Evidence record 1

Claim: RPE/RIR can produce repeatable load prescriptions in some lifters.

What the evidence actually supports: Small repeated-measures studies found good repeatability when participants were trained or familiarized, but repeatability is not the same as objective accuracy.

Recommended app rule: Track user-specific RPE bias over time. Do not assume that a new user's RPE 8 has the same meaning as a trained user's RPE 8.

Evidence status/citation: Peer-reviewed direct evidence. Helms et al. (2017), *JSCR*, n=12; Lovegrove et al. (2022), *JSCR*, n=15. Helms DOI, Lovegrove DOI

Evidence record 2

Claim: RIR estimates are less reliable when many repetitions remain or the exercise/set is unfamiliar.

What the evidence actually supports: Participants can misjudge how many repetitions remain. Accuracy generally improves nearer failure and with experience; pooled evidence is heterogeneous.

Recommended app rule: For high-repetition, novel, technically complex, or low-confidence sets, use RPE/RIR as advisory and require performance, technique, symptoms, or completion data before progressing.

Evidence status/citation: Peer-reviewed adjacent evidence. Mansfield et al. (2020), n=20; Hughes et al. (2020), n=21; Steele et al. (2017), n=141; Halperin et al. (2022), 16 publications and 414 participants in the exploratory meta-analysis. Mansfield DOI, Hughes DOI, Steele DOI, Halperin DOI

Evidence record 3

Claim: RPE/RIR is superior to fixed percentages for long-term strength.

What the evidence actually supports: Some trials favor RIR, but the RIR groups often trained at a higher achieved intensity. Other trials show no between-group difference. Meta-analyses do not establish a clear RPE-specific advantage.

Recommended app rule: Do not make “RPE is scientifically superior” a product claim. Support both percentage-based and RPE/RIR-based prescriptions, with RPE/RIR allowed to correct stale 1RM estimates and day-to-day variation.

Evidence status/citation: Peer-reviewed direct and adjacent evidence is mixed. Helms et al. (2018), *Frontiers in Physiology*, n=21, found no significant between-group strength or hypertrophy difference; Graham & Cleather (2021), *JSCR*, n=31, favored RIR but with higher achieved intensity; Hickmott et al. (2022), 15 studies, and Zhang et al. (2021), 8 studies/166 athletes, do not establish RPE superiority. Helms full text, Graham DOI, Hickmott DOI, Zhang full text

Evidence record 4

Claim: RPE/RIR can be the primary within-session adjustment signal.

What the evidence actually supports: The literature supports it conditionally, not universally: the strongest transfer is to familiar resistance exercises in trained users and sets within a modest distance of failure.

Recommended app rule: Use it as the primary *within-session* adjustment input where the context is familiar and data quality is adequate; require corroboration for high-rep, novel, painful, or technically unstable work.

Evidence status/citation: Peer-reviewed adjacent evidence translated into a product rule. The conditional boundary is evidence-informed; the exact app routing is Product-design convention.

Counterpoints and evidence gaps

- Fixed percentages are not automatically more objective: a stale 1RM and changing daily readiness can make a percentage misleading.

- RPE/RIR is not automatically subjective noise: it can be repeatable and useful, but the user's calibration must be learned.
- The evidence base is concentrated in trained men, powerlifters, laboratory protocols, and familiar exercises. It does not prove the same performance for recreational adults, mixed-modality sessions, pain-affected movements, or unsupervised beginners.
- No direct study located here validates a complete deterministic rule such as "RPE 7 for two sessions means add 2.5%." The adjustment size and exposure count remain product choices.
- No direct study located here validates a universal RPE/RIR mapping across all exercises, rep ranges, modalities, devices, or users.

2. Deloading and repeated-failure thresholds

Evidence table

Evidence record 1

Claim: Training to failure creates fatigue and short-term performance impairment.

What the evidence actually supports: Meta-analytic evidence supports acute fatigue and performance costs from failure training, but it does not convert those effects into a multi-session deload threshold.

Recommended app rule: Use failure exposure as one fatigue-context feature, not as a standalone automatic deload trigger.

Evidence status/citation: Peer-reviewed adjacent evidence. Vieira et al. (2022), systematic review/meta-analysis. PubMed

Evidence record 2

Claim: Deloads are commonly used every four-to-eight weeks for about a week.

What the evidence actually supports: A survey of 246 competitive strength and physique athletes reported typical deloads of 6.4 ± 1.7 days every 5.6 ± 2.3 weeks. It describes practice, not optimal timing or causal benefit.

Recommended app rule: Treat 5–7 days and roughly 4–8 weeks as plausible starting parameters only when a programme needs a scheduled recovery microcycle; expose them as configurable and testable.

Evidence status/citation: Peer-reviewed adjacent evidence. Rogerson et al. (2024), *Sports Medicine–Open*, n=246. The authors explicitly call for empirical evaluation. Full text

Evidence record 3

Claim: Expert consensus supports deloading as a reduction in training stress to manage fatigue and preparedness.

What the evidence actually supports: Consensus defines the practice and offers terminology; consensus cannot establish that one timing, load reduction, or volume reduction is optimal.

Recommended app rule: Encode a deload as a state that reduces training stress while preserving the session purpose where safe. Do not present consensus as efficacy evidence.

Evidence status/citation: Established coaching methodology / peer-reviewed consensus. Bell et al. (2023), international Delphi consensus. PMC

Evidence record 4

Claim: A planned deload at a particular point preserves adaptation.

What the evidence actually supports: A 2026 within-subject study in 19 untrained young men found that a specific deload arrangement during weeks 4 and 8 did not hinder hypertrophy or 10RM strength-endurance. It did not test trained athletes, reactive triggers, mixed modalities, or the best timing. Other studies have found different strength outcomes.

Recommended app rule: Use the study as narrow reassurance that a particular planned reduction need not erase progress—not as a universal deload schedule.

Evidence status/citation: Peer-reviewed direct evidence, narrow population and protocol. Pancar et al. (2026), *Scientific Reports*, n=19. Full text; see also Coleman et al. (2024), *PeerJ*, PMC

Counterpoints and evidence gaps

- “Deload” is not one intervention: it can reduce sets, repetitions, load, effort, frequency, or some combination. Studies that use different definitions cannot answer the same question.
- A scheduled deload and a reactive response to repeated failures are different mechanisms.
- Reducing training because of pain is not the same as deloading for accumulated fatigue; pain may require a safety or clinical pathway.
- The exact frequency, duration, and magnitude of deloading remain unresolved. No dose-response study located here establishes an optimal rule for recreational adults.
- A “two missed sessions” rule may be useful for a product’s workflow, but it must be labelled as a conservative operational heuristic, not a physiological threshold.

3. Minimum data before trusting a trend

Evidence table

Evidence record 1

Claim: Reliability statistics can inform whether a change is meaningful.

What the evidence actually supports: Typical error, standard error of measurement, coefficient of variation, limits of agreement, and minimum detectable change are more informative than a raw correlation alone.

Recommended app rule: For each exercise or metric, estimate repeatability where data allow; suppress automatic progression when change is smaller than the estimated error band.

Evidence status/citation: Peer-reviewed adjacent evidence. Hopkins (2000), *Sports Medicine*; Atkinson & Nevill (1998), *Sports Medicine*.

Evidence record 2

Claim: 1RM/e1RM reliability is transferable across exercises and populations.

What the evidence actually supports: Reliability varies with exercise, method, loading, standardization, population, and familiarity. A reliable laboratory 1RM protocol does not validate a universal app-derived e1RM formula.

Recommended app rule: Keep e1RM modality- and exercise-specific. Record formula/version, reps, load, RPE/RIR, technique/context, and whether the set was valid for trend use.

Evidence status/citation: Peer-reviewed adjacent evidence. Grgic et al. (2020), systematic review of 1RM reliability. PubMed

Evidence record 3

Claim: A three-point moving average proves a three-session trend window.

What the evidence actually supports: Moving averages can reduce noise in a particular measurement context, but a three-point average studied for a specific resistance-training signal is not a universal longitudinal e1RM rule.

Recommended app rule: Use a declared smoothing function as a product parameter, validate it against historical replay and user-level error, and expose the number and quality of observations behind a decision.

Evidence status/citation: Peer-reviewed adjacent evidence. Chiu & Salem (2010) studied a specific three-point moving-average context. DOI

Evidence record 4

Claim: Two or three successful comparable exposures are enough to trust progression.

What the evidence actually supports: No direct source located here validates “two,” “three,” or “3–5” exposures as a universal minimum for e1RM or volume trends.

Recommended app rule: Keep 2–3 or 3–5 exposures only as a transparent, configurable product confirmation heuristic. Mark the resulting confidence as heuristic, not scientifically validated.

Evidence status/citation: Product-design convention. Evidence gap explicitly retained.

Counterpoints and evidence gaps

- More data do not automatically mean better data if the exercise, range of motion, device, rest interval, or technique changes.
- A short window reacts quickly but is noisy; a long window is stable but can hide a genuine change. This is a control-system trade-off, not a known physiological constant.
- A trend threshold should be tied to the decision cost. A small uncertain improvement may justify “hold,” while a suspected safety deterioration may justify a conservative reduction immediately.
- No validated universal minimum exists for mixed strength/conditioning data, wearable data, device-derived power, or self-reported RPE.

4. Recovery and HRV-based same-day load modulation

Evidence table

Evidence record 1

Claim: HRV-only modulation is superior for strength training.

What the evidence actually supports: Small strength studies do not establish superiority. Examples include a trial in 20 young men and a trial in 21 older women reporting no clear HRV-guided advantage for strength, hypertrophy, or function.

Recommended app rule: Do not allow HRV alone to increase load, volume, intensity, or exercise risk. An isolated low HRV should usually produce an uncertainty flag, not an automatic demotion; a corroborated deterioration can cap intensity.

Evidence status/citation: Peer-reviewed direct evidence is limited and mixed. de Oliveira et al. (2019), n=20, DOI; Bittencourt et al. (2024), n=21, DOI

Evidence record 2

Claim: HRV is a direct measure of local muscle, tendon, pain, or injury readiness.

What the evidence actually supports: HRV reflects autonomic regulation and is affected by measurement protocol, sleep, illness, stress, posture, breathing, and other factors. It did not distinctly track local strength recovery or creatine kinase in a small resistance-exercise study.

Recommended app rule: Never use HRV as a medical, injury, or tissue-capacity gate. Keep pain, symptoms, technique, performance, and user-reported recovery as separate fields.

Evidence status/citation: Peer-reviewed adjacent evidence. Thamm et al. (2019), n=10 trained men; Bellenger et al. (2016). Thamm DOI

Evidence record 3

Claim: Multi-signal readiness is validated for a mixed recreational app.

What the evidence actually supports: Combining HRV with perceived recovery, resting heart rate, performance, sleep, and well-being is more plausible than HRV-only control and appears in individualized studies, but no validated fusion algorithm for recreational mixed strength/conditioning was found.

Recommended app rule: Use a transparent multi-signal evidence record rather than a black-box readiness score. Show which signals were present, stale, missing, or contradictory.

Evidence status/citation: Peer-reviewed adjacent evidence plus product-design convention. Nuuttila et al. (2017), individualized endurance training; Alfonso et al. (2025), n=28 experienced cyclists, combined signals. Nuuttila DOI, Alfonso full text

Evidence record 4

Claim: A consumer device's absolute HRV threshold transfers across users and devices.

What the evidence actually supports: No universal cross-device or cross-user threshold was found. Personal baselines and protocol consistency are necessary because measurement properties and context vary.

Recommended app rule: Normalize within user and device/protocol; require a minimum valid baseline before using HRV directionally; hold or downgrade confidence when measurement quality is poor.

Evidence status/citation: Not found for a universal threshold. Product-design convention informed by adjacent HRV-monitoring literature.

Counterpoints and evidence gaps

- HRV may help decide *when to distribute hard work* without telling the system whether a specific joint, tendon, or muscle is safe.

- A lower HRV can reflect a useful training stimulus, illness, sleep loss, measurement artifact, or unrelated stress. The same number can have different meanings.
- Wearable readiness scores may combine proprietary inputs and may not expose raw HRV, preprocessing, artifact rejection, or baseline logic.
- No adequately powered trial located here compares fixed programming, HRV-only control, and multi-signal control in recreational mixed strength/conditioning.
- No validated universal rule says “red HRV means skip,” “green HRV means progress,” or “HRV below X means reduce load by Y%.”

5. Return to training after a layoff

Evidence table

Evidence record 1

Claim: Three or more weeks off implies a known percentage reduction.

What the evidence actually supports: Some narrow studies report retained strength or muscle thickness after several weeks, while broader detraining reviews show that effects vary with training status, duration, modality, and outcome.

Recommended app rule: Use time off as a caution multiplier for confidence and volume, not a deterministic load equation.

Evidence status/citation: Peer-reviewed adjacent evidence. Ogasawara et al. (2013), PubMed; Gavanda et al. (2020), PMC; Encarnação et al. (2022), systematic review, MDPI

Evidence record 2

Claim: Detraining is the same across strength, cycling, rowing, and technical exercises.

What the evidence actually supports: Cardiorespiratory, strength, hypertrophy, skill, and connective-tissue responses have different time courses. Evidence from one modality cannot be used as a cross-modality conversion table.

Recommended app rule: Keep layoff handling modality-specific. A return after a cycling gap must not automatically prescribe a barbell or rower reduction using the same formula.

Evidence status/citation: Peer-reviewed adjacent evidence translated into a product boundary. Mujika & Padilla (2000) and later detraining literature support modality- and outcome-specific interpretation. Part I PubMed

Evidence record 3

Claim: There is a validated “one week = 10% down” or “three weeks = 20% down” rule.

What the evidence actually supports: No validated time-off-to-percentage-load equation was found. Such percentages are coaching or product conventions.

Recommended app rule: First session after a gap: reduce complexity and/or volume conservatively, cap effort, observe technique and RPE/RIR, then update from current evidence and next-day response.

Evidence status/citation: Not found. Product-design convention only.

Counterpoints and evidence gaps

- Retained 1RM does not guarantee normal readiness for volume, conditioning, coordination, or connective-tissue loading.
- A layoff caused by travel is not equivalent to one caused by illness, injury, immobilization, severe sleep loss, or medication change.
- The first session after a gap is both training and measurement. Making it a maximal test can destroy the information needed for safe re-entry.
- No universal percentage rule was found across trained status, age, exercise, body region, modality, or reason for absence.
- Injury, surgery, illness, or unexplained symptoms require a separate clinical return-to-training pathway; the rules engine must not improvise one.

6. Pain versus non-painful fatigue

Evidence table

Evidence record 1

Claim: A 5/10 pain threshold is a universal safe training rule.

What the evidence actually supports: The threshold belongs to specific tendinopathy rehabilitation protocols. A pilot in patellar tendinopathy and broader reviews do not validate a universal threshold across diagnoses or recreational training.

Recommended app rule: Never display “5/10 is safe” as a general law. Ask about location, quality, onset, function, progression, and next-day response; route red flags out of autonomous training.

Evidence status/citation: Peer-reviewed adjacent evidence; universal rule not found. Sprague et al. (2021), n=15 pilot, DOI; Tran et al. (2025), 16 trials/1,026 participants, low/very-low certainty, DOI

Evidence record 2

Claim: Pain and exertional fatigue can be treated as the same signal.

What the evidence actually supports: Pain, fatigue, soreness, technique breakdown, and breathlessness have different meanings and cannot be safely collapsed into one RPE-like field. Return-to-sport criteria are inconsistent and often not operationalized.

Recommended app rule: Store pain/symptoms separately from effort and fatigue. Pain or neurological/cardiopulmonary red flags must block automatic progression; non-painful fatigue can be handled by conservative repeat/hold logic when safe.

Evidence status/citation: Peer-reviewed adjacent evidence plus product-design convention. Habets et al. (2018) reviewed 35 Achilles return-to-sport studies and found only 19 described criteria, with inconsistent operational cutoffs. DOI

Evidence record 3

Claim: A general traffic-light pain model is validated for this app.

What the evidence actually supports: Clinical traffic-light tools and pain-monitoring models are useful practice precedents, but no general validation across injuries, exercises, modalities, and recreational users was found.

Recommended app rule: A traffic-light state can be implemented as a conservative safety UX, explicitly labelled product policy. It must not diagnose or grant medical clearance.

Evidence status/citation: Product-design convention. The clinical precedent is condition-specific; general app validation was not found.

Evidence record 4

Claim: Holding a prescription until the user explicitly acknowledges a pain-stop event is evidence-based.

What the evidence actually supports: No study located here validates an app acknowledgement gate. It may be a defensible safety interaction because it prevents silent progression after an unresolved symptom, but that is an engineering decision.

Recommended app rule: After a pain-stop, block automatic progression for the affected movement/pattern until the user records a symptom update; escalate red flags. Label the gate as a safety policy, not science.

Evidence status/citation: Not found. Product-design convention only.

Counterpoints and evidence gaps

- Blanket “never train with pain” may be too conservative for diagnosed, supervised rehabilitation, while generic “pain is okay” is unsafe for an app that cannot diagnose.
- Pain intensity alone is insufficient: location, quality, function, trajectory, and next-day response matter.
- Pain may be absent while capacity is still reduced, and pain may persist without indicating tissue damage; the rules engine must not infer a diagnosis.
- No general traffic-light threshold, acknowledgement gate, or automatic substitution rule was validated for mixed recreational strength and conditioning.
- Any chest pain, fainting/dizziness, acute traumatic event, neurological symptoms, severe shortness of breath, or rapidly worsening pain should leave the autonomous training pathway.

7. Confidence-scoring and uncertainty communication

Evidence table

Evidence record 1

Claim: Appropriate reliance is safer than maximizing trust.

What the evidence actually supports: Human-automation research distinguishes appropriate reliance from simple trust. Explanations can reduce or increase over-reliance depending on task difficulty and whether the user can verify the recommendation.

Recommended app rule: Make verification cheap: show the exact drivers and what would change the decision. Use a hold/ask-more-data state for high-impact uncertainty instead of persuasive prose.

Evidence status/citation: Peer-reviewed adjacent evidence. Lee & See (2004); Bussone et al. (2015); Vasconcelos et al. (2023). Lee & See, Bussone, Vasconcelos

Evidence record 2

Claim: Ranges, frequencies, denominators, and context can improve interpretation of uncertain outcomes.

What the evidence actually supports: Risk-communication research supports consistent formats, explicit denominators, and context; different displays work differently depending on task and numeracy.

Recommended app rule: Prefer a plain-language range or evidence statement to a pseudo-precise scalar. Test the UI with the actual training decision rather than assuming more detail is safer.

Evidence status/citation: Peer-reviewed adjacent evidence. Trevena et al. (2013) consensus primer; Gigerenzer & Hoffrage (1995); Fernandes et al. (2018) visualization study. Trevena, Gigerenzer, Fernandes

Evidence record 3

Claim: Data coverage and freshness should be visible.

What the evidence actually supports: Clinical decision-support and HCI work supports understandability, evidence, and actionability as conditions for appropriate reliance. It does not establish one completeness threshold for this app.

Recommended app rule: Show “based on X comparable sessions,” “last valid HRV,” missing fields, conflicts, and whether the estimate is observed or inferred.

Evidence status/citation: Peer-reviewed adjacent evidence translated into a product rule. Exact labels and thresholds are Product-design convention.

Evidence record 4

Claim: Uncertainty should cause a useful action rather than merely lower a score.

What the evidence actually supports: Uncertainty-aware systems can abstain or target cases for review. However, friction and uncertainty displays can also reduce satisfaction or cause avoidance.

Recommended app rule: Map uncertainty to approved, held, reduced, blocked, needs_input, or needs_clinical_review. Do not let a low number silently change the prescription.

Evidence status/citation: Peer-reviewed adjacent evidence. Kompa et al. (2021) on abstention/uncertainty in medical ML; Kang et al. (2021) on uncertainty-guided review; Politi et al. (2011) on uncertainty trade-offs. Kompa, Kang, Politi

Evidence record 5

Claim: A red/amber/green confidence badge improves safety or adherence.

What the evidence actually supports: No direct study located here validates a particular badge, colour, threshold, smoothing rule, or effect on training outcomes.

Recommended app rule: If colours are used, pair them with text, reason codes, data coverage, and the action. Validate appropriate reliance and decision quality, not just perceived trust.

Evidence status/citation: Not found. Product-design convention only.

Counterpoints and evidence gaps

- Uncertainty is not cost-free: it can lower satisfaction, create avoidance, or increase acceptance without improving decisions.

- A calibrated model can still prescribe a bad action; calibration of the estimate is not validation of the policy.
- A user may over-trust a persuasive explanation or under-trust a correct conservative recommendation.
- No study located here establishes that a confidence UI in a consumer training app improves adherence, safety, performance, or health outcomes.
- The product should validate calibration, missing-data robustness, over-reliance, under-reliance, and whether the hold action is understood.

8. Juggernaut Method, RTS, and MacroFactor

Evidence table

Evidence record 1

Claim: MacroFactor deliberately smooths noisy data and avoids reacting to one or a few abnormal observations.

What the evidence actually supports: Its public material describes trend weight, recent weighting, gradual weekly adjustments, and a data-sufficiency hold. The exact V3 equations and smoothing parameters are not public.

Recommended app rule: Use smoothing and damped updates as a product precedent. Store the raw observations alongside the estimate and expose the reason for a hold or update.

Evidence status/citation: Product-design convention. Weight Trend, V3 algorithm, weekly adjustment logic

Evidence record 2

Claim: MacroFactor's minimum logging requirements are universal scientific minimums.

What the evidence actually supports: Public requirements such as nutrition on at least 4 of 7 days and weight at least once weekly are operational product gates, not universal statistical or physiological guarantees.

Recommended app rule: Treat minimum data thresholds as versioned product parameters. Test them on the Hybrid Engine's own metrics; do not cite MacroFactor's thresholds as science.

Evidence status/citation: Product-design convention. Expenditure version and requirements, logging frequency

Evidence record 3

Claim: MacroFactor has independently validated true TDEE measurement.

What the evidence actually supports: MacroFactor's public accuracy material describes prediction of observed weight change from estimated expenditure and intake. That is predictive validity in its own analysis, not direct validation against chamber calorimetry or doubly labelled water.

Recommended app rule: Describe MacroFactor as product precedent. Do not claim that its current estimate is a gold-standard measurement or that its internal accuracy analysis transfers to training-state estimation.

Evidence status/citation: Product-design convention; independent direct validation not found in the reviewed public material. Algorithm accuracy

Evidence record 4

Claim: JuggernautAI is a reproducible, peer-reviewed algorithm.

What the evidence actually supports: The official product pages describe individualized inputs, proprietary algorithms, and feedback across sets, sessions, weeks, blocks, and programmes. They do not disclose equations, weights, thresholds, or a full decision table.

Recommended app rule: Borrow the idea of structured programming with feedback, but keep THE Hybrid Engine's rules explicit, versioned, deterministic, and auditable.

Evidence status/citation: Product-design convention. JuggernautAI official description, official app site

Evidence record 5

Claim: RTS and the original Juggernaut Method are scientific validation.

What the evidence actually supports: RTS documents RPE/RIR targets, fatigue percentages, lift-specific charts, performance downturns, and autoregulated adjustments as coaching methodology. The Juggernaut Method documents wave progression, working-max updates, and rep-record feedback. These are practice precedents, not controlled validation of the complete systems.

Recommended app rule: Use them to generate candidate rule families and terminology. Mark every adopted rule as coaching methodology or product design unless a direct study supports it.

Evidence status/citation: Established coaching methodology. RTS materials: Beginning RTS, fatigue percentages, performance downturns. Juggernaut: official method article, book record

Evidence record 6

Claim: The Hybrid Engine should copy either product wholesale.

What the evidence actually supports: MacroFactor is nutrition/weight feedback; Juggernaut and RTS are primarily resistance-training coaching systems. Their domains, measures, and proprietary choices differ from a mixed strength/conditioning app.

Recommended app rule: Combine principles, not code or claims: modality-specific metrics, structured plan, feedback, smoothing, confidence, hold states, reason codes, and deterministic authority.

Evidence status/citation: Product-design convention informed by established coaching methodology. Cross-modality equivalence remains unsupported.

Counterpoints and evidence gaps

- Product documentation is evidence of what a creator says the product does, not independent validation of efficacy or generalizability.
- MacroFactor's public documentation explicitly leaves important V3 implementation details proprietary; no responsible audit can reconstruct its exact formula from marketing or help pages.
- JuggernautAI's current product should not be treated as identical to the original Juggernaut Method.
- RTS and Juggernaut are strongest as named coaching precedents for autoregulation and periodization, not as evidence that every threshold or progression rule is optimal for recreational mixed-modality users.
- No peer-reviewed study located here validates the complete RTS system, the complete original Juggernaut Method, or the current JuggernautAI algorithm in recreational self-directed adults.

Counterpoints/Challenges

The strongest objections to the common assumptions

1. **“If it is common in coaching, it must be validated.”** Common practice can be useful and still be a heuristic. The bundle should retain named coaching precedent while refusing to upgrade it into direct science.
2. **“More signals automatically make the decision more accurate.”** More signals can add noise, missingness, contradictions, and false precision. The engine should show signal quality and conflicts, not just average them into a larger score.
3. **“The number of sessions is the evidence.”** Three noisy, non-comparable sessions are not stronger than one high-quality standardized observation. Count, comparability, error, and context must be separate fields.
4. **“RPE/RIR solves day-to-day variation.”** It can help, but users can misestimate effort and may need familiarization. RPE is a measurement with error, not a truth oracle.
5. **“HRV is readiness.”** HRV is autonomic information, not local tissue capacity, pain status, or medical clearance.
6. **“Pain is either harmless or an automatic stop.”** Both extremes are unsafe. Some diagnosed rehabilitation protocols permit monitored pain; a general app cannot infer the diagnosis or transfer the protocol.
7. **“A confidence score makes the system safe.”** An uncalibrated score can create false reassurance. Safety comes from calibrated estimates, explicit limits, abstention, reason codes, and appropriate user action.
8. **“MacroFactor/Juggernaut proved the architecture.”** They provide compelling precedents, but the Hybrid Engine has different outcomes, modalities, data quality, and safety boundaries.

Evidence gaps that must remain visible

- No validated universal consecutive-failure or missed-session threshold.
- No validated universal e1RM/volume trend window or minimum number of exposures.
- No validated universal HRV cutoff, cross-device threshold, or HRV-only strength gate.
- No validated time-off-to-percentage-load-reduction equation.
- No validated general traffic-light pain model for recreational mixed training.
- No validation of a post-pain-stop acknowledgement gate.
- No evidence that a consumer training confidence meter improves adherence, performance, or safety.
- No independent, reproducible validation of MacroFactor’s current V3 expenditure algorithm against a gold-standard expenditure measure in the public sources reviewed.
- No peer-reviewed validation of the complete RTS system, complete original Juggernaut Method, or current JuggernautAI algorithm in recreational self-directed adults.

Actionable Next Steps

Evidence-safe rules to implement first

1. **Preserve the deterministic authority.** Every adaptive output should include the engine version, state, action, reason codes, data-quality flags, and the observed inputs used.

2. **Separate observed data from estimated state.** Keep raw sets, reps, load, RPE/RIR, symptoms, sleep, HRV, device metrics, and completion status separate from e1RM, fatigue, readiness, and trend estimates.
3. **Use RPE/RIR conditionally.** Require familiarization and learn user-specific bias. Use corroboration for high-repetition, novel, painful, or technically unstable work.
4. **Replace hard-coded folklore thresholds with explicit heuristics.** If the app uses “two misses,” “three exposures,” “5–7 days,” or a percentage reduction, label it `product\_heuristic`, version it, and make it testable.
5. **Use corroborated deterioration.** A repeated performance decline beyond estimated error plus fatigue, pain, readiness, or technique evidence is stronger than a single failed set.
6. **Make HRV advisory.** Normalize within user and protocol; never allow HRV alone to escalate training or authorize loading through pain.
7. **Make layoff return a calibration state.** Lower complexity and/or volume conservatively, cap effort, observe current RPE/RIR and technique, and progress only from new evidence. Route illness, injury, surgery, and red flags outside the autonomous engine.
8. **Keep pain separate.** After a pain-stop, stop or modify the affected movement, record structured symptoms, block automatic progression for that movement/pattern, and require an explicit symptom update. Label this as safety policy, not validated science.
9. **Use confidence as an action-linked state, not decoration.** Prefer `approved`, `held`, `reduced`, `blocked`, `needs\_input`, and `needs\_clinical\_review` over an unexplained percentage.
10. **Validate the policy itself.** Replay difficult cases: missing data, contradictory devices, poor sleep, pain, illness, repeated failure, long layoff, false-positive HRV, overestimated RPE, and a desire to make up missed work.

Minimum evidence tests for the rules engine

- ****Determinism:**** identical versioned input produces identical output.
- ****Missingness:**** missing RPE, HRV, sleep, heart rate, or device data cannot be silently treated as normal.
- ****Conflicts:**** contradictory device and user data produce a visible conflict state, not an averaged fiction.
- ****Pain:**** pain never becomes “fatigue” merely because the user wants to continue.
- ****No overreaction:**** one bad session does not force a large progression or demotion unless a safety rule is triggered.
- ****Trend error:**** changes within the user’s estimated measurement error do not cause automatic progression.
- ****Layoff:**** first return session is conservative and informative; the engine does not apply an invented percentage law.
- ****Modality separation:**** cycling, Air Bike, rowing, SkiErg, and barbell metrics cannot be silently exchanged.
- ****Recovery:**** HRV-only changes cannot escalate intensity; multi-signal contradictions reduce confidence.
- ****Missed work:**** the engine does not stack excessive work to “make up” missed sessions.
- ****Auditability:**** every change has a reason code and evidence status.

Final recommendation

Build the adaptive layer now, but build it as a **versioned, deterministic state estimator and bounded decision policy**. Use RPE/RIR, smoothing, multi-signal context, and explicit hold states. Do not market any exact session count, deload schedule, HRV cutoff, pain threshold, or layoff percentage as validated science unless a future study directly tests that rule.

The first valuable version is not an AI coach. It is a transparent engine that can say:

> “Hold progression. The last two comparable exposures are within measurement noise; HRV is missing; readiness is lower; no pain was reported. The session purpose is preserved, but there is not enough reliable evidence to increase the dose.”

That statement is scientifically honest, operationally useful, and testable.

Human Logic Layer — bounded coaching judgment

Evidence status and scope

This section defines the intended **human-like behaviour** of the product. It is not presented as a validated physiological model. The evidence bundle supports the need for conservative, explainable, uncertainty-aware decisions; the specific interaction rules below are **product-design conventions and safety policies**.

“Human logic” must not mean an unconstrained AI coach, a hidden intuition score, or permission to override the deterministic engine. It means that the engine considers the athlete’s real situation and chooses the next useful move while respecting hard safety limits.

The rule engine remains authoritative:

```

Observed data + athlete context
↓
Training-state estimate with uncertainty
↓
Bounded human-logic policy
↓
Validated session decision
↓
Deterministic explanation
  
```

No AI model is required for this layer. The first implementation should use typed inputs, ordered rules, reason codes, and deterministic message templates. A future AI coach may translate the result into natural language, but it must not bypass or rewrite the decision.

What the human-logic layer is trying to preserve

The system should behave like a calm, experienced coach who understands that the athlete is a person living a real life:

- Preserve the purpose of the session where safely possible.
- Account for sleep, energy, stress, soreness, pain, illness, work, time and equipment constraints.
- Distinguish “the athlete feels flat” from “the movement is painful” and from “the device data is unreliable.”
- Avoid cancelling a useful session because of one soft negative signal.

- Avoid progressing because of one unusually good signal.
- Reduce the dose before abandoning the training habit when the athlete has a difficult day.
- Never shame, punish, or make the athlete repay missed training with stacked volume.
- Give the athlete the next useful move, not merely a score or a warning.
- Explain what was observed, what was inferred, what changed, and what would change the decision next time.

Decision priority order

The policy must evaluate signals in this order. Lower-priority context cannot override a higher-priority safety state.

1. **Hard safety gate:** chest pain, fainting, serious dizziness, acute injury, severe illness symptoms, dangerous technique, or other configured red flags. Stop or block the session and route to appropriate professional/urgent guidance. Do not diagnose.
2. **Pain and symptom state:** pain is not interchangeable with exertional fatigue. Stop, modify, or substitute the affected movement as appropriate; do not automatically progress that movement until the symptom state is updated.
3. **Immediate training quality:** technique, control, completion, local failure, unusual RPE, and whether the intended work was actually performed.
4. **Session purpose:** strength, hypertrophy, aerobic base, threshold, intervals, skill, recovery, or minimum viable maintenance. Preserve the purpose with the smallest safe dose.
5. **Real-world constraints:** available time, equipment, work or life stress, motivation, and willingness to train today.
6. **Recent comparable evidence:** performance trends, RPE/RIR calibration, completed volume, missed work, and repeated deterioration.
7. **Advisory signals:** HRV, wearable recovery scores, one poor night of sleep, one unusual device reading, or one low-motivation report. These may lower confidence or support a hold; they must not independently escalate, block, or authorise risky loading.

Context-to-action policy

Evidence record 1

Situation: Readiness is low but performance and technique are acceptable

Human interpretation: The athlete may complete the purpose, but the data does not justify escalation

Default engine action: Complete or hold the current dose; do not progress

What the engine must not do: Treat low readiness as proof of incapacity

Evidence record 2

Situation: Readiness and performance are both poor, without pain

Human interpretation: The planned dose is probably too expensive today

Default engine action: Reduce intensity, volume, density, or complexity by one controlled lever; offer a minimum viable or recovery session

What the engine must not do: Force the full plan or declare the athlete failed

Evidence record 3

Situation: One poor session with otherwise stable history

Human interpretation: This may be noise, context, equipment error, or a bad day

Default engine action: Hold, gather context, and repeat or proceed conservatively

What the engine must not do: Trigger a large demotion or deload from one observation

Evidence record 4

Situation: Good readiness but poor performance

Human interpretation: The signals conflict

Default engine action: Hold progression, inspect technique, sleep, equipment, data quality and recent load

What the engine must not do: Assume the readiness score is correct and push harder

Evidence record 5

Situation: Poor readiness but unexpectedly strong performance

Human interpretation: The signals conflict in the opposite direction

Default engine action: Complete the planned purpose if safe, but avoid automatic escalation

What the engine must not do: Treat one good session as proof that recovery concerns are irrelevant

Evidence record 6

Situation: The athlete has limited time

Human interpretation: A shortened useful session is better than an all-or-nothing failure

Default engine action: Keep the warm-up and highest-value primary work; remove accessories or finishers first

What the engine must not do: Cram missed work or randomly cut the main stimulus

Evidence record 7

Situation: The athlete is unmotivated but physically safe

Human interpretation: The barrier may be mental energy, not physical inability

Default engine action: Offer a minimum viable session with a clear stopping point

What the engine must not do: Shame the athlete or prescribe punishment

Evidence record 8

Situation: A session was missed

Human interpretation: The plan must move forward, not collect debt

Default engine action: Roll to the next useful session; preserve recovery spacing

What the engine must not do: Stack sessions, double volume, or “make up” everything

Evidence record 9

Situation: Several sessions were missed or there was a long gap

Human interpretation: Current capacity is uncertain

Default engine action: Enter a calibration state; use a conservative, informative exposure

What the engine must not do: Apply an invented universal percentage reduction

Evidence record 10

Situation: Pain appears in a movement

Human interpretation: Pain requires a separate symptom pathway

Default engine action: Stop, modify or substitute; record the symptom; block automatic progression for that movement until reviewed

What the engine must not do: Re-label pain as normal fatigue because the athlete wants to continue

Evidence record 11

Situation: HRV or wearable data is missing

Human interpretation: The estimate is less certain, not automatically worse

Default engine action: Use the remaining reliable inputs; lower confidence or hold progression

What the engine must not do: Treat missing data as normal or fabricate a value

Evidence record 12

Situation: Device data conflicts with user report or completed work

Human interpretation: There is a data-quality conflict

Default engine action: Prefer transparent conflict handling and conservative action; request one useful clarification

What the engine must not do: Average incompatible signals into a false precise score

Evidence record 13

Situation: The athlete asks to push through a blocked state

Human interpretation: Motivation is not a safety override

Default engine action: Offer a safe alternative or recovery action

What the engine must not do: Override a hard gate because the athlete insists

The exact reduction amount, number of confirming exposures, and trend window must remain versioned heuristics unless directly validated. The policy is about the **decision shape**, not a claim that one percentage or count is scientifically correct.

The minimum viable session rule

For a difficult but non-dangerous day, the engine should preserve the smallest dose that still serves the session's purpose. This may mean:

- one primary strength movement instead of the full accessory menu;
- fewer sets while retaining the target movement pattern;
- an easier aerobic session instead of intervals;
- technique work, mobility, or recovery conditioning instead of high intensity;
- a short check-in and warm-up that permits the athlete to stop honestly if capacity does not improve.

This is a practical adherence and fatigue-management policy, not proof that a particular “minimum dose” produces a defined adaptation. The engine must show which part of the original purpose was preserved and which part was deferred.

The no-punishment and no-make-up rules

Missing a session is an event to interpret, not a failure to punish. The default behaviour is:

1. Record the missed session and the known reason, if provided.
2. Do not add the missed volume to the next session.
3. Preserve recovery spacing and the order of important training exposures.
4. Resume with the next useful session, using a calibration state if the gap is long enough to make current capacity uncertain.
5. Explain: “You are not behind. We are protecting the next productive exposure.”

This exact policy is a product convention. The evidence bundle found no universal validated missed-session threshold or make-up rule.

OODA reset points

The engine should use a small decision loop at three points:

Observe	→	record symptoms, context, performance and data quality
Orient	→	interpret them against today's purpose and recent comparable history
Decide	→	choose proceed, hold, reduce, substitute, recover or block
Act	→	execute, monitor, and re-evaluate when new information appears

Run the loop:

- before the session;

- during the session after unexpected pain, technique loss, output collapse, abnormal symptoms, or major RPE mismatch;
- after the session before updating progression.

An OODA reset may change the rest of the session, but it must not silently rewrite the historical record or retroactively pretend that incomplete work was completed.

Required decision contract

Every human-logic decision should expose enough information for a user, coach, test, or audit to understand it:

```
type HumanLogicDecision = {
  status:
    | "approved"
    | "held"
    | "reduced"
    | "repeated"
    | "substituted"
    | "recovery"
    | "calibration"
    | "blocked"
    | "needs_input"
    | "needs_clinical_review";
  mode:
    | "normal"
    | "conservative"
    | "minimum_viable"
    | "recovery"
    | "calibration"
    | "blocked";
  sessionPurpose: string;
  action: string;
  prescription: unknown;
  reasonCodes: string[];
  observedSignals: string[];
  estimatedState: string[];
  conflicts: string[];
  missingData: string[];
  confidenceState: "high" | "moderate" | "low" | "insufficient";
  evidenceStatus: "coaching_precedent" | "product_design_convention" |
    "safety_policy";
  recheckAt: "next_set" | "mid_session" | "next_session" | "next_block";
};
```

The `prescription` may be typed more precisely in the implementation. It must never be an opaque free-form paragraph. Identical versioned inputs must produce identical decisions.

Human-readable explanation template

Use deterministic templates before adding AI:

```
What I noticed: [observed facts].
What that means today: [bounded interpretation and confidence].
What we are doing: [specific action and preserved session purpose].
Why: [reason codes in plain language].
What would change it: [next check or missing information].
```

Preferred voice: calm, direct, practical, experienced, and non-punitive. Avoid diagnosis, motivational theatre, false certainty, and phrases such as “your nervous system is fried” unless the product has an explicitly supported definition—which it currently does not.

Required regression cases

Before calling this layer complete, replay at minimum:

- normal readiness / normal performance;
- one poor night of sleep but normal performance;
- low readiness plus poor performance;
- good readiness plus poor performance;
- low motivation with no safety concern;
- a time-limited session;
- one missed session;
- a longer training gap;
- non-painful local fatigue;
- movement-specific pain;
- chest pain or dizziness red flag;
- missing HRV;
- missing heart rate during short intervals;
- conflicting wearable, machine, and self-report data;
- a user asking to make up missed work;
- a user asking to override a blocked state.

The acceptance criterion is not that the engine always finds a way to train. It is that it makes a safe, proportionate, explainable, context-aware decision and does not invent certainty where the evidence is weak.

Preserved file: [hybrid-engine-evidence-audit-mechanisms-7-8.md](#)

THE Hybrid Engine — research audit for mechanisms 7-8

Research date: 1 August 2026

Scope: mechanism 7, confidence-scoring and uncertainty communication in adaptive systems; mechanism 8, MacroFactor’s publicly documented expenditure-estimation and coaching behavior.

Executive judgment

Mechanism 7 is scientifically defensible as a **measurement and communication discipline**: define what uncertainty means, calibrate it against observed outcomes, expose data coverage and horizon, communicate ranges or frequencies when appropriate, and attach uncertainty to a useful action such as review, clarification, or no automatic update. The literature does **not** validate a generic confidence meter, any particular color/label/threshold, or the proposition that showing confidence automatically improves decisions, trust, adherence, or outcomes.

MacroFactor is a credible **product precedent** for a feedback loop that combines logged intake with a smoothed weight trend, gates updates when data are insufficient, and makes gradual weekly changes. Its public material is not independent scientific validation of true total daily energy expenditure (TDEE), nor does it publicly document a calibrated numerical uncertainty interval for each estimate. Its accuracy claims are primarily first-party analyses of predicted versus observed weight change.

The clean evidence-bundle wording is therefore:

> “Mechanism 7 follows evidence-informed principles for calibrated, action-linked uncertainty communication. Mechanism 8 has product precedent in MacroFactor’s documented intake/weight feedback, smoothing, data sufficiency, and weekly-adjustment workflow. Neither establishes that a particular confidence UI or expenditure algorithm is clinically or scientifically validated for THE Hybrid Engine.”

Evidence labels used here

- **Scientific evidence:** peer-reviewed or conference research that directly studies the relevant phenomenon, usually in a different task or population.
- **Product precedent:** a product creator’s public documentation of what the product does; useful for feasibility and design precedent, not proof of efficacy.
- **Design inference:** a reasonable translation from adjacent evidence that still requires product-specific testing.
- **Unknown/proprietary:** not specified in the public sources reviewed; absence of documentation is not proof that the feature does not exist.

Mechanism 7 — confidence-scoring and uncertainty communication

First, separate the concepts

“Confidence” can mean at least four different things:

1. **Predictive uncertainty:** how much the outcome may vary even with the same inputs, versus uncertainty caused by limited data or model knowledge. The aleatoric/epistemic distinction is useful, but it is not a complete user interface specification ([Hüllermeier & Waegeman, 2021](<https://link.springer.com/article/10.1007/s10994-021-05946-3>)).
2. **Calibration:** whether stated probabilities or intervals match observed frequencies or error coverage. A model that says “80%” should be right about 80% of the time in the relevant operating conditions; a high score alone is not evidence of calibration ([Guo et al., 2017](<https://proceedings.mlr.press/v70/guo17a.html>); [Arrieta-Ibarra et al., 2022](<https://jmlr.org/papers/v23/22-0658.html>)).
3. **Data/model reliability:** whether the current estimate is based on enough, recent, representative, and internally consistent observations.
4. **User trust:** whether a person is willing to rely on the system. Trust is a human response, not a readout of model accuracy. The safety target is **appropriate reliance**—avoiding both over-reliance and under-reliance—not maximum trust ([Lee & See, 2004](https://journals.sagepub.com/doi/10.1518/hfes.46.1.50_30392); [Bhatt et al., 2021](<https://dl.acm.org/doi/10.1145/3461702.3462571>)).

For a personalized adaptive system, “confidence 72%” is therefore incomplete unless it says: confidence in **what outcome**, over **what horizon**, conditional on **which data**, and with **what consequence** if the estimate is wrong.

Evidence-based communication principles

Principle	What the evidence supports	Audit status and boundary
Name the target, horizon, denominator, and base rate	Frequency formats can improve Bayesian reasoning, and natural frequencies are often easier to understand than abstract probabilities; verbal terms such as “likely” are interpreted variably and improve when paired with numbers (Gigerenzer & Hoffrage, 1995; Akl et al., Cochrane; Padilla et al., 2021).	Evidence-based communication principle. It does not establish a single best wording or prove that a frequency display is always preferable.
Prefer a range/distribution when the decision depends on plausible outcomes	In a controlled transit-choice task, uncertainty displays such as quantile dotplots/CDFs improved decisions relative to hiding uncertainty; HCI work also finds that displays can increase cognitive load or confuse users depending on task and context (Fernandes et al., 2018; Greis et al., 2017).	Moderate, task-dependent evidence. A nutrition app should test whether a range improves action selection, not assume that more visual uncertainty is better.
Expose data coverage and freshness	Clinical trust studies repeatedly identify understandability, perceived accuracy/competence, evidence, and actionability as trust conditions (Schwartz et al., 2022). Showing “based on 4 of 7 nutrition days and 1 weigh-in” is a design translation of this principle.	Evidence-informed design inference. The exact completeness threshold and wording are product choices unless validated on the product’s outcomes.
Link uncertainty to an action	In clinical decision support, uncertainty has been used to target cases for review; one sleep-medicine implementation reported improved agreement and reduced review time when uncertainty guided review prioritization (Kang et al., 2021).	Useful precedent, not general proof. The action may be “ask for more data,” “hold the update,” “show a conservative range,” or “invite human review”; the best action depends on risk and cost.
Make verification easier, not merely the explanation longer	Explanations can reduce over-reliance when they lower the cost of checking an AI recommendation, but effects depend on task difficulty, explanation difficulty, and incentives (Vasconcelos et al., 2023). In clinical decision support, fuller explanations increased trust but could also increase over-reliance (Bussone et al., 2015).	Conditional evidence. “Why” text is not automatically a safety feature; it should expose evidence, assumptions, and what would change the result.

Optimize appropriate reliance, not trust or satisfaction	Trust-in-automation research frames the goal as appropriate reliance. Automation bias can occur when users accept recommendations too readily, especially when verification is complex (Lee & See, 2004; Lyell & Coiera, 2017).	Strong safety principle, but the intervention still needs testing. A visible low-confidence signal may create under-reliance or algorithm avoidance as well as reduce over-reliance.
Use active verification selectively	Cognitive-forcing interventions reduced over-reliance in a controlled AI-decision study, but worsened subjective ratings and affected people differently (Bućinca et al., 2021).	Conditional design pattern. Friction is defensible for high-consequence or low-confidence decisions, but not automatically appropriate for every low-stakes coaching update.
Treat adaptive-system confidence as time-varying	A changing personal state, changing behavior, missing logs, and model updates make “confidence” a temporal claim. Showing the evidence window, last update, and what changed is a reasonable extension of calibration and trust principles.	Design inference, not direct validation. The literature does not give THE Hybrid Engine a validated update cadence, lookback window, or threshold.

What is only product design

The following may be sensible implementation choices, but they are not established by the literature above:

- a circular confidence meter, badge, star rating, or “high/medium/low” label;
- green/amber/red colors or a particular textual threshold such as “review below 60%”;
- mapping missing days, trend volatility, or model disagreement to one scalar score;
- a specific smoothing constant, minimum sample size, lookback window, decay rate, or “stable” rule;
- automatically changing a recommendation when confidence crosses a threshold;
- deciding how much explanation to reveal by default, or whether a tooltip improves adherence;
- a claim that showing confidence increases trust, motivation, weight loss, safety, or long-term retention.

Those claims require product-specific usability, calibration, prospective decision-quality, and—if health outcomes are claimed—outcome research. A model can be calibrated while users misunderstand it; users can trust a model that is not calibrated; and an accurate estimate can still produce a poor recommendation if the action policy is wrong.

Counterpoints and evidence gaps for mechanism 7

Uncertainty is not cost-free. In a breast-health decision experiment, communicating uncertainty was associated with lower decision satisfaction among cancer patients, although the authors discuss this as a possible tradeoff rather than an argument for hiding uncertainty ([Politi et al., 2011](<https://pubmed.ncbi.nlm.nih.gov/20860780/>)). Explanations can increase acceptance without improving complementary team performance, and confidence scores can improve trust calibration without improving the assisted decision itself ([Bansal et al., 2021](<https://dl.acm.org/doi/10.1145/3411764.3445717>); [Zhang et al., 2020](<https://dl.acm.org/doi/10.1145/3351095.3372852>)).

The largest transfer gap is ecological: much of the evidence uses classification, diagnosis, forecasting, or short laboratory tasks. There is limited evidence that a continuously updated, personal energy-balance estimate with a confidence display improves nutrition logging, adherence, health outcomes, or user calibration over months. Clinical studies often have small samples or hypothetical cases, and visualization results are highly task-dependent. The evidence supports disciplined uncertainty communication; it does not validate a particular nutrition-product UX.

Mechanism 8 — what MacroFactor publicly documents

The points below describe first-party public documentation available on the research date. They should be read as **what MacroFactor says it does**, not as independent confirmation that the method measures true TDEE.

Topic	Publicly documented claim	What it establishes—and what it does not
Initial versus personalized estimate	The help material says the initial estimate is formula-based and becomes personalized from observed intake and weight data (Why is my expenditure different from a TDEE calculator?; initial expenditure guidance).	Product workflow precedent. The initial formula and later feedback loop should not be presented as the same evidence source.
Smoothing / trend weight	Weight Trend is described as a moving average that gives greater emphasis to recent weigh-ins; decisions use the trend rather than the raw scale value. MacroFactor also describes its energy-balance widget as looking back over roughly three weeks (Weight Trend; Energy Balance widget).	Strongly documented smoothing and noise-management behavior. The exact filter, weights, outlier handling, and responsiveness parameters are not public.
Minimum data requirements	For V3, the public version article lists nutrition on at least 4 of 7 days and weight at least once per week as minimums; it recommends daily nutrition and more frequent weighing. Older V1/V2 minimums are listed separately (Expenditure version).	A product data gate, not a universal scientific sufficiency threshold. “Minimum to update” is not the same as “enough for a precise estimate.”
Missing data and holding	MacroFactor says insufficient nutrition causes expenditure to hold until logging is consistent; missing weight can be linearly interpolated, while missing nutrition is less accurately estimated. Its V3 article says the algorithm can estimate some unlogged intake and pauses when more than three days in a seven-day period are missing (nutrition logging frequency; V3 article).	A practical missing-data policy. The public documents do not independently validate the imputation error or establish that the same policy is optimal for all users.
Weekly adjustments	MacroFactor says expenditure is	Product precedent for delayed, damped

	estimated continuously from intake and trended weight, then the next check-in adjusts the target. It describes additional smoothing logic so a one-week stall does not immediately produce a large cut; after roughly 3–4 weeks of consistent data it is more confident that expenditure changed (weight-loss/gain adjustments).	intervention. “Weekly adjustment” should not be described as a weekly-only expenditure estimator.
Wearables and activity data	A help article says wearable energy-expenditure data are not used. A later optional modifier says step counts can be incorporated if the user enables Step-Informed Updates; it explicitly distinguishes step counts from wearable calorie estimates (wearables; Expenditure Modifiers).	Current public nuance: no imported wearable-calorie estimate, but optional step-count information may affect updates. The exact contribution and defaults are not fully specified in the public material.
V3 creator-reported behavior	MacroFactor’s V3 article says updates are generally smaller and durable trends may be detected sooner; it also says the exact “secret sauce” is not disclosed. The article reports internal missing-intake tests and comparison claims (Expenditure V3).	Evidence of stated product intent and internal testing. It is not an independently reproduced algorithm specification or validation study.
Current optional modifiers	The November 2025 modifier article describes Step-Informed Updates and Predictive Goal Adjustment, including internal estimates of short- and longer-term error improvements (Expenditure Modifiers).	Product evolution. The reported percentages are creator-reported analyses; they do not establish causal improvement for the general user population.

What is proprietary or unknown

The public V3 article explicitly says the novel technique is not fully disclosed. The reviewed public material does not specify, in a reproducible way:

- the exact V3 equations, priors, coefficients, state variables, or update weights;
- the precise smoothing kernel, outlier rules, decay rates, and responsiveness/stability tradeoff;
- how body-composition changes, illness, menstrual-cycle effects, unusual fluid shifts, logging bias, or food-database error are separated from expenditure;
- whether each expenditure estimate has a numerical prediction interval, coverage target, or calibrated confidence score;
- how data completeness, volatility, or step counts map to any user-facing confidence statement;
- the complete current default behavior across app versions, goals, and optional modifiers;
- an independently reproducible dataset, codebase, preregistered protocol, or external replication.

It is therefore unsafe to infer that MacroFactor implements mechanism 7 merely because it smooths weight, holds updates, or uses language such as “more confident” after several weeks. Those are uncertainty-aware

product behaviors, but public documentation does not show a calibrated uncertainty communication system in the technical sense.

Product precedent versus scientific validation

MacroFactor's own accuracy article is unusually clear about the distinction: it describes metabolic-chamber measurement as the gold-standard route but says its analysis prioritizes predictive validity. The reported evaluation predicts weight change from estimated expenditure and intake, then compares that prediction with observed weight change; it does not directly measure expenditure in a chamber or with doubly labeled water ([Algorithm Accuracy](https://macrofactor.com/algorithm-accuracy/)).

The page reports an internal cohort of 748 challenge participants after exclusions and gives median prediction-error comparisons after several weeks. Those numbers can support the narrower statement that MacroFactor reports useful predictive performance in its own data. They cannot support the stronger statements "measures true TDEE," "scientifically validated," or "generalizes to all users." The analysis is first-party, the cohort is product-selected, partial logging is excluded by a heuristic, and the raw data/code and independent replication are not provided in the cited material.

A 2024 peer-reviewed Journal of Sports Sciences paper used MacroFactor for nutrition/weight tracking in a resistance-training study ([Refalo et al., 2024](https://www.tandfonline.com/doi/full/10.1080/02640414.2024.2321021)). That is evidence that the app can be used as a tracking tool in a research protocol; it is **not** validation of MacroFactor's expenditure estimator.

Audit verdict

Mechanism	Evidence-bundle status	Safe claim
8. MacroFactor	Strong product precedent; weak/unclear independent scientific validation of expenditure measurement.	Cite the documented intake/weight feedback loop, trend smoothing, data gates, missing-data holding, and damped weekly updates as precedent. Label accuracy numbers as first-party predictive analyses.

Evidence gaps that should remain explicit in the bundle

1. No cited study establishes that a confidence score in a consumer nutrition app improves weight outcomes, adherence, or user calibration over time.
2. No cited independent study validates MacroFactor's current expenditure algorithm against a gold-standard expenditure measure.
3. The public MacroFactor materials do not provide enough algorithm detail to reproduce V3 or its current optional modifiers.
4. Product minimums such as 4/7 nutrition days and 1 weigh-in/week are operational rules, not universal physiological or statistical guarantees.

5. Predictive validity from weight change is not the same as measurement validity for true energy expenditure.
6. The most important tests for THE Hybrid Engine remain product-specific: calibration by user subgroup and horizon, robustness to missing/biased logs, over- and under-reliance, whether uncertainty causes helpful review or harmful disengagement, and whether recommendations improve outcomes.

Preserved file: [design.md](#)

THE Hybrid Engine — Product and Technical Design

Status: living source of truth for the current Hybrid Engine build.

This document was drafted from the supplied THE-HYBRID-ENGINE1-main archive, its source files, build notes, changelog, schema, and the TrainHeroic-style builder screenshots. It describes what is currently implemented, what is intentionally parked, and what still needs a product decision.

1. How Claude must use this document

1. Read this document before changing code.
2. Treat the current implementation sections as facts about the supplied build.
Treat the open decisions section as unresolved; do not silently choose a direction that changes the product.
3. Inspect the existing source before replacing or restructuring it. The app is already a working local-first PWA with integrations and a separate coach surface. Do not reduce it to a visual mockup.
4. Preserve working behaviour unless a change is explicitly requested.
5. Keep the athlete app and coach app as separate entities with one explicit, tested data boundary between them.
6. Never put workout logging/result fields into a coach-authored prescription.
The coach writes targets; the athlete logger writes actual results.
7. After every material change, run the relevant checks and record the result in the changelog or implementation notes.
8. Do not copy TrainHeroic branding, private assets, or proprietary code. The screenshots are a reference for layout and interaction quality.

2. Product definition

THE Hybrid Engine is a hybrid strength-and-conditioning training system. It has two connected products:

- Athlete app: a phone-friendly training logger that presents today's work, records completed sets and conditioning sessions, adapts conditioning to recovery, and shows history/progress.
- Coach app: a TrainHeroic-inspired authoring tool that organises programs into weeks, days, and sessions, prescribes exercises set by set, and publishes a session to an athlete's calendar.

The app is designed around a short loop:

~~~text

Plan → Schedule → Train → Log → Recover → Review → Progress

~~~

The major product principles are:

- Local-first and usable offline.
- Fast, low-friction interaction on the gym floor.
- A calm, premium, instrument-like visual language rather than a noisy social fitness feed.
- Strength and conditioning in one system, including hybrid sessions.
- Recovery information should guide decisions without overwhelming the user.
- Prescription data and logged results must remain separate.

3. System boundary

~~~mermaid

flowchart LR

Coach["Coach app /coach"] -->|Target-only session snapshot| Bridge["HybridEmit boundary"]

Bridge --> Supabase["Supabase assignments"]

Supabase --> Athlete["Athlete PWA"]

Athlete --> Local["Local-first state"]

WHOOP["WHOOP"] -->|Recovery/API or HR broadcast| Athlete

~~~


Entity 1 — Athlete PWA

Files:

- index.html — app shell, markup, CSS, screens, and navigation slots.
- app.js — local-first engine, rendering, logging, conditioning, import, sync,

WHOOOP wiring, and interactions.

- service-worker.js — static shell caching and update handling.
- manifest.json and icons/ — install surface.

The athlete application runs locally without an account. Cloud sync and WHOOOP are optional enhancements.

Entity 2 — Coach app

Files:

- coach/index.html — coach builder shell and TrainHeroic-style layout.
- coach/js/app.js — coach library model, editor rendering, local persistence, prescriptions, supersets, auth, cloud library sync, and assignment publishing.
- coach/js/emit.js — the only supported conversion boundary from coach data to athlete-phone workout data.
- coach/js/config.js — coach Supabase client configuration.
- coach/coach-builder-trainheroic.html — standalone visual prototype/reference;

not the primary coach runtime.

Boundary rule

HybridEmit converts a coach session into the athlete phone's workout shape. It preserves supported targets such as reps, time, RPE, and supersets, but never writes athlete-owned result fields such as aVal, aVal2, felt, done, or note into a target prescription.

4. Repository map

Area	Responsibility
app.js	Athlete state, UI, logger, calendar, library, import, conditioning, progress, sync
hybrid-engine-design-mock.html	Original athlete design source
coach/index.html	Coach builder UI shell

coach/js/app.js	Coach authoring and publishing logic
coach/js/emit.js	Coach-to-athlete contract
supabase-schema.sql	Athlete sync plus coach library, links, programs, assignments, and RLS
netlify/functions/	Server-side WHOOP OAuth, sync, webhook, token storage, and integration status
integrations/whoop-adapter.js	Browser-side normalized WHOOP contract
native/android-app/	Android WebView shell with native HR, wake-lock, and file bridges
native/windows/	Tauri Windows wrapper
checks/	PWA, browser, coach, emit, WHOOP, deployment, security, and torture checks

5. Users and roles

Athlete

The athlete uses the phone app to:

- See today's planned session.
- Start or resume a session.
- Log sets, actual weight, reps, seconds, RPE, completion, and notes.
- Use the rest timer.
- Swap an exercise during a session while retaining targets and logged work.
- Run standalone or hybrid conditioning sessions.
- Review recap, history, exercise history, PRs, and progress.
- Connect WHOOP or use simulated conditioning data.

Coach

The coach uses the coach app to:

- Create and organise programs.
- Navigate weeks and seven day slots.
- Author sessions.
- Add exercises and set prescriptions.
- Change prescription measurement columns.
- Add cues, suggested swaps, and points of performance.
- Link adjacent exercises into supersets.
- Publish a session to a date on a phone calendar.

Self-coached user

The same Supabase account can act as both coach and athlete. The coach app's Assign to phone flow supports this directly.

The database also contains the groundwork for a coach-to-athlete relationship, token-gated invites, and multiple athletes. The visible coach UI is not yet a complete multi-athlete management product.

6. Athlete navigation and screens

The primary athlete bottom navigation is:

1. Home
2. Training
3. Library
4. Settings

The logger is a detail view of Training, not a separate primary tab.

Home

Home is designed to be read at a glance. It contains:

- Welcome/header area.
- Sunday-first week strip.
- Today's session or honest rest-day state.
- Start today's session or Resume action.
- WHOOP recovery/strain rings.
- Readiness advice based on recovery and planned-versus-felt RPE.
- Today's heart-rate-zone card.
- Weekly zone targets.
- Small summary/stat cards.
- Quick actions for creating or adding a session.

The week strip can open History for a selected day. Planned days and trained days are visually distinguished.

Training

Training is the session map and live workout surface. It shows:

- Session header and recovery interpretation.
- Progress through the session.
- Blocks and exercises.
- Superset groups.
- Prescription summaries.

- One expanded exercise logger at a time.
- A final Mark session complete action.

Tapping an exercise opens its set-by-set table in place. Previous/Next controls allow guided movement through the session.

Logger

The logger is intentionally compact and gym-focused.

Each exercise row shows:

- Letter marker: A, B, or C1/C2/C3 for supersets.
- Exercise name.
- Prescription summary.
- Current completion state.

The expanded table uses:

~~~text

SET · TARGET · KG · REPS · RPE · ✓

~~~

Depending on the exercise mode, only relevant fields are displayed. Ticking a set can autofill blanks from the previous session, starts the programmed rest timer, and advances naturally through a superset.

The logger owns actual-result fields. It must not mutate the coach target model.

Library

Library is the central shelf for saved work. It has three internal areas:

- Sessions — saved strength/hybrid session templates.
- Conditioning — standalone conditioning formats and recent sessions.
- Progress — strength, RPE, recovery, zone, and conditioning trends.

The Sessions area contains:

- Search.
- Create Session Template.
- Saved-session rows/cards.
- Exercise summaries.
- Add/schedule action.
- Three-dot menu with edit, duplicate, and delete.

Home's plus action can either create a new strength/conditioning session or add a saved session from the Library to a chosen date.

Calendar and scheduling

The app supports recurring weekday scheduling and one-off date scheduling.

- A session can be added to today or a future date.
- Long-pressing a session opens Move, Delete, or Cancel.
- Move uses a date picker and must move the session rather than duplicate it.
- Delete uses confirmation and records deletion tombstones for sync safety.
- A coach-assigned session is removed from its underlying assignment when deleted or rescheduled.

History and recap

History shows completed and incomplete sessions by day, including logged sets, per-set notes, conditioning results, and session status.

Recap after completion shows:

- Volume.
- Sets completed.
- Planned-versus-felt RPE verdict.
- Heart-rate zones for hybrid sessions.
- Records/PRs.
- Links to exercise history.

Exercise history and progress

Exercise history is reachable from recap, History rows, the logger history link, and Progress. It contains best sets, estimated 1RM trends, and past sessions.

Progress contains:

- Session count.
- Weekly volume.
- Day streak.
- Training-volume chart.
- Planned-versus-felt RPE chart.
- WHOOP recovery chart.
- Conditioning zone bars.
- Interval progression cards.
- Top lifts.

Charts are inline SVG and should remain dependency-free unless explicitly changed.

Conditioning

Conditioning supports:

- Steady-state Zone 2.
- Intervals.

- Tempo.
- Custom format.
- Free run.
- Simulated HR demo.
- WHOOP HR Broadcast through Web Bluetooth on supported browsers.
- Native Android HR bridge through window.AndroidHR.

The screen shows live BPM, current zone, phase/round timer, zone-time banking, zone-coloured HR trace, and vibration/audio cues where supported.

Results include duration, average/max HR, zone-time breakdown, HR recovery, and estimated calories. Real sessions affect conditioning progression; demo sessions do not.

Import

The importer accepts:

- Written/pasted workout text.
- Photo/screenshot OCR using bundled Tesseract or native Android OCR.
- Voice dictation using browser speech or the native Android bridge.

The parser creates a workout template, asks inline questions only where genuine ambiguity exists, learns approved shorthand into a synced lexicon, and opens the result in the Builder.

Settings

Settings contains:

- Local/cloud sync status.
- Sign in, sign out, password reset, and Sync now.
- WHOOP connect, sync, and disconnect.
- Training profile including age, resting HR, and observed max HR.
- Conditioning zone targets.
- Gym setup: bar weight and owned plates.
- Export backup.
- Import/restore backup with confirmation.
- Reset local data.

7. Coach navigation and builder

The coach app is a desktop-oriented authoring surface with a responsive layout.

Coach shell

- Dark navy top bar.
- Program selector showing the active program name.

- Messages, notifications, avatar, and sync status controls.
- Dark icon rail containing Coach Home, Athletes, Teams, Library, Analytics, Gym

Tools, and Support.

- Light workout-navigation panel.
- Warm off-white editor workspace.

The current visible implementation makes Library the functional rail item. The other rail items are visual placeholders until their products are built.

Week/day navigation

The left workout panel shows:

- Current week label.
- Seven numbered day buttons.
- Rest-day state for empty days.
- Session Preview.
- Session title.
- Exercise letters and prescription summaries.

The top program menu can:

- Switch programs.
- Create a new program.
- Rename the current program.

The week menu can:

- Switch weeks.
- Add a week.

Coach editor

For a populated day, the editor shows:

- Week/day heading.
- Assign to phone.
- Save.
- Delete session.
- Editable session title.
- Coach Instructions field with character count.
- Section label such as Strength/Power.
- Section selector.
- Trophy/control area and section menu.

Each exercise card contains:

- Letter marker.
- Exercise selector.
- Set count.
- Remove exercise control.

- Exercise Instructions field.
- Video thumbnail/play affordance.
- Edit Swaps link.
- Suggested Swaps text.
- Collapsible Points of Performance list.
- Prescription summary.
- Save Prescription button.
- Editable set-by-set prescription table.
- Add set and Remove set controls.

Adjacent exercises can be linked with a chain control. Linked exercises are labelled A1/A2, B1/B2, and so on in the preview and are transmitted as a superset block to the phone.

Prescription table

The table has a set-number column and configurable measurement columns. Current measure options are:

- Reps
- Weight (lb)
- Weight (kg)
- Weight (%)
- Weight (LWP+)
- RPE
- Time (min:sec)
- Distance (miles)
- Distance (yd)
- Distance (ft)
- Distance (inches)
- Distance (meters)
- Height (inches)
- Calories (cal)

The dropdown is white, scrollable, and opens over the table. The screenshots show the same dropdown at different scroll positions.

Coach assignment

Assign to phone:

1. Coach opens a populated day.
2. Coach selects Assign to phone.
3. Coach selects a calendar date.
4. The session is converted through HybridEmit.
5. The target-only snapshot is inserted into assignments.

6. The athlete app reconciles assignments during sync/foreground.

7. The phone materializes the assignment as a calendar workout.

The coach app requires the same account or an active coach-athlete relationship.

8. Data model

Athlete local state

Local storage key: hybrid-engine-v1.

```
~~~js
```

```
{
```

```
workouts: [
```

```
{
```

```
id,
```

```
name,
```

```
days: [0..6],
```

```
dates: ["YYYY-MM-DD"],
```

```
blocks: [
```

```
{
```

```
id,
```

```
heading,
```

```
minutes,
```

```
format,
```

```
superset,
```

```
exercises: [
```

```
{
```

```
id,
```

```
name,
```

```
mode,
```

```
tempo,
```

```
rest,
```

```
sets: [{ t, rpe }]
```

```
}
```

```
]
```

```

    }
  ]
}
],
sessions: [
{
  id,
  workoutId,
  name,
  date: "YYYY-MM-DD",
  status: "active" | "completed" | "incomplete",
  startedAt,
  completedAt,
  blocks: [
    // workout snapshot plus athlete-owned set fields:
    // aVal, aVal2, felt, done, note
  ]
}
],
settings: {}
}
~~~

```

Supported athlete strength modes:

- reps_kg — Reps + Kilos.
- amrap — Max reps.
- seconds — Seconds.
- reps_seconds — Reps + Seconds.
- reps — Reps only.
- completion — For completion.

Conditioning blocks use kind: conditioning, condFmt, targetZone, and minutes, and intentionally have no exercises.

Coach local state

Local storage key: hybrid-coach-v1.

```
~~~js
{
  programs: [
    {
      id,
      name,
      weeks: [
        {
          days: [sessionOrNull, sessionOrNull, sessionOrNull,
            sessionOrNull, sessionOrNull, sessionOrNull,
            sessionOrNull]
        }
      ]
    }
  ],
  sel: { p, w, d }
}
```

```
~~~
~~~js
session = {
  title,
  note,
  section,
  exercises: [
    {
      id,
      name,
      cols: ["Reps", "Weight (lb)"],
      sets: [["5", "155"], ["5", "155"]],
      cues,
      swaps,
      pop: ["Point of performance"],
    }
  ]
}
```

link: false

}

]

}

~~~

## Cloud state

Athlete state is stored as one JSON object in `app_state`, protected by row-level security. The coach authoring library is stored as one JSON object in `coach_library`.

Coach-athlete relationships use token-gated `coach_athletes` rows. Programs and cross-account handoff use programs and assignments. An assignment stores a self-contained phone-shaped session snapshot so the athlete does not depend on the coach UI remaining open.

## Ownership rule

~~~text

Coach target: `t`, `rpe`, `tempo`, `rest`, `prescription structure`

Athlete result: `aVal`, `aVal2`, `felt`, `done`, `note`, `completedAt`

~~~

This separation is a hard contract.

## 9. Sync and reliability rules

- Local storage is always the first write path.
- Cloud sync is optional and debounced.
- Workouts and sessions merge record-by-record by ID.
- Scheduled days are unioned.
- Logged history is retained.
- Deletions use tombstones so old devices cannot resurrect records.
- Settings merge additively where required, including conditioning history, progression, and import lexicon.
- WHOOP daily cache and device registry remain device-local where specified.
- Coach-originated phone workouts are re-derived from assignments and must not be pushed back as user-owned coach templates.
- Incoming cloud/backup data is sanitized before installation.
- Storage-full errors must be visible to the user.

- Active sessions must survive reload and stale sessions must be filed as incomplete or dropped if empty.

## 10. WHOOP and conditioning rules

WHOOP OAuth, token exchange, refresh, webhook verification, encryption, and provider API access stay server-side in Netlify Functions.

The browser receives normalized fields only:

- source
- date
- recoveryScore
- sleepPerformance
- hrvMs
- restingHr
- strain
- capturedAt

Heart-rate conditioning uses:

- Tanaka max-HR estimate:  $208 - 0.7 \times \text{age}$ .
- Heart-Rate Reserve when resting HR is available.
- Percentage-of-max fallback otherwise.
- Recovery, Conditioning, and Overload bands.
- WHOOP recovery-based daily zone adjustment.
- Autoregulated progression for canonical interval formats.

The user can run a simulated-HR demo without a band. Demo sessions must never change earned progression.

## 11. Visual design language

### Athlete app

- Dark navy/charcoal shell.
- Warm matte surfaces and recessed input wells.
- Brushed brass/gold brand accent.
- Gold represents completed/done states.
- Green is reserved for heart-rate zones.
- Calm typography with tabular figures for live numeric columns.
- One open logger card at a time.
- Clear touch targets, haptic/vibration cues where supported, and reduced-motion support.

## Coach app

- Dark navy top bar and icon rail.
- Warm off-white editor surface.
- Light blue-grey exercise header bars.
- White prescription dropdown menus.
- Compact desktop density similar to the supplied screenshots.
- Thin borders, restrained shadows, small rounded corners, and clear hierarchy.

The browser chrome, laptop frame, glare, reflections, and Windows taskbar from the supplied photographs are not part of the product.

## 12. Current implementation gaps and placeholders

These are facts to preserve in the design backlog rather than silently treating them as complete:

- Coach rail destinations other than Library are currently visual/placeholder states.
- Coach Edit Swaps currently displays a coming-later toast.
- Coach video thumbnails are presentation placeholders, not a complete media library.
- The coach editor's + Add section action currently follows the same path as adding an exercise and needs a deliberate section/block design.
- The coach editor exposes more measurement types than the athlete logger can natively consume. Unsupported types currently fall back to the nearest phone mode while the coach snapshot retains the original source information where supported by the boundary.
- Full training-block/progression programming remains parked; weekly day-chip scheduling exists.
- Android and Windows wrappers exist, but wrapper changes are separate from the web application and should not be mixed into ordinary UI work.
- Browser-dependent smoke tests require Playwright. Static PWA, WHOOP, deployment, and contract checks are expected to remain green.

## 13. Acceptance criteria

### Athlete

- The app opens without an account and saves locally.



- A user can create a session template, edit it, duplicate it, schedule it, and delete it with confirmation.
- A user can add a session to a date and move it without duplicating it.
- A user can start, resume, log, and complete a strength session.
- Set completion starts the correct rest timer.
- The rest timer survives reload/screen lock where the platform supports it.
- Logger fields match the selected exercise mode.
- Per-set notes appear in History.
- A mid-session exercise swap retains targets and logged work.
- A hybrid session can interleave strength and conditioning.
- Conditioning results appear in History and Progress.
- Import from text, photo, and voice handles genuine ambiguity without silently inventing values.
- Cloud sync merges two devices without losing scheduled work or history.

## Coach

- A coach can create/select/rename programs.
- A coach can add weeks and navigate seven days per week.
- Empty days show a rest-day state and can receive a new session.
- A coach can edit the session title and instructions.
- A coach can add/remove exercises and sets.
- Each prescription column is selectable from the supported measurement menu.
- Set values are editable and update the live summary.
- Adjacent exercises can be linked/unlinked as supersets.
- A coach can save and delete a session with confirmation.
- Assignment publishes a target-only snapshot to the phone.
- The phone receives the assignment without receiving logger-owned result fields.

## Security and deployment

- No WHOOP client secret or session secret appears in browser files.
- WHOOP callbacks and webhooks remain server-side and authenticated.
- RLS prevents cross-user data access.
- Service worker never caches authenticated function routes.
- PWA, WHOOP, deployment, emit, and security checks remain green.

## 14. Open decisions requiring the product owner

Answer these before Claude performs a large redesign:

1. Is the Coach app now the main priority, or is the athlete app still the main product with the coach builder as a secondary surface?
2. Should the TrainHeroic screenshots define only the coach visual design, or

should their full measurement system also become the athlete logger's system?

3. Should the coach app start with the seeded sandbox program, or should new accounts start completely empty like the athlete Library?

4. What are the exact seven foundational movement patterns?

5. Is the target audience only you/self-coached, or must multi-coach/multi-athlete management be completed now?

6. Should coach-created sessions become reusable athlete templates, scheduled one-off sessions, or both?

7. What should + Add section create: a named block, a new training category, or another exercise group?

8. Should exercises have a real media library now, or are video placeholders acceptable for the next milestone?

9. Which coach metrics must survive into the phone logger: reps, weight, RPE, time, distance, calories, height, percentage, or all of them?

10. What is the required mobile behaviour for the coach builder? Responsive desktop layout, mobile editing, or desktop-only authoring?

11. What app name, logo, colours, and terminology are final?

12. Which current features are mandatory for the next release, and which should be hidden or parked?

## 15. Required working method for future changes

Before changing code:

1. Identify the affected entity: athlete, coach, bridge, backend, or native wrapper.
2. State the user flow being changed.
3. Update this document if the data model or behaviour changes.
4. Implement the smallest coherent change.
5. Test the affected flow in a real browser where possible.
6. Run the relevant static/contract checks.
7. Check mobile and desktop layout for UI changes.
8. Record the change and any remaining limitation.

The final goal is not merely a similar-looking screen. The goal is one coherent

training system in which the builder, schedule, logger, history, recovery data, cloud sync, and coach-to-athlete handoff all agree on the same data ownership rules.